



데이터베이스



1. 데이터베이스 기본

KPC 기본반

1. 데이터베이스 기본	3
데이터와 정보.....	4
데이터베이스(Database).....	6
데이터독립성과 3 단계 스키마	8
데이터베이스관리시스템(DBMS).....	11
데이터 언어(Data Language).....	15
데이터 무결성(Data Integrity)	17

데이터와 정보

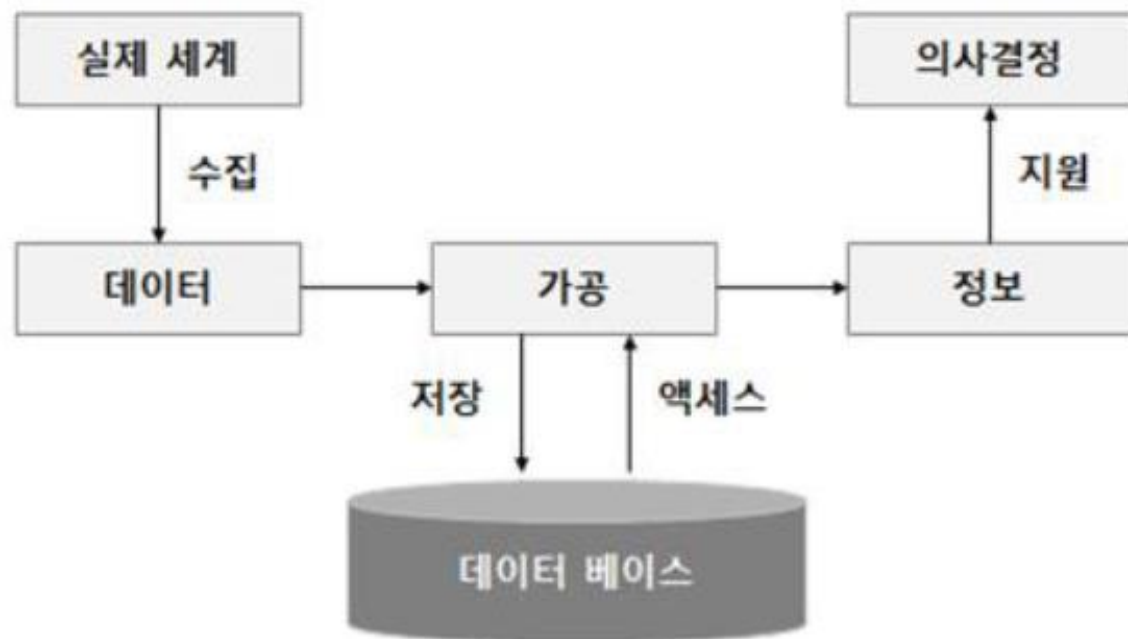
1. 데이터(Data)와 정보(Information) 개요

□ 데이터(Data)와 정보(Information) 정의

구분	설명
데이터(Data)	현실세계에서 관찰이나 측정을 통해 수집한 단순한 사실이나 값으로, 가공되지 않은 상태를 의미 숫자로 표현되는 수치는 물론 어떤 문자들로 표현되는 스트링, 텍스트, 이미지, 그래픽스도 포함됨
정보(Information)	의사결정에 도움을 줄 수 있는 유용한 형태로, 자료를 가공(처리)해서 얻을 수 있는 결과를 의미 어떤 상황에 대한 적절한 결정을 할 수 있게 하는 지식으로서 데이터의 유효한 해석이나 데이터 상호간의 관계를 의미

데이터와 정보

□ 데이터(Data)와 정보(Information)의 관계



데이터와 정보

2. 데이터와 연관시스템과 정보처리

□ 데이터와 연관시스템

구분	시스템유형	설명
데이터	OLTP	현실의 객체와 사실관계로부터 데이터를 수집하여 기록 및 열람하기 위한 시스템 유형
정보	OLAP, Mining	수집된 데이터에 Sum, Count, Grouping 등의 연산(가공)을 통해 의미를 부여함
지식	KMS, EKP	정보를 수집하는 과정 및 이를 업무에 활용하는 방법에 대해 체계적으로 정의, 평가, 보상하기 위한 시스템
지혜	BI	기업 내 정형 및 비정형 데이터/정보/지식으로부터 의사결정을 위한 가치 제공

데이터와 정보

□ 정보처리(information processing)

- 데이터에서 정보를 추출하는 과정 또는 방법

한빛 쇼핑몰 주문 내역

주문 번호	주문 일자	제품명	판매 금액
1	2013-01-10	냉장고	50만원
2	2013-02-12	세탁기	30만원
3	2013-03-03	세탁기	30만원
4	2013-04-05	에어컨	70만원
5	2013-05-15	에어컨	80만원
6	2013-06-19	에어컨	70만원
7	2013-07-07	에어컨	70만원
8	2013-08-12	냉장고	40만원
9	2013-10-11	청소기	10만원
10	2013-12-27	전자레인지	15만원

데이터

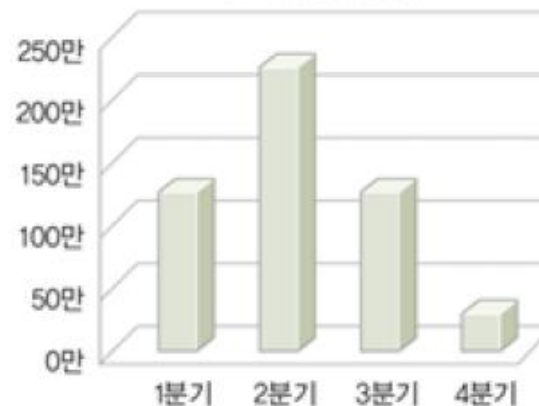
정보 처리

제품별 총 판매액

제품	총 판매액
에어컨	290만원
냉장고	90만원
세탁기	60만원
전자레인지	15만원
청소기	10만원

정보

분기별 총 판매액



데이터베이스

1. 데이터베이스(Database) 개요

□ 데이터베이스(Database) 정의

- 다수 사용자의 정보 요구사항을 충족시키기 위하여, 서로 관계 있는 데이터를 최소한의 중복으로 통합해 놓은 데이터의 집합체
- 특정 조직의 여러 사용자가 공유하여 사용할 수 있도록 통합해서 저장한 운영 데이터의 집합

□ 데이터베이스 정의가 함축하고 있는 의미



특징	설명
통합 데이터 (integrated data)	동일한 데이터가 원칙적으로 중복되어 있지 않다는 것을 의미 최소의 중복 또는 통제된 중복이 허용된 통합된 데이터
저장 데이터 (stored data)	컴퓨터가 접근할 수 있는 자기 디스크나 메모리 등 저장매체에 저장된 데이터
운영 데이터 (operational data)	단순한 입출력 데이터나 일시적으로 생성된 데이터가 아닌 한 조직의 고유 기능을 수행하기 위해 필요한 데이터의 집합
공유 데이터 (shared data)	조직에 있는 여러 사용자들이 응용 시스템을 통해 공동으로 소유, 유지, 이용하는 데이터

데이터베이스

2. 데이터베이스(Database)의 특성 및 장점

□ 데이터베이스(Database)의 특징

특징	설명
실시간 접근 (real-time accessibility)	수시적이고 비정형적인 질의(query)에 대하여 실시간 응답 사용자의 데이터 요구에 실시간으로 응답
계속적인 변화 (continuous evolution)	새로운 데이터의 삽입, 기존 데이터의 삭제, 갱신으로 항상 그 내용이 변할 뿐만 아니라 이러한 변화 속에 항상 현재의 정확한 데이터 유지
동시 공유 (concurrent sharing)	여러 사용자가 동시에 원하는 데이터에 접근하여 이용할 수 있어야 하고 동일한 내용의 데이터를 여러 사람이 동시에 공용할 수 있도록 지원
내용기반 참조 (contents reference)	데이터 레코드들의 주소나 위치에 의해서가 아니라 사용자가 요구하는 데이터의 내용, 즉 값(Value)에 따라 참조

데이터베이스

□ 데이터베이스(Database)의 장점

장점	설명
중복성 감소	개별적 파일이 갖는 중복성을 데이터베이스를 통해 감소시켜 기억장소 감소와 전반적 통제권강화의 효과를 가져올 수 있음
데이터 공유	응용 프로그램이 처리하기 위한 데이터가 서로 공유되어 제공받을 수 있는 환경 제공
표준화	데이터베이스를 중앙에 통제함으로써 데이터를 표현하는 데 모든 적용 가능한 표준을 확립할 수 있음
무결성	데이터베이스에 데이터가 정확하다는 것을 보증하는 것 데이터베이스 중앙통제를 통해 무결성 통제를 한층 더 쉽게 할 수 있음
불일치 최소화	데이터 수정이 필요한 경우 한 번에 일관성 있는 수정이 가능하게 해 줌

데이터독립성과 3단계 스키마

1. 데이터 독립성(Data Independency) 개요

□ 데이터 독립성(Data Independency) 정의

- 데이터베이스 내의 데이터와 이들을 사용하는 응용 프로그램 및 저장구조가 서로 영향을 받지 않는 성질
- 데이터의 논리적 구조나 물리적 구조가 변경되더라도 응용 프로그램이 영향을 받지 않도록 하는 데이터 베이스 성질

데이터독립성과 3단계 스키마

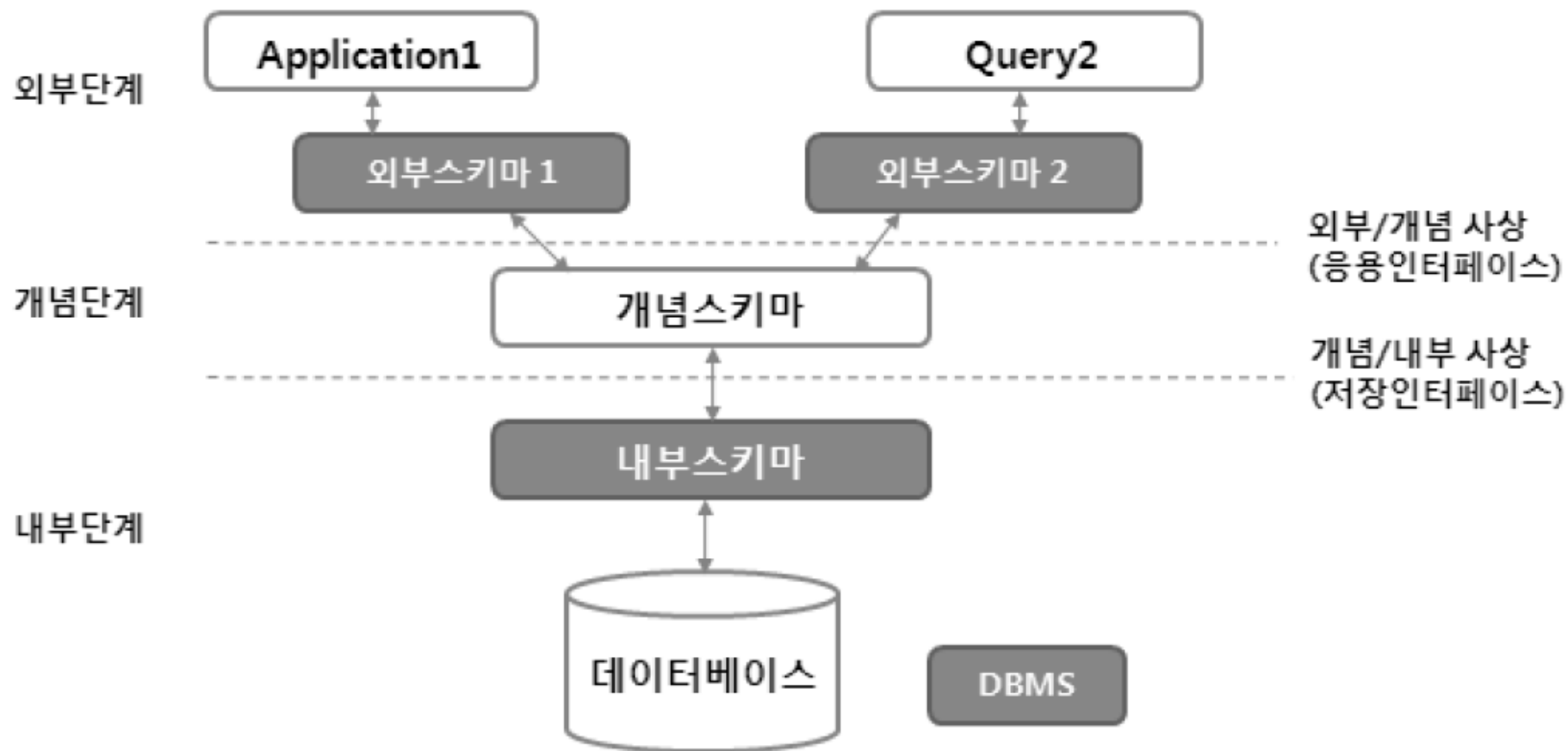
2. 데이터 독립성의 종류 및 데이터 구조간의 사상

□ 데이터 독립성 종류

종류	설명	사례
논리적 데이터 독립성	- 개념 스키마가 변경되더라도 외부 스키마에 영향을 주지 않음 - 또한 외부 스키마가 변경되더라도 개념 스키마에는 영향을 주지 않음	UI의 변경은 Entity 또는 Table의 구조에 영향을 미치지 않음(이미 관리되고 있는 속성의 경우)
물리적 데이터 독립성	- 내부 스키마가 변경되더라도 개념 스키마에 영향을 주지 않음 - 또한 개념 스키마가 변경되더라도 내부 스키마에는 영향을 주지 않음	Table이 저장되는 Disk를 hdd1에서 hdd2로 변경하는 경우 Table의 구조와 활용 방법에 영향을 미치지 않음

데이터독립성과 3단계 스키마

□ 데이터 구조간의 사상과 데이터 독립성



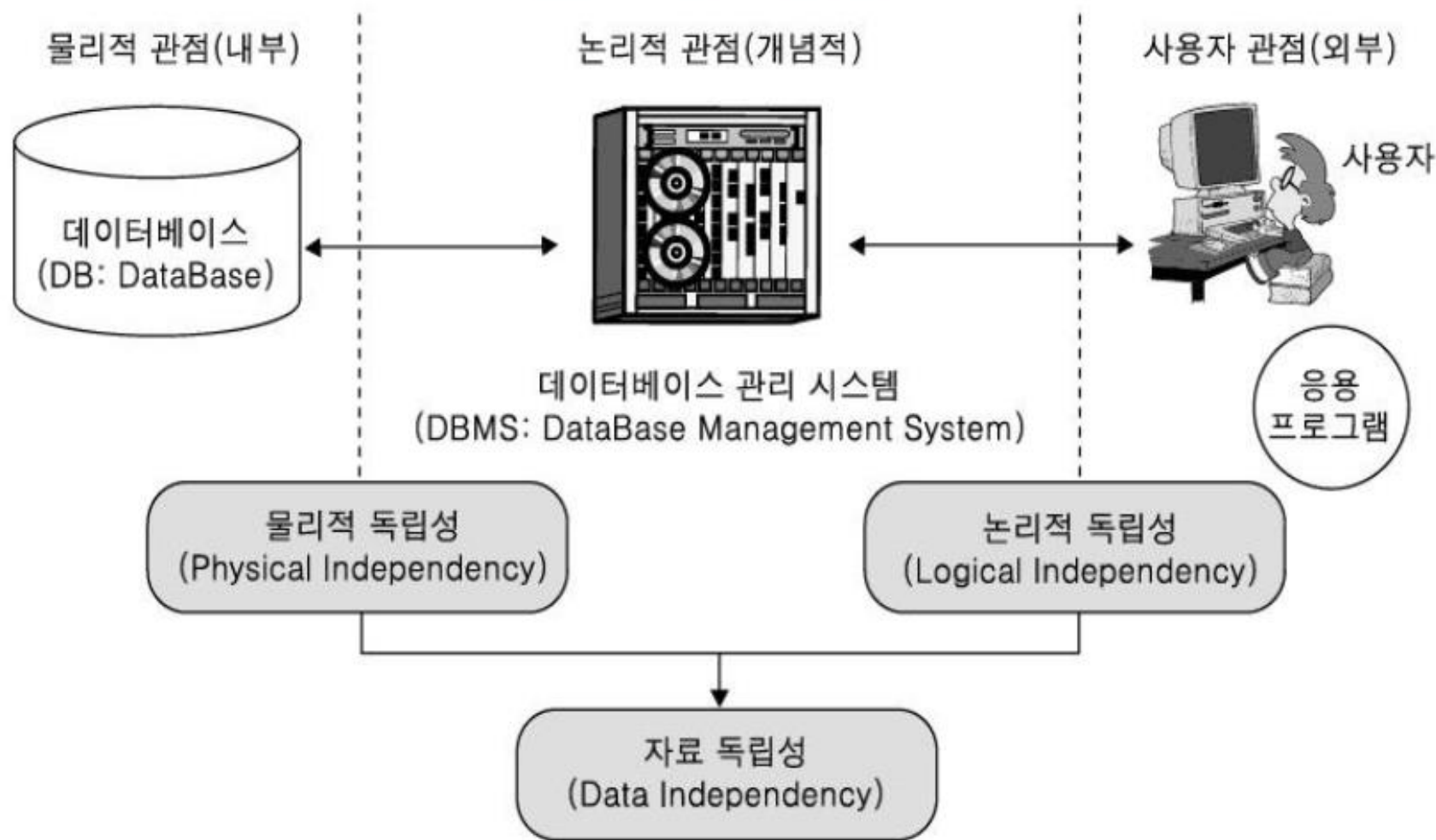
- 논리적 구조 사상은 논리적 데이터 독립성을 지원하고, 물리적 구조 사상은 물리적 데이터 독립성을 지원

데이터독립성과 3단계 스키마

사상	설명
외부/개념 사상 (External/ Conceptual schema)	특정 외부 스키마와 개념 스키마간 대응 관계 정의. 응용인터페이스라 하고 개념 스키마의 어떤 개체의 변경이나 삭제가 되어도 실제 이용하지 않은 외부 스키마에는 영향이 없도록 정의하여 논리적 독립성의 제공해 줌
개념/내부 사상 (Conceptual/ Internal schema)	개념 스키마와 내부 스키마간 대응관계를 정의. 저장인터페이스라 하고 개체간의 각 속성이 내부 필드와 어떻게 대응되는가를 정의. 내부 스키마 변경 시 개념/내부 사상만 정확히 변경시켜주면 개념 스키마는 영향이 없도록 정의하여 물리적 독립성을 제공

데이터독립성과 3단계 스키마

□ 3단계 데이터베이스 구조



데이터독립성과 3단계 스키마

종류	설명	사례
외부스키마 (External Schema)	데이터베이스 외부에서 접근 및 사용, 제어하기 위한 수준	응용 프로그램 UI(로그인, 사용자 등록 등)
개념스키마 (Conceptual Schema)	데이터의 논리적/물리적 구조와 속성 및 관계를 정의, 표현한 표준	데이터모델(Entity, Attribute, Relationship, Table, Column 등)
내부스키마 (Internal Schema)	실제 데이터가 저장되는 물리적 구조	저장 장치 구성(RAID 유형, 저장 위치 등)

데이터베이스관리시스템(DBMS)

1. 데이터베이스관리시스템(DBMS)의 개요

□ 데이터베이스관리시스템(Database Management System)의 정의

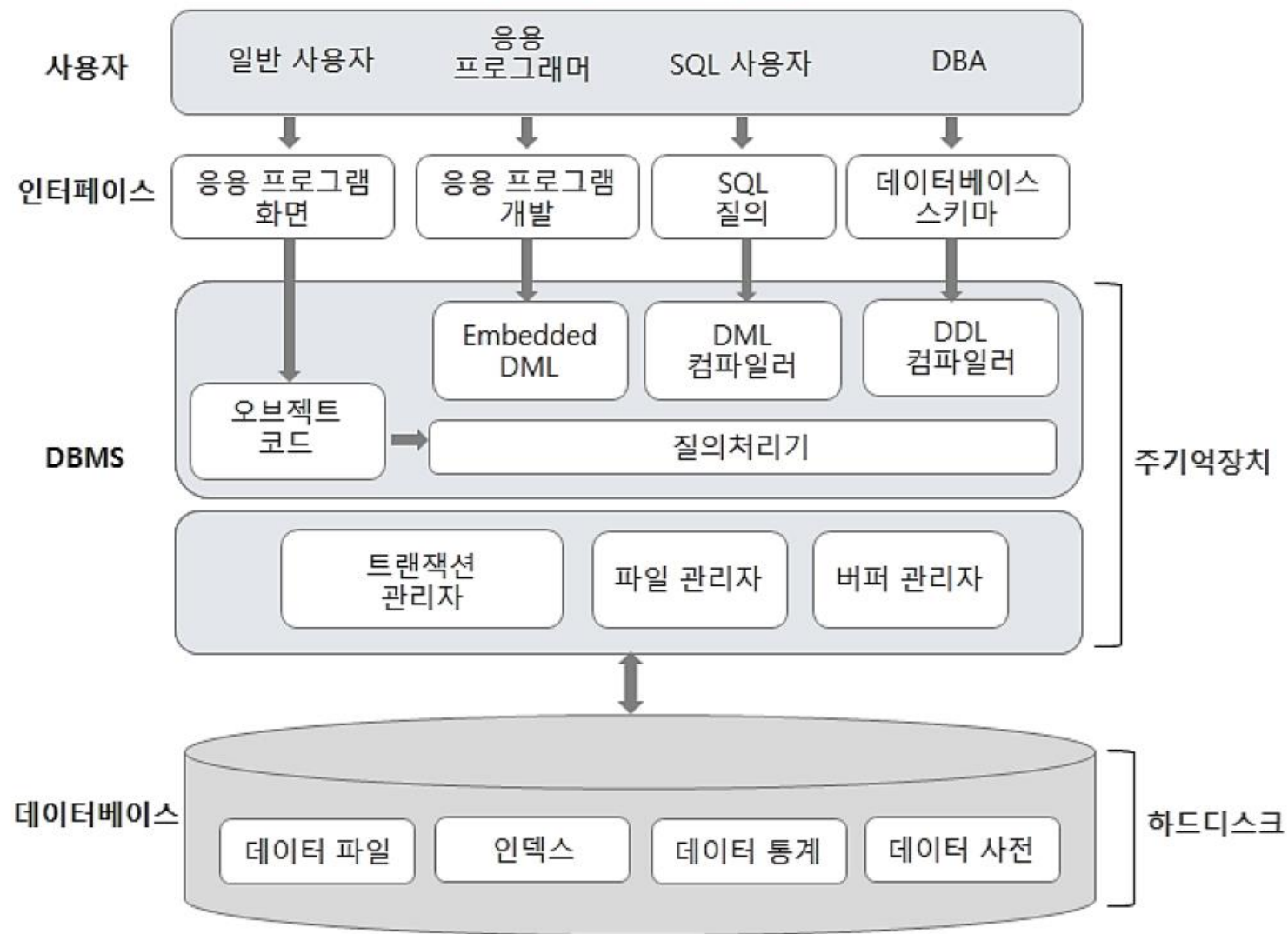
- 사용자와 데이터베이스 사이에서 사용자가 원활하게 데이터를 입/출력할 수 있게 해주는 소프트웨어
- 응용 프로그램과 데이터 사이의 중재자로서 모든 응용 프로그램들이 데이터베이스를 공유할 수 있게 관리해 주는 소프트웨어 시스템
- 조직에 필요한 데이터를 데이터베이스에 통합하고 저장하고 관리하는 소프트웨어

□ 데이터베이스관리시스템(DBMS) 주요기능

주요기능	설명	데이터언어
정의기능	데이터베이스 구조를 정의하거나 수정할 수 있음	DDL
조작기능	데이터를 삽입, 수정, 검색하는 연산을 수행	DML
제어기능	데이터를 항상 정확하고 안전하게 유지	DCL

데이터베이스관리시스템(DBMS)

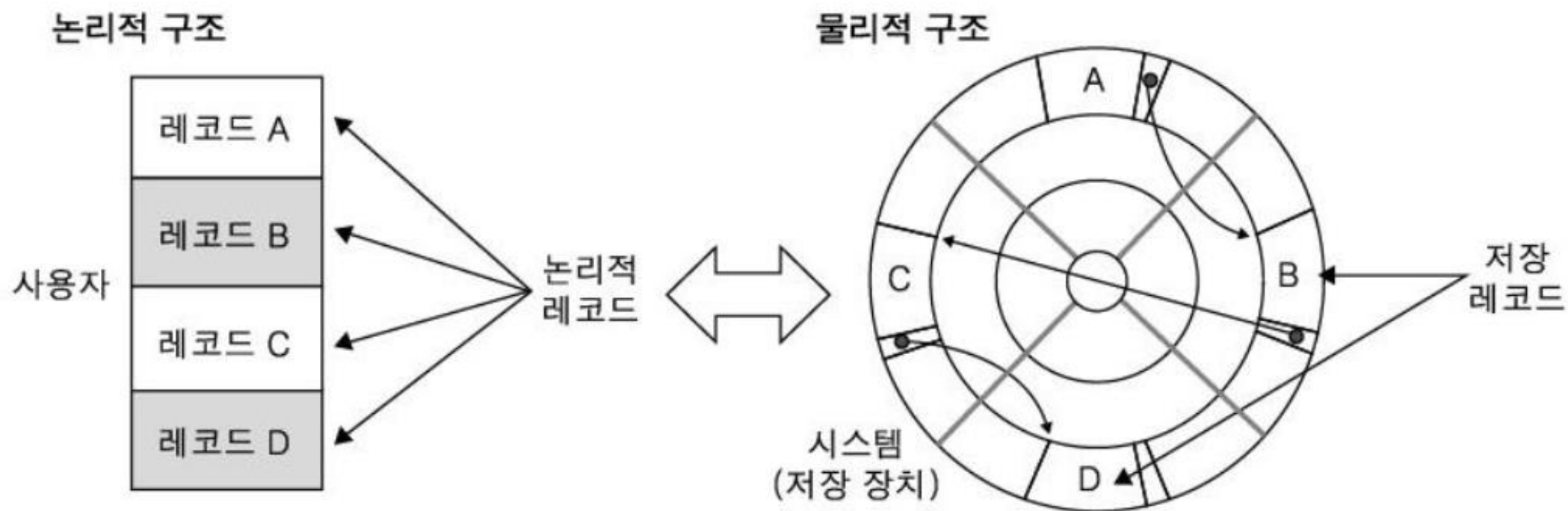
□ 데이터베이스관리시스템(DBMS) 구성도



데이터베이스관리시스템(DBMS)

3. 데이터 저장 구조 및 DBMS 종류

□ 데이터 저장 구조



데이터베이스관리시스템(DBMS)

□ DBMS 종류

구분	RDBMS	OODBMS	ORDBMS
저장자료	문자형 데이터 위주	데이터와 연관 프로그램 (메소드) 저장	데이터와 연관 프로그램 (메소드) 저장
자료모델	테이블 구조 관계 기본키(Primary key)	엔티키간 포인팅방식 객체식별(OID)	RDBMS + OODBMS
지원 자료형	미리 정의된 일반정보 타입만 지원	비정형 객체타입 지원	사용자 정의 및 비정형 객체타입 지원
DB 액세스 방식	SQL 질의어 사용	OQL(Object Query Language)	SQL 확장 질의어 사용 (SQL3)

데이터 언어(Data Language)

1. 데이터 언어(Data Language) 개념

□ 데이터 언어(Data Language) 정의

- 데이터베이스를 정의하고 접근하기 위해 데이터베이스 관리 시스템과의 통신 수단으로써의 언어
- 사용자와 데이터베이스 간의 통신 수단으로써 데이터베이스의 정의(DDL), 조작(DML), 제어(DCL)

□ 데이터 언어(Data Language) 의 종류

종류	설명
데이터 정의어(DDL)	데이터베이스 및 테이블의 구조를 정의하거나 변경
데이터 조작어(DML)	데이터의 삽입, 삭제, 검색과 수정 등을 처리
데이터 제어어(DCL)	데이터베이스 사용자의 권한을 제어

데이터 언어(Data Language)

2. 데이터 정의어(Data Definition Language)

□ 데이터 정의어(DDL, Data Definition Language)

- 데이터베이스의 논리적/물리적 구조를 정의하는 언어로 데이터베이스 관리자나 설계자가 데이터 정의어를 이용해 정의하거나 변경하고 이는 컴파일 후 데이터 사전에 저장

□ 데이터 정의어(DDL)의 유형

명령어	기능
CREATE	Schema, Domain, Table, View, Index 를 정의
ALTER	Table에 대한 정의를 변경하는 데 사용
DROP	Schema, Domain, Table, View, Index를 삭제

데이터 언어(Data Language)

3. 데이터 조작용어(Data Manipulation Language)

□ 데이터 조작용어 (DML, Data Manipulation Language)

- 데이터베이스 사용자가 응용 프로그램이나 질의어를 통하여 저장된 데이터를 실질적으로 처리하는데 사용하는 언어

□ 데이터 조작용어 (DML)의 유형

명령어	기능
SELECT	데이터베이스 객체로부터 데이터를 검색
INSERT	데이터베이스 객체에 데이터를 입력
DELETE	데이터베이스 객체에 데이터를 삭제
UPDATE	기존에 존재하는 데이터베이스 객체 안의 데이터 수정

데이터 언어(Data Language)

4. 데이터 제어어(Data Control Language)

□ 데이터 제어어(DCL, Data Control Language)

- 데이터의 보안(security), 무결성(integrity), 데이터 회복(recovery), 병행 수행(concurrency) 등을 정의하는 데 사용
- 데이터베이스 관리자가 데이터 관리를 목적으로 사용

□ 데이터 제어어(DCL)의 유형

명령어	기능
COMMIT	데이터베이스 조작 작업이 정상적으로 완료되었음을 관리자에게 알려줌
ROLLBACK	데이터베이스 조작 작업이 비정상적으로 종료되었을 때 원래 상태로 복구
GRANT	데이터베이스 사용자에게 사용권한을 부여
REVOKE	데이터베이스 사용자의 사용권한을 취소

데이터 무결성(Data Integrity)

1. 데이터 무결성(Data Integrity) 개요

□ 데이터 무결성(Data Integrity) 정의

- 데이터의 중복이나 누락이 없는 정확성과 원인과 결과 의미의 연속성이 보장되는 일관성이 확보된 상태
- 허가를 받은 사용자가 수행하는 모든 작업에 있어 데이터베이스에 저장된 데이터의 일관성(Consistency)과 정확성(Correctness)을 유지하기 위한 데이터베이스 성질
- 인가되지 않은 무효 갱신으로부터 데이터를 안전하게 보호하기 위한 DB설계 규칙

□ 데이터 무결성(Data Integrity) 보장의 필요성

- 데이터의 정확성 확보로 신뢰할 수 있는 정보 제공 가능
- 데이터의 일관성 유지를 통해 자료의 효율적인 관리 가능
- 사용자 및 어플리케이션 개발자들의 생산성 향상
- 데이터 무결성 규칙의 강제로 약간의 성능 저하를 가져오지만 데이터 신뢰도 증가

데이터 무결성(Data Integrity)

2. 데이터 무결성의 종류 및 무결성을 보장하기 위한 기능

□ 데이터 무결성의 종류

무결성	개념	제약조건 예
개체 무결성 (Entity Integrity)	기본키는 반드시 값을 가져야 함 기본키는 유일성을 보장하는 최소한의 집합	Primary Key, NOT NULL
참조 무결성 (Referential Integrity)	외래키 속성은 참조할 수 없는 값을 지닐 수 없음 외래키 값은 그 외래키가 기본키로 사용된 릴레이션의 기본키 값이거나 NULL만 허용	Foreign Key
속성 무결성 (Attribute Integrity)	컬럼은 지정된 데이터 형식을 반드시 만족하는 값만 포함	Character, Date, LONG 등
키 무결성 (Key Integrity)	한 릴레이션에 같은 키 값을 가진 튜플들은 허용 안 됨	Primary Key + Unique Index
사용자 정의 무결성 (User Define Integrity)	모든 데이터는 업무 규칙 (Business Rule)을 준수해야함	Trigger, CHECK, DEFAULT Value

데이터 무결성(Data Integrity)

□ 데이터 무결성의 실무 적용 상세

무결성 요소	설명
개체무결성	Book 엔티티 타입에서 Book_no 라는 PK를 통해 데이터의 중복(Duplicate)과 누락(Null)을 방지하는 역할을 수행 즉, 엔티티 무결성을 Primary key를 통해 유지하는 역할
참조무결성	Member와 Order 엔티티타입의 관계에서 Order는 반드시 member에 등록된 사용자가 신청을 해야 하는 관계. Order에 들어가는 Member_no의 값은 member의 값을 반드시 참조하여 들어가야 데이터 무결성을 유지
속성무결성	Order 속성은 데이터 타입이 varchar2를 많이 사용하여 정의 데이터 타입에는 number 형식이 들어가면 안되는 무결성 규칙 존재. Member_no가 Not Null 로 반드시 데이터가 들어가야 하는 의미
사용자무결성	Book에 Price라는 속성에 들어가는 단가 정보가 주문된 여러가지 도서 정보를 종합하여 계산된 (Derived Column) OrderItem의 Buy_Amt는 반드시 일관성이 있는 값을 보유 주문된 단가 정보의 총합이 Buy_Amt값과 일치해야 하는 데이터 무결성 특징을 가지고 있음.

감사합니다!



데이터베이스



2. 관계형 데이터베이스

KPC 기본반

2. 관계형 데이터베이스.....	20
릴레이션(Relation)	21
데이터베이스 키(Database Key)	24
무결성 제약조건(Integrity Constraint)	28

릴레이션(Relation)

1. 릴레이션(Relation) 개요

□ 릴레이션(Relation) 정의

- 하나의 개체에 관한 데이터를 2차원 테이블의 구조로 저장한 것으로 파일 관리 시스템 관점에 파일(File)에 대응되는 개념

□ 릴레이션(Relation) 관련 용어

용어	한글용어
Relation	릴레이션, 테이블
Relational data model	관계 데이터 모델
Relational database	관계 데이터베이스
Relational algebra	관계대수
Relationship	관계

릴레이션(Relation)

□ 데이터와 릴레이션

도서 1, 축구의 역사, 굿스포츠, 7000
도서 2, 축구하는 여자, 나무수, 13000
도서 3, 축구의 이해, 대한미디어, 22000
도서 4, 골프 바이블, 대한미디어, 35000
도서 5, 피겨 교본, 굿스포츠, 8000



도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구하는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

- 첫 번째 행(1, 축구의 역사, 굿스포츠, 7000)의 경우 네 개의 집합에서 각각 원소 한 개씩 선택하여 만들어진 것으로 이 원소들이 관계(relationship)를 맺고 있음

릴레이션(Relation)

□ 릴레이션 간의 관계

도서(도서번호, 도서이름, 출판사, 가격)

주문(도서번호, 고객번호, 판매가격, 주문일자)

고객(고객번호, 이름, 주민번호, 주소, 핸드폰)

[릴레이션 간의 관계]

- 1) 릴레이션 내에서 생성되는 관계: 릴레이션 내 데이터들의 관계
- 2) 릴레이션 간에 생성되는 관계: 릴레이션 간의 관계

릴레이션(Relation)

□ 릴레이션 스키마(Relation Schema)

- 릴레이션의 논리적 구조로 릴레이션의 이름과 릴레이션에 포함된 모든 속성 이름으로 정의
- 릴레이션을 내포(Relation intension)라고 하기도 하고 정적인 특징을 포함
 - 스키마의 표현: 도서(도서번호, 도서이름, 출판사, 가격)

속성(애트리뷰트),
열(column) 이라고도 함
(차수=4)

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구하는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

도서

스키마(내포)
Schema

인스턴스(외연)
Instance

튜플(tuple),
행(row) 이라고도 함
(카디널리티=5)

릴레이션(Relation)

□ 릴레이션 인스턴스(Relation Instance)

- 어느 한 시점의 릴레이션에 존재하는 튜플들의 집합으로 릴레이션 외연(extension)이라 하며 동적인 특징을 가짐
 - 튜플(tuple): 릴레이션의 행으로 튜플이 가지는 속성의 개수는 릴레이션 스키마의 차수와 동일하고 릴레이션 내의 튜플들은 서로 중복이 없어야 함
 - 카디날리티(cardinality): 튜플의 수

□ 릴레이션 용어 정리

릴레이션 용어	같은 의미 용어	파일 시스템 용어
릴레이션(relation)	테이블(table)	파일(file)
스키마(schema)	내포(intension)	헤더(header)
인스턴스(instance)	외연(extension)	데이터(data)
튜플(tuple)	행(row)	레코드(record)
속성(attribute)	열(column)	필드(field)

릴레이션(Relation)

3. 릴레이션(Relation)의 특성 및 위배 사례

□ 릴레이션(Relation)의 특성

특성	설명
튜플의 유일성	하나의 릴레이션에는 동일한 튜플이 존재할 수 없음
튜플의 무순서	하나의 릴레이션에서 튜플 사이의 순서는 무의미
속성의 무순서	하나의 릴레이션에서 속성 사이의 순서는 무의미 함
속성의 원자성	속성 값으로 원자 값만 사용 가능

릴레이션(Relation)

□ 릴레이션 특성을 위반한 사례

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
5	피겨 교본	굿스포츠	8000
6	피겨 교본, 피겨 기초	굿스포츠	8000

동일한 투플이
중복되면 안 됨

속성의 값은 단일 값이어야 함

데이터베이스 키

1. 데이터베이스 키(Database Key)의 개요

□ 데이터베이스 키(Database Key)의 정의

- 릴레이션에서 튜플을 유일하게 구별하는 속성 또는 속성들의 집합
- 릴레이션은 중복된 튜플을 허용하지 않기 때문에 각 튜플에 포함된 속성 중 하나는 반드시 값이 달라야 함
- 키가 되는 속성은 반드시 값이 달라 튜플들을 서로 구별을 할 수 있어야 함

□ 키의 특성

특성	설명
유일성(uniqueness)	하나의 릴레이션에서 모든 튜플은 서로 다른 키 값을 가져야 함
최소성(minimality)	꼭 필요한 최소한의 속성들로만 키를 구성

데이터베이스 키

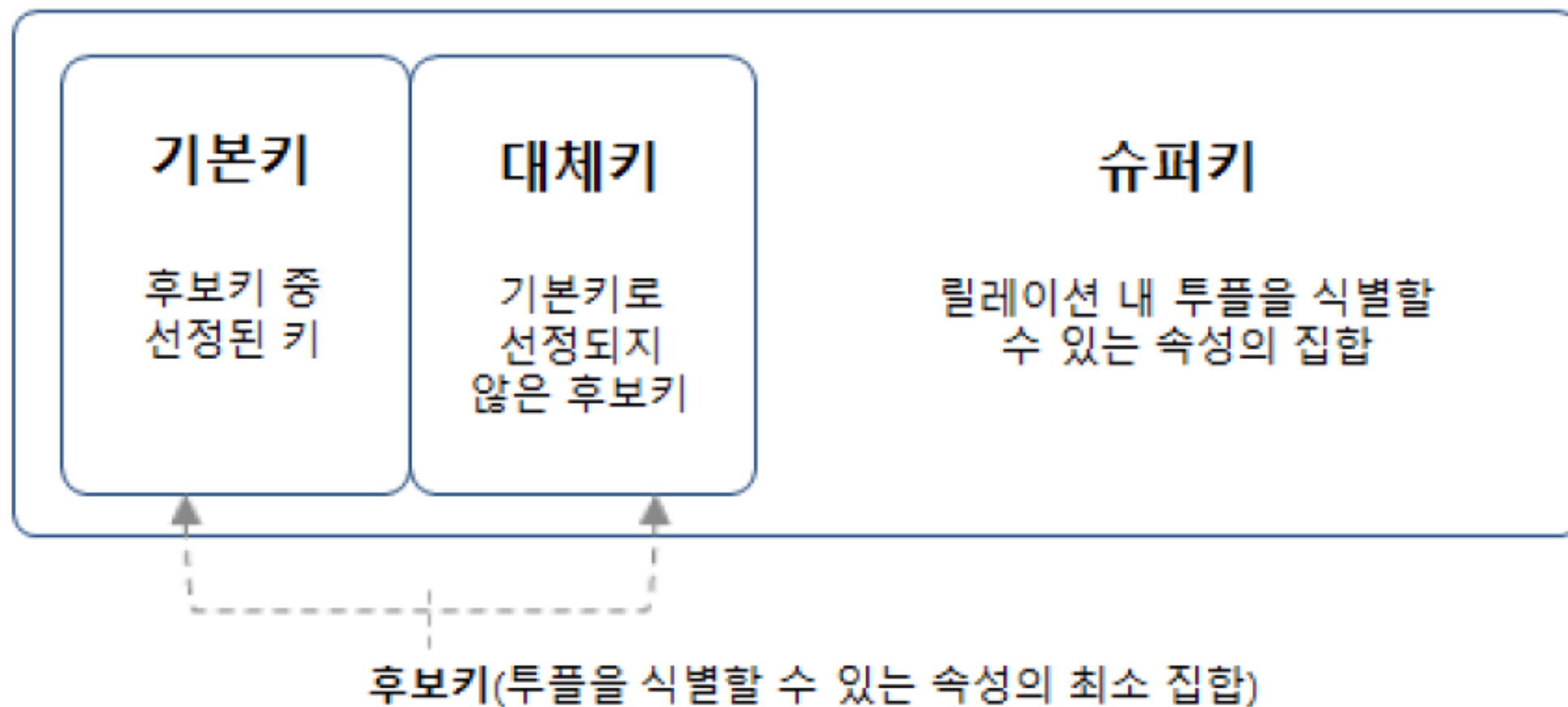
2. 데이터베이스 키의 종류 및 포함관계

□ 데이터베이스 키의 종류

종류	설명
수퍼키(super key)	튜플을 유일하게 식별할 수 있는 하나의 속성 혹은 속성의 집합 유일성은 만족하나 최소성을 만족하지 않는 키
후보키(candidate key)	튜플을 유일하게 식별할 수 있는 속성의 최소 집합 키의 특성인 유일성과 최소성(Not Null)을 만족하는 키
기본키(primary key)	여러 개의 후보키 중에서 하나를 선정하여 테이블을 대표하는 키 후보키가 여러 개라면 릴레이션의 특성을 반영하여 하나를 선택
대체키(alternate key)	여러 개의 후보키 중에서 기본키로 선정되고 남은 나머지 키 기본키를 대체할 수 있는 키
외래키(Foreign Key)	다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합

데이터베이스 키

□ 데이터베이스 키의 포함관계



데이터베이스 키

3. 기본키(Primary Key)

□ 기본키(Primary Key)의 정의

- 여러 개의 집합체를 담고 있는 하나의 엔티티 타입에서 각각의 엔티티를 구분할 수 있는 결정자

□ 기본키(Primary Key) 선정 시 고려사항

- 릴레이션 내 튜플을 식별할 수 있는 고유한 값을 가져야 함
- NULL 값은 허용하지 않음
- 키 값의 변동이 일어나지 않아야 함
- 최대한 적은 수의 속성을 가진 것이라야 함
- 향후 키를 사용하는 데 있어서 문제 발생 소지가 없어야 함

□ 릴레이션 스키마에서의 기본키 표시

- 릴레이션 이름(속성1, 속성2, 속성N)
- 릴레이션 스키마를 표현할 때 기본키는 밑줄을 그어 표시

예시) 고객(고객번호, 이름, 주민번호, 주소, 핸드폰)

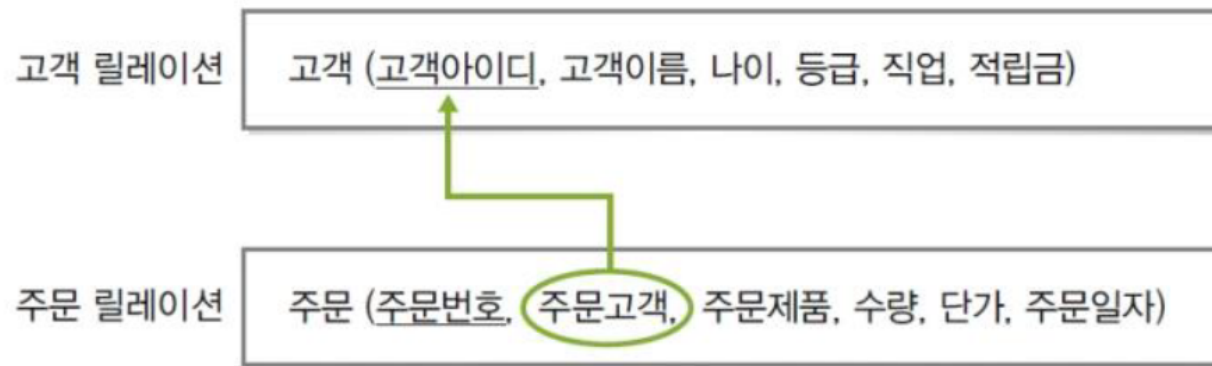
도서(도서번호, 도서이름, 출판사, 가격)

데이터베이스 키

4. 외래키(Foreign Key)

□ 외래키(Foreign Key)의 정의

- 다른 릴레이션의 기본키를 참조하는 속성으로 관계 데이터 모델의 특징인 릴레이션 간의 관계(Relationship)을 표현



- 주문 릴레이션의 주문고객은 고객 릴레이션의 고객 아이디를 참조

□ 외래키(Foreign Key)의 특징

- 관계 데이터 모델의 릴레이션 간의 관계를 표현함
- 다른 릴레이션의 기본키를 참조하는 속성임
- 참조하고(외래키) 참조되는(기본키) 양쪽 릴레이션의 도메인은 서로 같아야 함
- 참조되는(기본키) 값이 변경되면 참조하는(외래키) 값도 변경됨
- NULL 값과 중복 값 등이 허용됨
- 자기 자신의 기본키를 참조하는 외래키도 가능함
- 외래키가 기본키의 일부가 될 수 있음

데이터베이스 키

□ 외래키(Foreign Key)의 사용목적

구분	설명
데이터 모델의 운영규칙 명시	개념적 데이터 모델의 엔터티간에 운영규칙을 나타내기 위함
관계의 명확성	외래키를 정의하는 것은 2 번째 단계에서 작성한 엔터티간의 관계와 다소 중복되는 것처럼 보이지만 외래키를 정의하는 것은 부모-자식 관계를 보다 확실하게 하기 위함
데이터의 중복성 확인	외래키 선정작업은 부모-자식 엔터티의 선택, 있을 수 있는 중복관계 및 수퍼타입-섭브타입 지적을 재검토할 기회를 갖기 위함
테이블간 참조 매커니즘	관계 데이터베이스를 설계할 때 외래키는 관련 테이블간에 기본적인 참조장치로서 작용
엔터티의 의미 부여	외래키는 의미있는 이름을 부여하기 위함
데이터의 참조 무결성	데이터의 참조 무결성을 확정하는 것 허용된 데이터 수치만 데이터베이스 내에 저장할 수 있음 참조하는 테이블과의 연결고리를 만듦으로서 무결성을 유지하기 위함

데이터베이스 키

□ 릴레이션 간의 참조 관계

- 주문 릴레이션은 고객 릴레이션의 기본키인 고객번호와 도서 릴레이션의 기본키인 도서번호를 참조하여 주문을 생성
- 주문 릴레이션의 기본키는 주문번호이고 주문번호에 따라 고객번호, 도서번호를 관리

고객

고객번호	이름	주민번호	주소	핸드폰
1	박지성	810101-1111111	영국 맨체스터	000-5000-0001
2	김연아	900101-2222222	대한민국 서울	000-6000-0001
3	장미란	830101-2333333	대한민국 강원도	000-7000-0001
4	추신수	820101-1444444	미국 클리블랜드	000-8000-0001

기본키

도서

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구하는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

기본키

참조

주문

주문번호	고객번호	도서번호	판매가격	주문일자
1	1	1	7000	2014-07-01
2	1	2	13000	2014-07-03
3	2	5	8000	2014-07-03
4	3	2	13000	2014-07-04
5	4	4	35000	2014-07-05
6	1	3	22000	2014-07-07
7	4	3	22000	2014-07-07

기본키

외래키

참조

데이터베이스 키

□ 데이터 모델링의 무결성 위한 Key의 제약

구분	Key	설명
본질적 제약	PK, Unique Key	- 데이터 모델의 구조적 특성으로 인한 제약 - 반드시 Primary Key가 있어야 하고 테이블의 각 셀이 단일 값을 가짐(1차 정규형)
내재적 제약	FK, Check, Default, Not Null	- 데이터의 의미를 정확히 표현하고 오류를 방지 - 데이터베이스의 스키마에 지정하는 제약 - 영역 제약, 참조 무결성 제약
명시적 제약	Programmatically	- 프로그램에 명시하거나 사용자의 수작업으로 생성

- 데이터베이스 관리 시스템이 발전할수록 프로그램을 통한 명시적 제약 대신 스키마를 통한 내재적 제약 형태를 사용

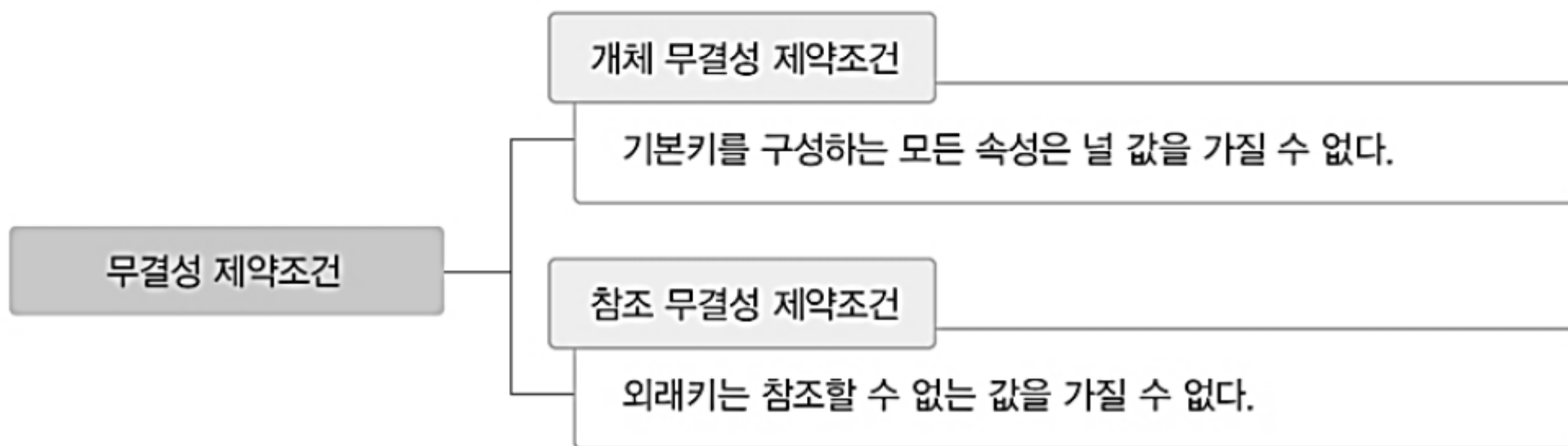
무결성 제약조건

1. 무결성 제약조건(Integrity Constraint) 개요

□ 무결성 제약 조건의 정의

- 데이터의 무결성을 보장하고 일관된 상태로 유지하기 위한 규칙
- 무결성: 데이터를 결함이 없는 상태로 정하고 유효하게 유지하는 것

□ 관계형 데이터 모델의 무결성 제약조건



무결성 제약조건

3. 개체 무결성 제약조건(entity integrity constraint)

□ 개체 무결성 제약 조건(entity integrity constraint) 개념

- 기본키를 구성하는 모든 속성은 널 값을 가질 수 없는 규칙으로 기본키 제약(primary key constraint)이라고도 함
- 릴레이션은 기본키를 지정하고 그에 따른 무결성 원칙으로 기본키는 NULL 값을 가져서는 안 되며 릴레이션 내에 오직 하나의 값만 존재해야 한다는 조건

고객아이디	고객이름	나이	등급	직업	적립금
apple	김현준	20	gold	학생	1000
NULL	정소화	25	vip	간호사	2500
carrot	원유선	28	gold	교사	4500
NULL	정지영	22	silver	학생	0

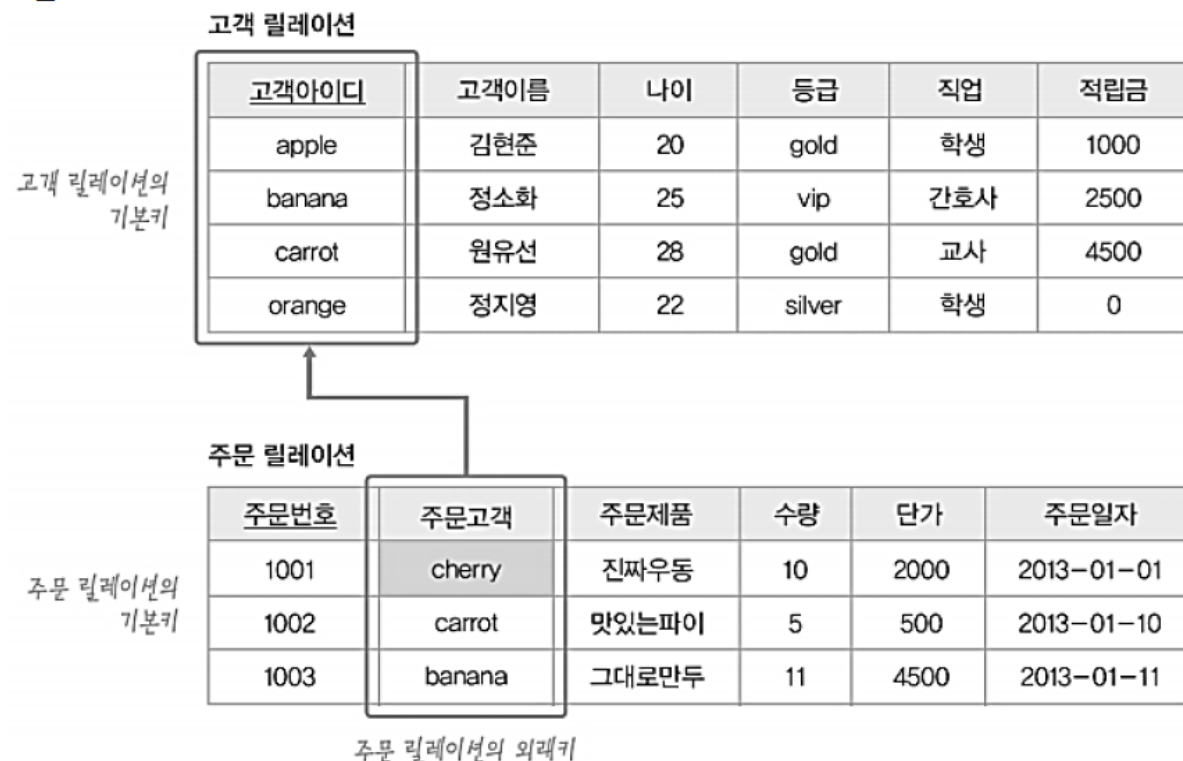
- 개체 무결성 제약 조건을 위반하여 위와 같이 고객 아이디를 null 값을 허용하게 되면, 튜플을 유일하게 식별할 수 없음

무결성 제약조건

4. 참조 무결성 제약조건(referential integrity constraint)

□ 참조 무결성 제약조건(referential integrity constraint) 개념

- 릴레이션 간의 참조 관계를 선언하는 제약조건으로 자식 릴레이션의 외래키는 부모 릴레이션의 기본키와 도메인이 동일해야 하며, 자식 릴레이션의 값이 변경될 때 부모 릴레이션의 제약을 받는다는 것
- 외래키는 참조할 수 없는 값을 가질 수 없는 규칙으로 외래키 제약(foreign key constraint) 이라고도 함



감사합니다!



데이터베이스



3. 데이터 모델링

KPC 기본반

3. 데이터 모델링	32
데이터베이스 구축	33
데이터베이스 모델링	35
개념적 모델링	40
논리적 모델링	44
물리적 모델링	46
식별자(Identifier)	49

데이터베이스 구축

1. 데이터 모델링(Data Modeling)과 데이터베이스 설계(Database Design)

□ 데이터 모델링(Data Modeling) 정의

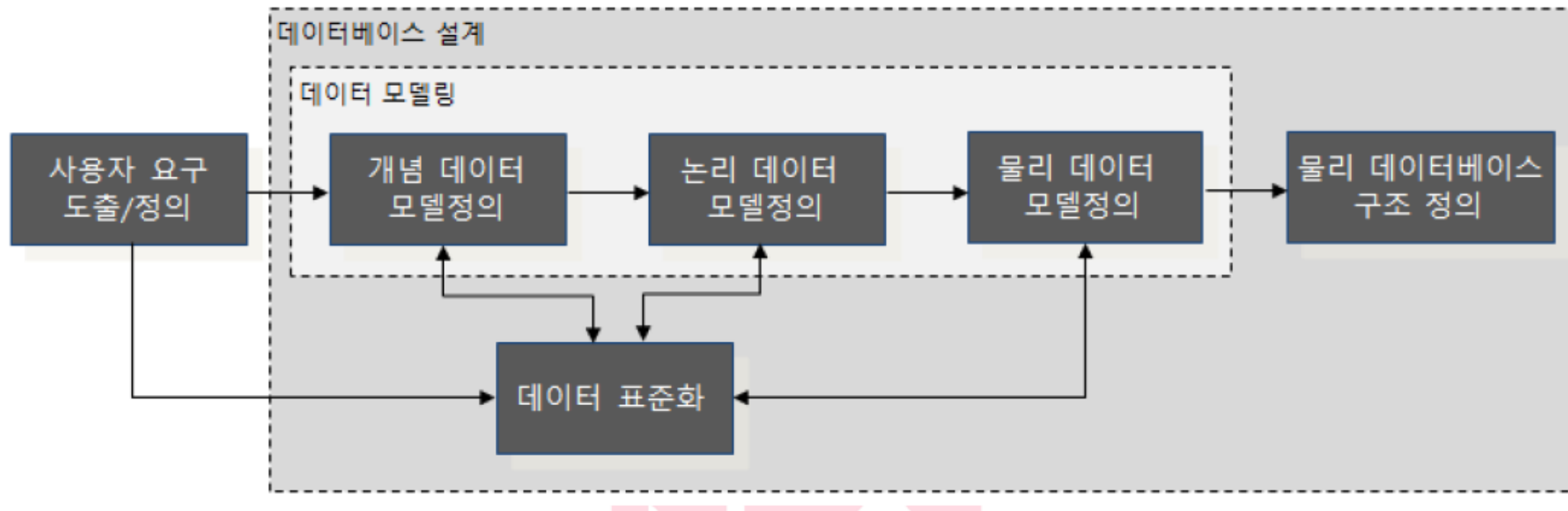
- 데이터베이스 설계 과정에서의 핵심 기법으로, 사용자 요구 사항을 분석하고 필요한 데이터 요소를 도출하여 적절한 데이터 구조를 정의

□ 데이터베이스 설계(Database Design) 정의

- 데이터베이스 설계는 데이터 모델링을 핵심기법으로 사용하여 사용자 요구사항을 분석하고 사용자가 요구하는 정보를 제공할 수 있도록 데이터베이스 구조와 접근 방법 등을 정의하는 활동

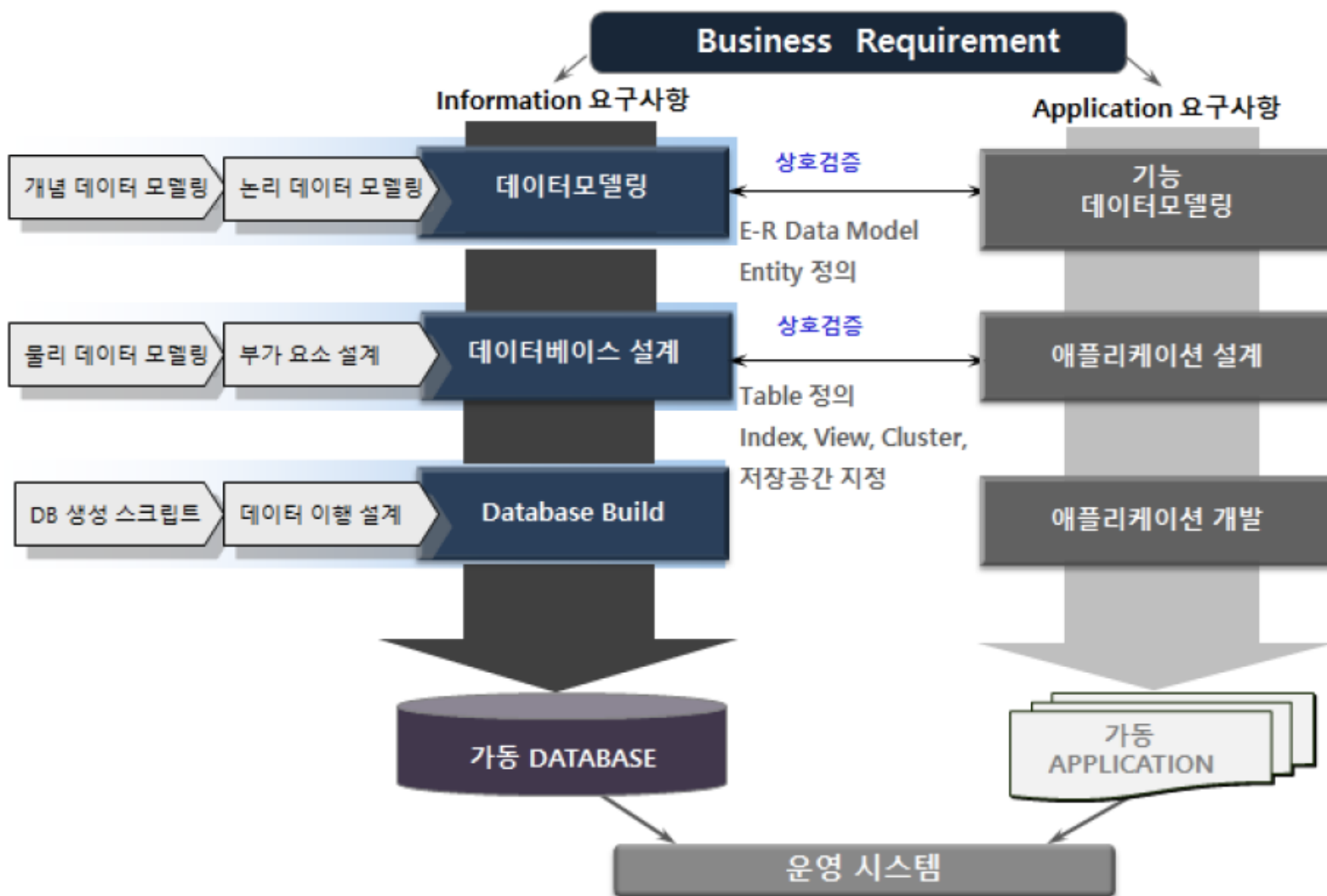
데이터베이스 구축

□ 데이터베이스 설계와 데이터 모델링의 관계



데이터베이스 구축

□ 데이터베이스 구축절차도



데이터베이스 구축

□ 데이터베이스 구축 단계별 주요 활동

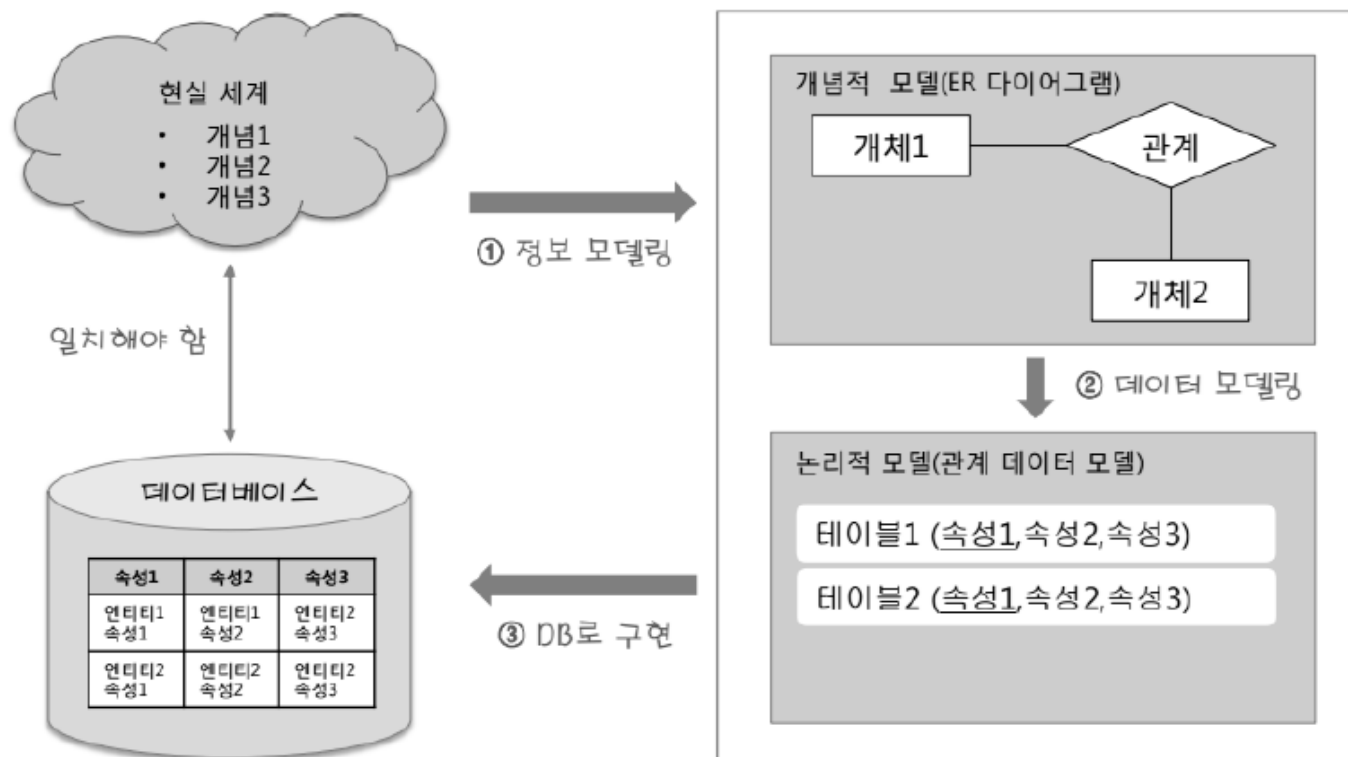
단계	주요활동	활동설명	주요산출물	산출물 설명
데이터 모델링	개념 데이터 모델링	- 데이터 모델의 주요 골격에 해당하는 핵심 엔터티와 그들간의 관계를 정의 - 전체 데이터 모델의 구도를 잡는 개념	Entity-Relationship Diagram	핵심 엔터티와 그들간 관계를 ER Diagram으로 표현
			엔터티 정의서	도출된 핵심 엔터티에 대한 정의 내역을 명세
	논리 데이터 모델링	- 개념 데이터 모델을 상세화하여 데이터에 대한 상세하고 구체적인 논리적 구조를 정의 - 사람이 결정해야 할 대부분의 사항들이 이 단계에서 도출되고 정의	Entity-Relationship Diagram	상세 정의된 논리적 데이터 구조를 ER Diagram으로 표현
			엔터티 정의서	도출된 모든 엔터티에 대한 상세 정의 내역을 명세
			속성 정의서	엔터티별 구성 속성에 대한 상세 정의 내역을 명세

데이터베이스 모델링

1. 데이터 모델링(Data Modeling) 개념

□ 데이터 모델링(Data Modeling) 정의

- 데이터베이스 설계 과정에서의 핵심 기법으로 사용자 요구 사항을 분석하고 필요한 데이터 요소를 도출하여 적절한 데이터 구조를 정의
- 업무 프로세스를 추상화하여 데이터베이스의 데이터로 표현하기 위한 설계 과정



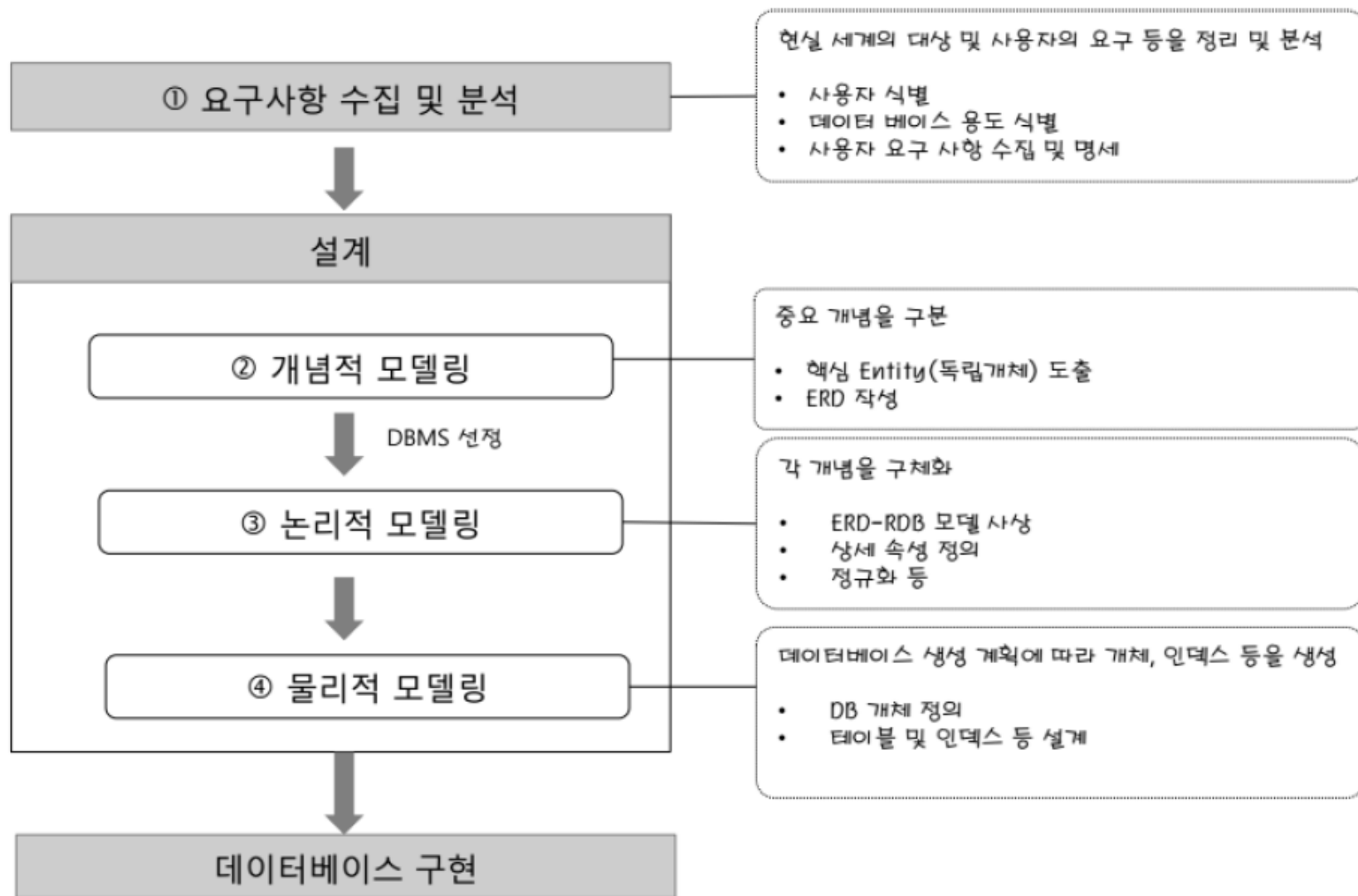
데이터베이스 모델링

□ 데이터 모델링(Data Modeling)의 3가지 관점

관점	설명
데이터 관점	업무가 어떤 데이터와 관련이 있는지, 데이터간의 관계는 무엇인 지에 대해 모델링하는 방법
프로세스 관점	업무를 통해 어떤 일을 처리하는지, 무엇을 해야 하는지를 모델링하는 방법
데이터와 프로세스 상관 관점	업무에서 일을 처리하는 방법에 따라 데이터가 어떻게 영향을 받는지 모델링하는 방법(상관모델링, CRUD Matrix 기법)

데이터베이스 모델링

□ 데이터 모델링 단계



데이터베이스 모델링

단계	설 명	수준
개념적 데이터 모델링	- 추상화 수준 높고 업무중심적, 포괄적인 수준 모델링 진행 - 핵심 엔티티 추출, 속성 및 관계 정의	추상적  구체적
논리적 데이터 모델링	- 구축 시스템 업무에 대해 Key, 속성, 관계 등을 정확히 표현, 재사용성을 높임. 식별자 확정, 정규화 수행	
물리적 데이터 모델링	- 데이터베이스 이식가능 하도록 성능, 물리적 성격을 고려하여 설계 - DBMS의 특성 및 종류, 구현환경을 감안한 스키마 도출 - 컬럼의 데이터 타입과 크기, 제약조건/인덱스 정의	

데이터베이스 모델링

□ 데이터 모델링 단계별 활동

관점	설명
엔티티 정의	엔티티 후보 선정, 엔티티 검증 및 확정
관계정의	관계(Relationship) 파악 관계 명칭 확정
개념모델작성	핵심 엔티티 배치, 키 엔티티 연결, 엔티티 간 관계 확정
속성 정의	최소단위 분할, 단일 값 검증, 가공속성 제거
식별자 정의	의미상 주어 확인, 상속 및 단절 전략, 식별자 확정
데이터 모델 상세화	정규화(1, 2, 3 정규화), M:M 관계 해소
데이터 모델 통합	엔티티 통합(수직, 수평, 차원적 통합), 엔티티간 관계 통합
이력관리 결정	이력관리 대상 선정, 이력관리 형태 결정, User Review

데이터베이스 모델링

4. 대표적인 데이터 모델링 표기법

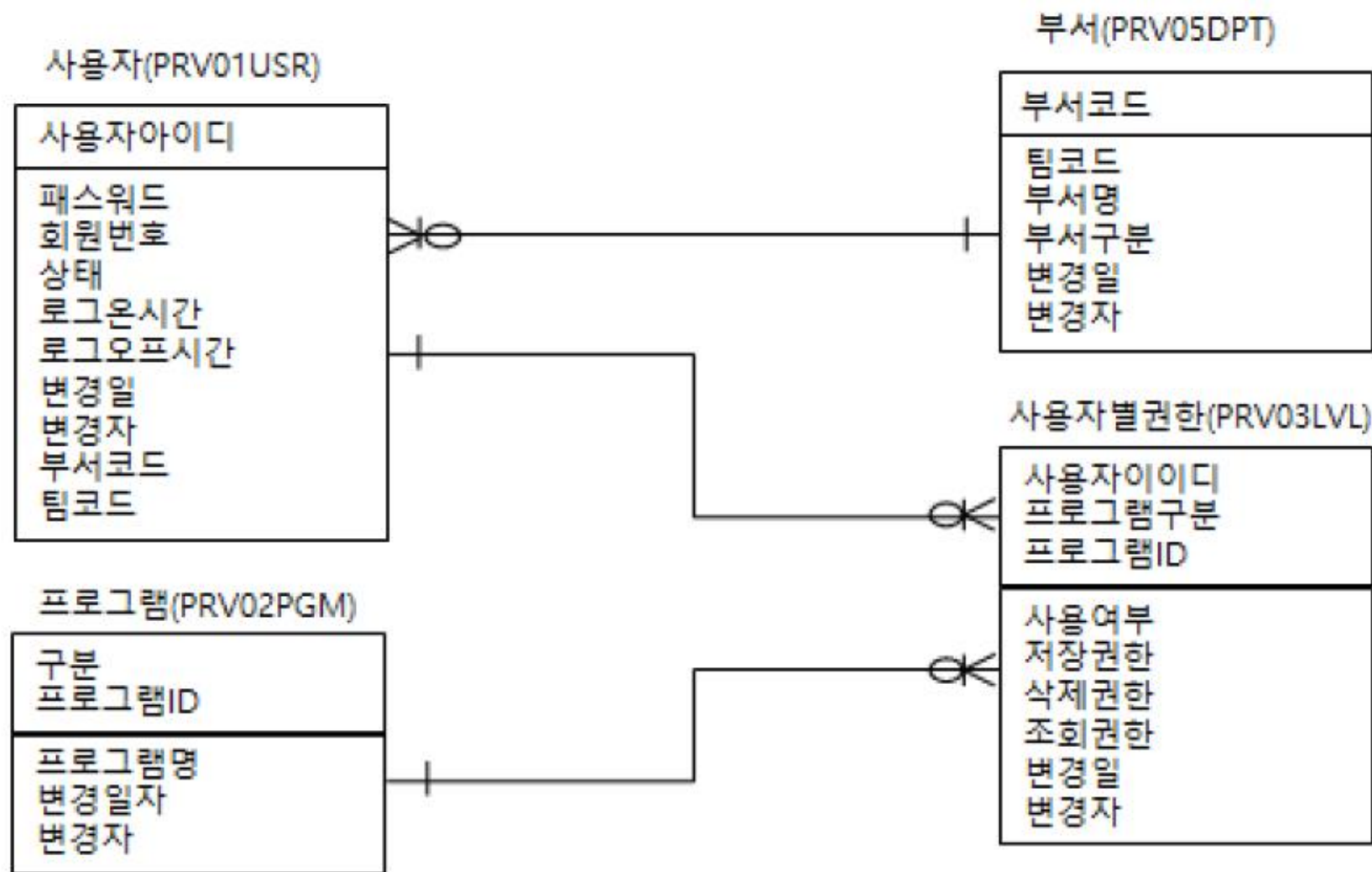
□ 정보공학 표기법(IE Notation)

- 제임스 마틴(James Martin)이 주창한 Information Engineering(I/E)에서 사용하는 데이터 모델 표기법
- 까마귀발 모양과 같은 형태의 관계 표현 때문에 Crow's Foot Model로도 불림
- 가장 널리 사용되는 표기법의 하나이지만 서브타입과 같은 개체 집합의 상세 표현 있어서 공간을 많이 차지하는 단점

Identifying	Non-identifying	의미
		0,1 , 또는 그 이상의 개체 허용
		1 또는 그 이상의 개체 허용
		0 또는 1 개체 허용
		정확히 1 개체 허용

데이터베이스 모델링

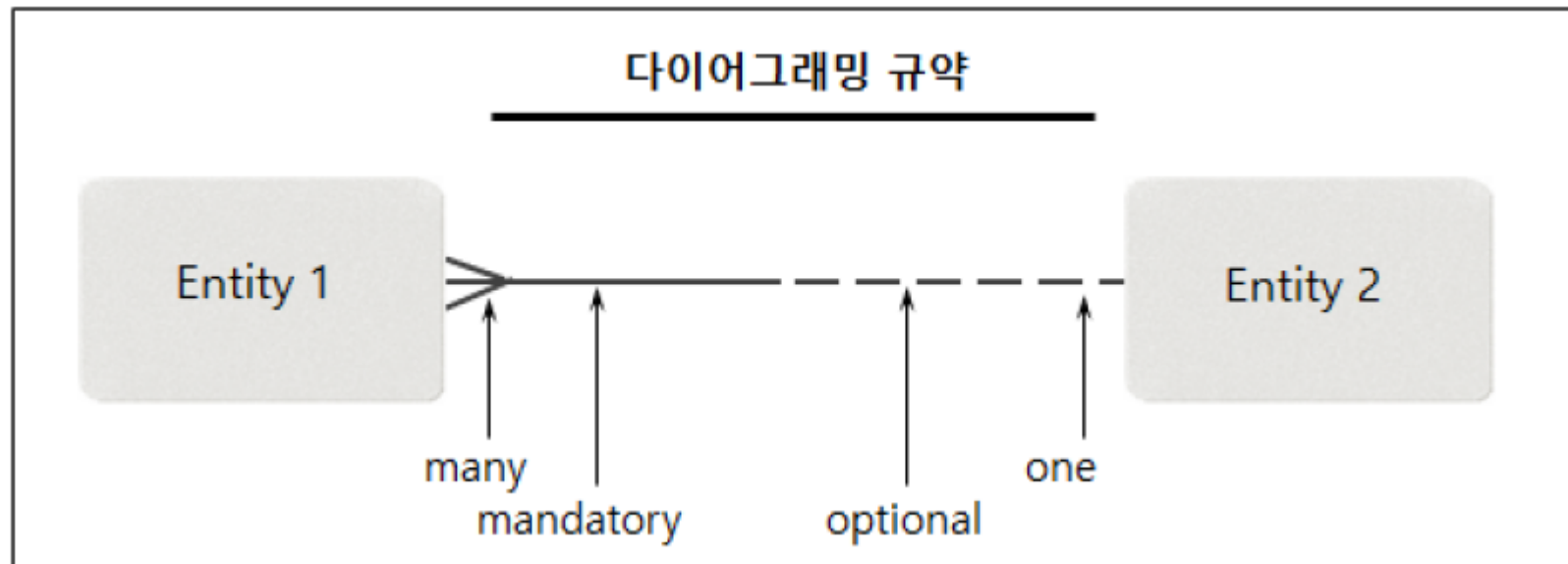
[작성사례]



데이터베이스 모델링

□ 바커 표기법(Barker's Notation)

- 1986년, 영국 컨설팅 회사 CACI에 근무하던 Richard Barker, Ian Palmer, Harry Ellis 등에 의해 처음 개발
- 후에 오라클에서 Case Method(Custom Development Method)로 채택하여 사용되고, Oracle CASE modeling tools에 적용
- 오늘날 널리 사용되는 표기법의 하나로 개체 집합과 그 관계의 표현에 있어서 직관적 이해성이 뛰어나고 구체적이고 상세한 표현성 및 공간 활용의 장점 등으로 인해 사용자가 지속적으로 증가



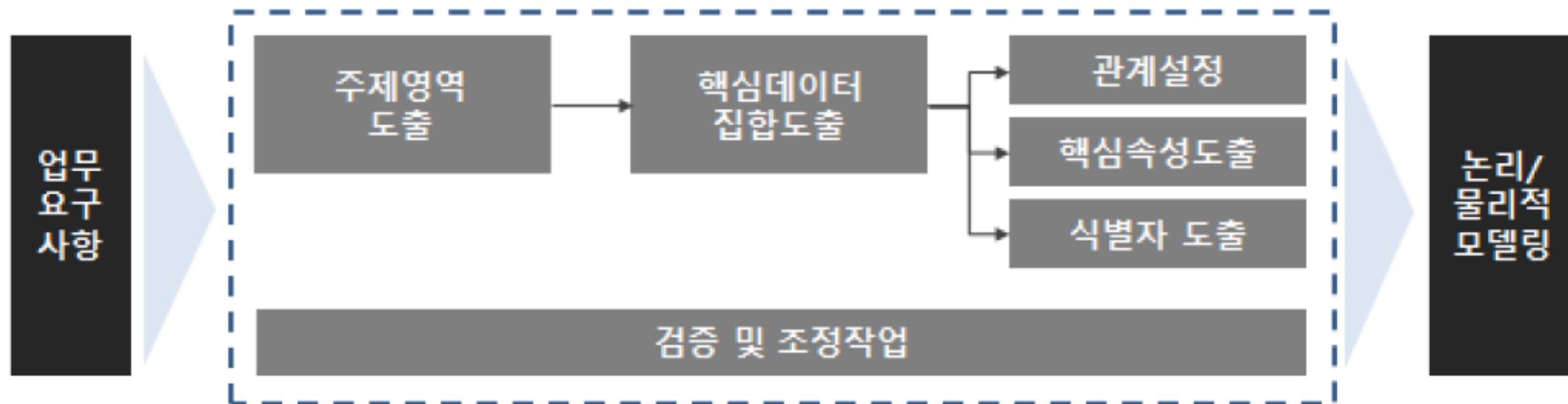
개념적 모델링

1. 개념적 모델링 개요

□ 개념적 모델링 정의

- 주제영역과 핵심 데이터 집합, 데이터 집합 간의 관계를 정의하는 개략적 데이터 설계 작업
- DBMS 클래스나 특정 DBMS와 독립적인 개념적 스키마를 기술하는 과정

□ 개념적 모델링 절차



개념적 모델링

절차	상세내용	기법
주제영역 선정	- 하위 주제 영역 또는 데이터 집합들로 구성 - 업무 기능과 대응	상향식, 하향식, Inside-out, 혼합식 설계방식
핵심데이터 집합선정(Entity)	- 데이터의 보관 단위로써 주제영역에서 중심이 되는 데이터 집합을 정의	독립중심, 의존중심, 의존특성, 의존연관데이터
관계설정 (cardinality)	- 업무적 연관성에 따라 개체간 갖는 relationship 설정	1:1, 1:N, M:N, 순환관계
핵심속성 정의 (Attribute)	- 데이터 집합의 특성을 나타내는 항목	원자단위검증, 유일값 존재검증, 가공값 유무판단, 관리수준 상세화 판단
식별자 정의 (Identifier)	- 데이터 집합을 유일하게 식별해 주는 속성(PK로 구현)	PK, CK, AK, FK로 구분

논리적 모델링

1. 논리적 모델링 개요

□ 논리적 모델링의 정의

- 업무의 모습을 사람이 이해하기 쉽게 표현함
- 업무 중심적이면서 데이터 관점의 모델링
- 개념적 모델링 단계에서 추출된 관계를 논리적 데이터베이스 구조로 변환시키는 과정

□ 논리적 모델링의 특징

- 개념적 설계 과정에서 생성된 데이터 모델을 데이터베이스 종류의 특성에 따른 논리적인 모델로 변환
- 논리설계에서 데이터베이스 종류를 관계형 데이터베이스로 설계했다면 RDB에 부합하는 릴레이션 관계를 설정하고 정규화를 거쳐 논리 모델을 완성
- 저장구조나 인덱스 설계와 같은 물리적인 세부사항은 고려하지 않음

논리적 모델링

□ 논리적 모델링의 주요 Task

주요 Task		내용
속성 상세화		개념 데이터 모델링에서 추출된 핵심 속성외에 필요한 모든 속성 도출 - 최소 단위 까지 분할(분할 및 통합의 기준은 업무의 요구사항에 준함) - 하나의 값만(Single Value)를 가지는지 검증 - 추출 속성(Derived Attribute)아닌지 검증
개체 상세화	식별자 확정	엔티티를 유일하게 결정짓는 식별자를 확정함 - 각 인스턴스를 유일하게 식별하여야 하고 NULL이 될 수 없음 - 자주 변경되지 않는 것이어야 함 인조 식별자를 사용하는 경우에도 그 개체의 의미상의 주어가 되는 본질 식별자 구분
	정규화	논리적 데이터 모델을 일관성 있고 중복을 제거하여 데이터 무결성을 유지하기 위한 바람직한 자료 구조로 만들기 위한 과정 대체로 적절하고 일관성을 유지하면서 중복이 없는 논리 데이터 모델 도출을 위해 3차 정규형이 사용됨
	M:M 관계 해소	M:M 관계는 교차엔티티(Interaction Entity)를 도출하여 1:M 관계로 해소
	이력관리 결정	이력으로 관리할 대상 엔티티 결정 이력관리 형태를 결정

논리적 모델링

□ 논리적 모델링 전환 과정

절차	설명
Relation전환	개념 모델링에서의 Entity를 유일성을 보장하는 기본 키를 지정하여 실제 데이터가 저장될 논리적 Relation 으로 전환
Relationship(관계)	Entity간에 연결고리를 1:1 또는 1:M 형태로 전환
정규화 수행	데이터 중복 저장, 이상 현상 방지를 위해 1NF ~ BCNF 정규형까지 모두 만족하는지를 검사하고, 오류 발생시 이전 단계를 재 검토
사용자 트랜잭션 검증	도출된 논리적 데이터 모델이 사용자가 원하는 트랜잭션을 모두 만족시키는지를 확인함
ERD 검증	수정 및 보완해야 될 사항에 대해서 ERD에 재 반영
무결성 제약 정의	관계형 DB에서 무결성을 만족하기 위한 제약조건을 설정

물리적 모델링

1. 물리적 모델링 개요

□ 물리적 모델링 정의

- 논리 데이터 모델을 특정 DBMS에 맞는 물리적인 스키마를 만드는 일련의 과정
- 작성된 논리적 모델을 실제 컴퓨터의 저장 장치에 저장하기 위한 물리적 구조를 정의하고 구현하는 과정
- 테이블 관계 이외의 데이터베이스와 관련된 사항, 즉 오브젝트, 접근방법, 트랜잭션, 분석, 저장방법에 대해 설계

□ 물리적 모델링의 특징

- 하나의 릴레이션이 물리적으로 하나 이상의 테이블이 될 수 있음
- 논리적 모델링을 기초로 시스템 환경(하드웨어, 운영체제, 디스크 용량, 네트워크, DBMS 제품 등)을 고려하여 성능 향상을 목적으로 물리적 모델링 수행
- 시스템 환경이 변경되면 물리적 모델링도 변경
- 성능을 고려하여 통계 테이블이 추가되거나 백업이나 복제 용도의 테이블이 추가 가능

물리적 모델링

2. 물리적 모델링의 주요 업무 및 주요 Task

□ 물리적 모델링의 주요 업무

주요업무	설명
내부 스키마 정의	논리적 스키마(릴레이션 스키마)를 기초로 DBMS의 특성과 구현환경 등을 고려하여 내부 스키마(물리적 데이터 구조)를 정의
속성 정의	속성 즉, 테이블 컬럼의 데이터 타입과 크기, 제약 조건 등을 정의
역정규화 및 인덱스 설계	데이터 사용량의 예측을 통해 역정규화 및 인덱스 설계 작업 수행 (역정규화는 논리모델링 단계에서도 수행 가능)

물리적 모델링

□ 물리적 모델링의 주요 Task

단계	과정	고려사항
일괄전환	Entity별 Table로의 전환	Sub Type 설계 방안
	식별자의 Primary Key 정의	Artificial Key 검토, PK 컬럼 순서 검토
	속성의 컬럼 전환	영문 컬럼명 매핑, 데이터 타입/길이 결정, Domain 정의, 컬럼의 순서
	관계의 컬럼 전환	참조무결성 규칙 및 구현방향 결정
구조조정	수퍼타입/서브타입 모델 전환	트랜잭션의 성격에 따라 전체 통합, 부분 통합, 개별 유저에 대한 의사 결정을 통해 데이터 모델 조정
성능향상	성능을 고려한 반정규화	SQL 활용 능력의 미흡으로 인한 빈번한 비정규화는 배제하도록 신중하게 검토

감사합니다!



데이터베이스



4. 데이터 정규화

KPC 기본반

4. 데이터 정규화	53
이상현상(Anomaly)	54
함수적 종속성(Functional Dependency)	56
정규화(Normalization)	58
보이스-코드(BCNF) 정규화	66
4 차/5 차 정규화	68
연결함정(Connection Trap)	71
반정규화(De-Normalization)	73

이상현상(Anomaly)

1. 이상현상(Anomaly) 개요

□ 이상현상(Anomaly) 정의

- 특정 릴레이션에 대한 데이터를 조작함에 있어, 원치 않는 현상이 발생하거나 조작으로 인하여 데이터 불일치/누락이 발생하는 현상
- 자료의 모순에 따라 자료 운영에서 유발되는 비합리적인 현상 및 행위
- 불필요한 데이터 중복으로 인해 릴레이션에 대한 데이터 삽입·수정·삭제 연산을 수행할 때 발생할 수 있는 부작용

□ 이상현상(Anomaly)의 발생원인 및 해결 방안

구분	설명
발생원인	- 여러 가지 사실들을 하나의 릴레이션으로 표현 - 속성들 간에 존재하는 여러 가지 종속관계에 대하여 정규화 되지 않음
해결방안	- 애트리뷰트들 간의 종속성(dependency)을 분석해서 하나의 릴레이션에는 하나의 종속성이 표현되도록 분해 => 스키마변환(schema transformation)

이상현상(Anomaly)

□ 이상현상(Anomaly)의 종류

삽입 이상	새 데이터를 삽입하기 위해 불필요한 데이터도 함께 삽입해야 하는 문제
갱신 이상	중복 튜플 중 일부만 변경하여 데이터가 불일치하게 되는 모순의 문제
삭제 이상	튜플을 삭제하면 꼭 필요한 데이터까지 함께 삭제되는 데이터 손실의 문제

□ 이상현상 사례

고객아이디	이벤트번호	당첨여부	고객이름	등급
apple	E001	Y	정소화	gold
apple	E005	N	정소화	gold
apple	E010	Y	정소화	gold
banana	E002	N	김선우	vip
banana	E005	Y	김선우	vip
carrot	E003	Y	고명석	gold
carrot	E007	Y	고명석	gold
orange	E004	N	김용욱	silver

이상현상(Anomaly)

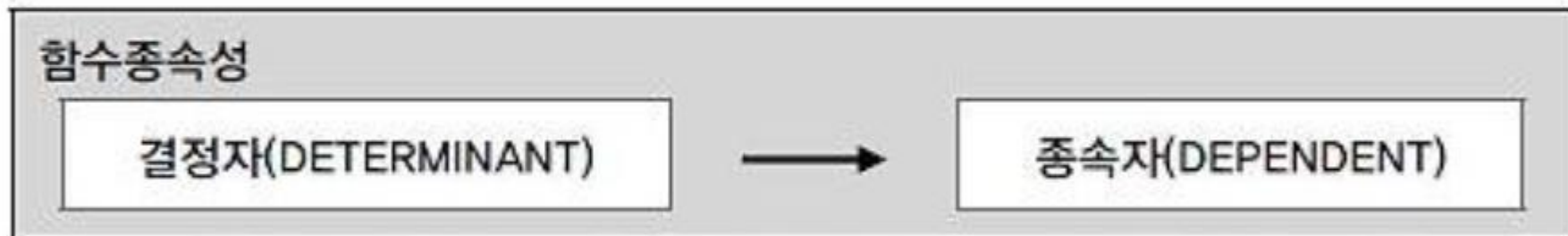
구분	설명
삽입이상	아직 이벤트에 참여하지 않은 아이디가 "melon" 이고, 이름이 "성원용", 등급이 "gold"인 신규 고객의 데이터는 이벤트참여 릴레이션에 삽입할 수 없음 삽입하려면 실제로 참여하지 않은 임시 이벤트번호를 삽입해야 함
갱신이상	아이디가 "apple"인 고객의 등급이 "gold"에서 "vip"로 변경되었는데, 일부 튜플에 대해서만 등급이 수정된다면 "apple" 고객이 서로 다른 등급을 가지는 모순이 발생
삭제이상	아이디가 "orange"인 고객이 이벤트 참여를 취소해 관련 튜플을 삭제하게 되면 이벤트 참여와 관련이 없는 고객아이디, 고객이름, 등급 데이터까지 손실됨

함수적 종속성(Functional Dependency)

1. 함수적 종속성(Functional Dependency) 개요

□ 함수적 종속성(Functional Dependency) 정의

- 데이터들이 어떤 기준 값에 의해 항상 종속되는 현상
- 속성집합 X의 값이 Y의 값을 유일하게 결정한다면 'Y는 X에 함수적으로 종속된다' 하며 $X \rightarrow Y$ 로 표기



종속자는 근본적으로 결정자에 함수적으로 종속성을 가지고 있음



이름, 출생지, 주소는 주민등록번호에 함수 종속성을 가지고 있음

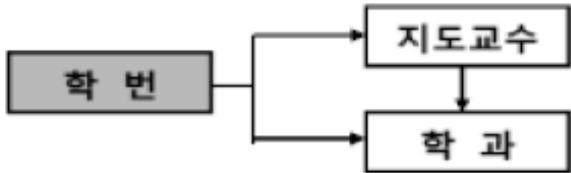
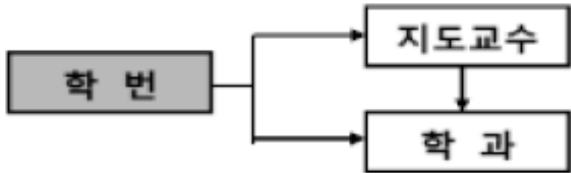
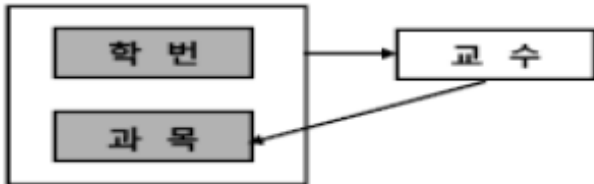
함수적 종속성(Functional Dependency)

□ 종속성(Dependency)의 유형

유형	내용
함수적 종속성	하나의 속성이 다른 속성을 결정 지음 릴레이션 R에서 $X \rightarrow Y$: X는 Y의 결정자, Y는 X의 종속자 완전 함수 종속, 부분 함수 종속, 이행 함수 종속 적용 : 1NF, 2NF, 3NF, BCNF
다중값 종속성	한 릴레이션에 둘 이상의 독립적인 다중값 속성이 존재하는 경우 이를 1NF로 전환하면 다중값 종속 발생 두개의 독립적인 다중값 속성을 서로 다른 두 개의 릴레이션으로 분리, 적용사례 : 4NF
결합 종속성	릴레이션 중에서 둘로 나눌 때에 원래의 관계를 회복할 수 없으나, 셋 또는 그 이상으로 분리시킬 때 원래의 복원할 수 있는 경우, 적용사례 : 5NF

함수적 종속성(Functional Dependency)

□ 함수적 종속성(Functional Dependency)의 종류

종류	내용	사례
완전함수 종속성	- $XY \rightarrow Z$ 일 때, $X \rightarrow Z$ 와 $Y \rightarrow Z$ 가 모두 성립 하지 않는 경우	학번, 과목번호 $\rightarrow$ 성적 
부분함수 종속성	- $XY \rightarrow Z$ 일때, $X \rightarrow Z$ 와, $Y \rightarrow Z$ 중 하나만 성립 - 제 2정규화 필요	학번 $\rightarrow$ 학과 
이행함수 종속성	- $X \rightarrow Y$ 이고, $Y \rightarrow Z$ 일 때 $X \rightarrow Z$ 가 성립 - 제 3정규화 필요	지도교수 $\rightarrow$ 학과 
결정자함 수 종속성	- 릴레이션이 모든 결정자가 후보키인 경우 결정자함수 종속임 - BCNF 필요	교수 $\rightarrow$ 과목 

정규화(Normalization)

1. 정규화(Normalization) 개요

□ 정규화(Normalization) 정의

- 이상현상을 야기하는 속성간의 종속관계를 제거하기 위해 릴레이션을 작은 여러 릴레이션으로 무손실 분해하는 과정
- 데이터의 이상현상(Anomaly)를 제거하기 위해 데이터의 함수적 종속성이나 조인 속성을 이용하여 분리, 통합하는 방법

□ 정규화(Normalization)의 목적

목적	내용
데이터의 중복 최소화	데이터 저장공간의 최소화
이상현상(Anomaly) 발생 방지	정보의 불일치 및 손실의 위험 최소화
데이터 구조의 안정성 유지	향후 새로운 요구에 유연하게 대처
종속성 제거	데이터의 일관성과 무결성 보장

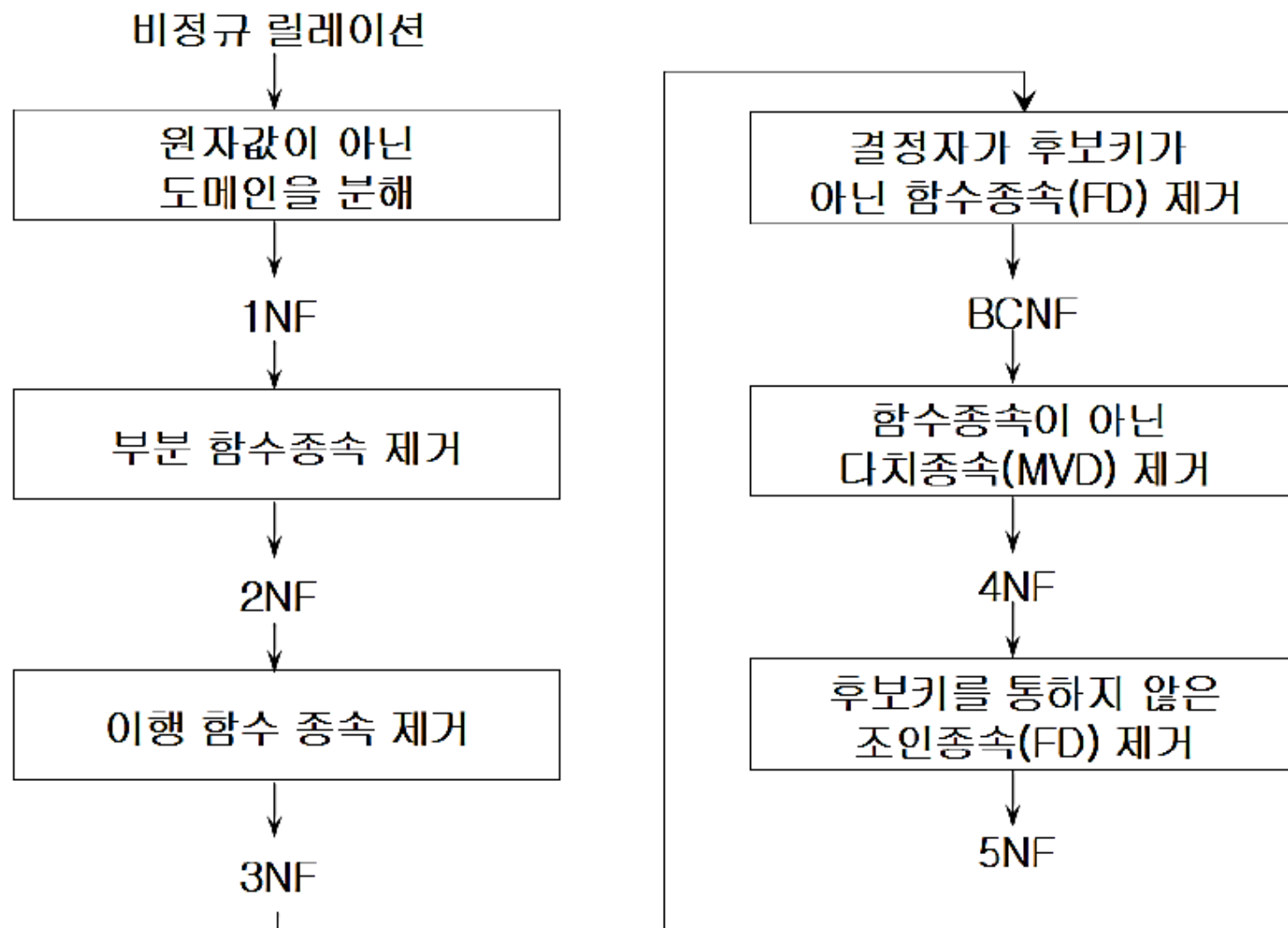
정규화(Normalization)

□ 정규화(Normalization)의 원칙★★

구분	내용
정보의 무손실	분해된 릴레이션이 표현하는 정보는 분해되기 전의 정보를 모두 포함하고 있어야 하며, 보다 더 바람직한 구조여야 함
데이터 중복성 감소	중복으로 인한 이상현상(Anomaly)의 제거
분리의 원칙	하나의 독립된 관계성은 하나의 독립된 릴레이션으로 분리하여 표현한다는 것

정규화(Normalization)

□ 정규화(Normalization) 절차



정규화(Normalization)

□ 정규화(Normalization)의 유형

구분	단계	내용
기초적 정규화	1차 정규화 (1NF)	· <u>반복되는 속성 제거</u> · 릴레이션 R에 속한 모든 도메인이 원자값(atomic value)만으로 되어있는 경우
	2차 정규화 (2NF)	· <u>부분함수 종속성 제거</u> · 릴레이션 R이 1NF이고 릴레이션의 기본키가 아닌 속성들이 기본키에 완전히 함수적으로 종속할 경우 · 결정자가 2개 이상일 때, 2개 중에서 하나의 결정자에 의해서만 함수종속성인 경우
	3차 정규화 (3NF)	· <u>이행함수 종속성 제거</u> · 릴레이션 R이 2NF이고 기본키가 아닌 모든 속성들이 기본키에 대하여 이행적 함수 종속성(Transitive FD)의 관계를 가지지 않는 경우, 즉 기본키 외의 속성들간에 함수적 종속적을 가지지 않는 경우 · 종속자들 간의 함수종속성이 있는 경우
	BCNF (Boyce/Codd NF)	· <u>결정자함수 종속성 제거</u> · 릴레이션 R의 모든 결정자가 후보키일 경우 · 결정자들 간의 함수적 종속이 있는 경우

정규화(Normalization)

진보적 정규화	4차 정규화 (4NF)	· <u>다중값 종속성 제거</u> 릴레이션 R에 MVD $A \twoheadrightarrow B$가 존재할 때 R의 모든 애트리뷰트들이 A에 함수종속 (즉, R의 모든 애트리뷰트 X에 대해 $A \rightarrow X$이고 A가 후보 키)이면 릴레이션 R은 제4정규형(4NF)에 속함 · BCNF를 만족시키면서 다중값 종속을 포함하지 않는 경우
	5차 정규화 (5NF)	· <u>결합 종속성 제거</u> 릴레이션 R에 존재하는 모든 조인 종속(JD)이 릴레이션 R의 후보키를 통해서만 성립된다면 릴레이션 R은 제5정규형(5NF) 또는 PJ/NF(Project-Join Normal Form)에 속한다. · 4NF를 만족시키면서 후보키를 통해서만 조인 종속이 성립되는 경우

정규화(Normalization)

□ 1차 정규화 정의

- 모든 엔티티 타입의 속성에는 하나의 속성값만을 가지고 있어야 하며 반복되는 속성의 집단은 별도의 엔티티 타입으로 분리
- 엔티티의 각 속성에 값이 반복집단이 없는 원자값(Atomic Value)으로만 구성되어 있어야 한다는 것을 의미

□ 1차 정규화 사례

1) 다중값 속성을 포함하는 이벤트참여 릴레이션

<u>고객아이디</u>	<u>이벤트번호</u>	당첨여부	등급	할인율
apple	E001, E005, E010	Y, N, Y	gold	10%
banana	E002, E005	N, Y	vip	20%
carrot	E003, E007	Y, Y	gold	10%
orange	E004	N	silver	5%

정규화(Normalization)

2) 다중값 속성을 제거하여 제 1 정규형을 만족하는 릴레이션

<u>고객아이디</u>	<u>이벤트번호</u>	당첨여부	등급	할인율
apple	E001	Y	gold	10%
apple	E005	N	gold	10%
apple	E010	Y	gold	10%
banana	E002	N	vip	20%
banana	E005	Y	vip	20%
carrot	E003	Y	gold	10%
carrot	E007	Y	gold	10%
orange	E004	N	silver	5%

정규화(Normalization)

3) 제 1 정규형은 만족하지만 이상현상이 발생하는 릴레이션

고객아이디	이벤트번호	당첨여부	등급	할인율
apple	E001	Y	vip	10%
apple	E005	N	vip	10%
apple	E010	Y	gold	10%
banana	E002	N	vip	20%
banana	E005	Y	vip	20%
carrot	E003	Y	gold	10%
carrot	E007	Y	gold	10%
orange	E004	N	silver	5%
grape	NULL	NULL	silver	5%

← 데이터 불일치로 인한 갱신 이상

← 데이터 손실로 인한 삭제 이상

← 삽입 불가로 인한 삽입 이상

정규화(Normalization)

4) 이상현상이 발생하는 이유

- 기본키인 {고객아이디, 이벤트번호}에 완전 함수 종속되지 못하고 일부분인 고객아이디에 종속되는 등급과 할인을 속성이 존재하기 때문

5) 문제해결방법

- 부분 함수 종속이 제거되도록 이벤트참여 릴레이션을 분해

정규화(Normalization)

4. 2차 정규화

□ 2차 정규화 정의

- 주 식별자에 종속적이지 않는 속성을 분리하여 부분함수 종속성을 제거
- 모든 속성이 주 식별자에 종속되도록 부분함수 종속성을 릴레이션으로 분해

□ 2차 정규화 사례

- 1) 제 1 정규형을 만족하지만 제 2 정규형은 만족하지 않는 릴레이션
 - 기본키에 완전 함수 종속이 되지 않는 등급과 할인을 속성 때문

정규화(Normalization)

2) 고객 릴레이션과 이벤트 참여 릴레이션으로 분리

분해 전의 이벤트참여 릴레이션

<u>고객아이디</u>	<u>이벤트번호</u>	당첨여부	등급	할인율
apple	E001	Y	gold	10%
apple	E005	N	gold	10%
apple	E010	Y	gold	10%
banana	E002	N	vip	20%
banana	E005	Y	vip	20%
carrot	E003	Y	gold	10%
carrot	E007	Y	gold	10%
orange	E004	N	silver	5%

부분 함수 종속을 제거하려고 분해

정규화(Normalization)

고객 릴레이션

<u>고객아이디</u>	등급	할인율
apple	gold	10%
banana	vip	20%
carrot	gold	10%
orange	silver	5%

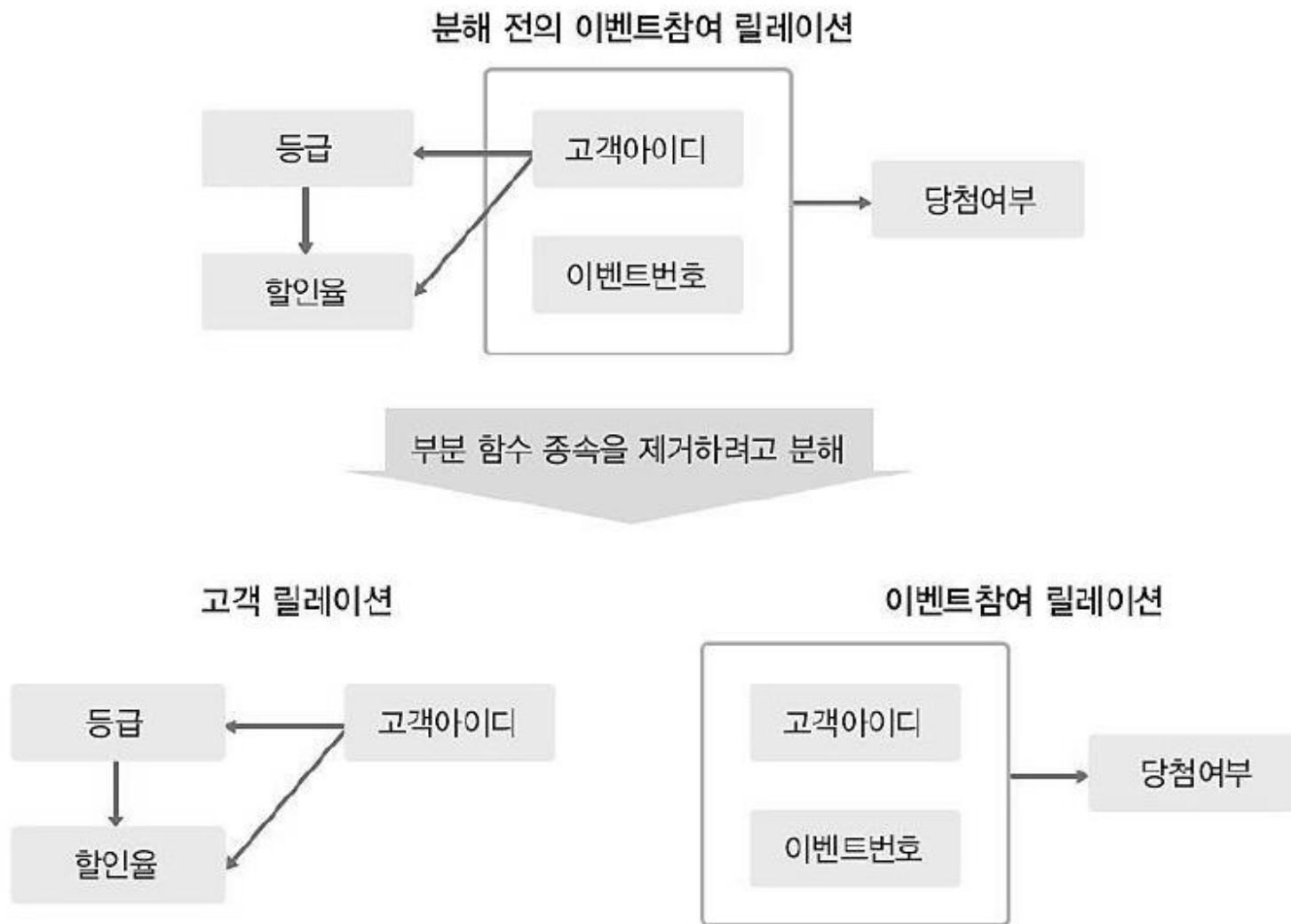
이벤트참여 릴레이션

<u>고객아이디</u>	<u>이벤트번호</u>	당첨여부
apple	E001	Y
apple	E005	N
apple	E010	Y
banana	E002	N
banana	E005	Y
carrot	E003	Y
carrot	E007	Y
orange	E004	N

- 고객 릴레이션과 이벤트참여 릴레이션은 모두 제 2 정규형에 속함

정규화(Normalization)

3) 고객 릴레이션과 이벤트참여 릴레이션의 함수 종속 다이어그램

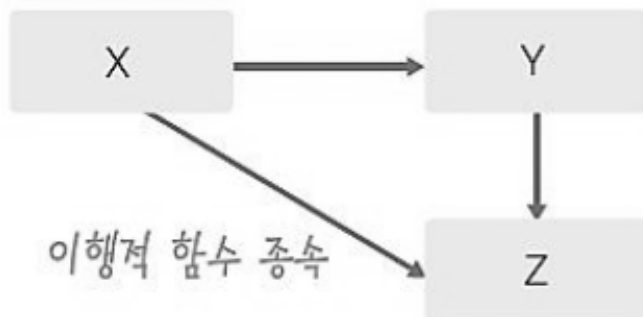


정규화(Normalization)

5. 3차 정규화

□ 3차 정규화 정의

- 식별자가 아닌 속성에 종속적인 속성의 릴레이션으로 분리하여 이행함수적 종속성을 제거
- 식별자가 아닌 모든 속성은 서로 간에 종속될 수 없음



릴레이션을 구성하는 세 개의 속성 집합 X, Y, Z 에 대해 함수 종속 관계 $X \rightarrow Y$ 와 $Y \rightarrow Z$ 가 존재하면 논리적으로 $X \rightarrow Z$ 가 성립되는데, 이것을 Z 가 X 에 이행적으로 함수 종속되었다고 함

정규화(Normalization)

□ 3 차 정규화 사례

1) 제 2 정규형은 만족하지만 이상현상이 발생하는 릴레이션

- 이상현상이 발생하는 원인은 이행 함수적 종속이 존재하기 때문

고객 릴레이션

고객아이디	등급	할인율
apple	gold	10%
banana	vip	20%
carrot	gold	10%
orange	silver	5%

고객아이디	등급	할인율
apple	gold	15%
banana	vip	20%
carrot	gold	10%
orange	silver	5%
NULL	bronze	1%

← 데이터 불일치로 인한 갱신 이상

← 데이터 손실로 인한 삭제 이상

← 삽입 불가로 인한 삽입 이상

정규화(Normalization)

2) 이행함수 종속을 제거하기 위해 2 개의 릴레이션으로 분리

분해 전의 고객 릴레이션

고객아이디	등급	할인율
apple	gold	10%
banana	vip	20%
carrot	gold	10%
orange	silver	5%

이행적 함수 종속을 제거하려고 분해

고객 릴레이션

고객아이디	등급
apple	gold
banana	vip
carrot	gold
orange	silver

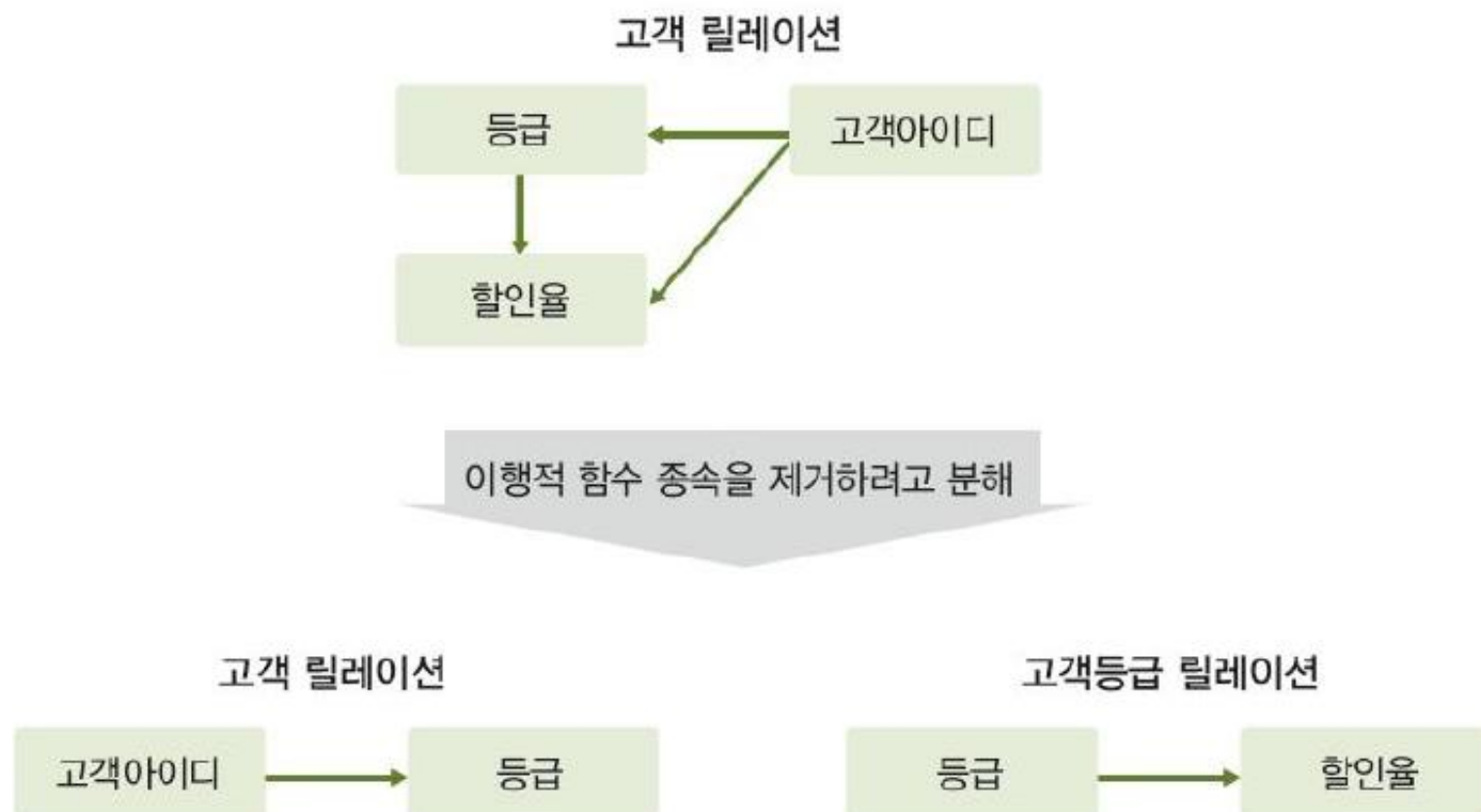
고객등급 릴레이션

등급	할인율
gold	10%
vip	20%
silver	5%

- 고객 릴레이션과 고객등급 릴레이션은 모두 제 3 정규형에 속함

정규화(Normalization)

3) 함수적 종속 다이어그램



정규화(Normalization)

6. 정규화의 문제점 및 해결방안

□ 정규화의 문제점

- 빈번한 Join 연산의 증가로 시스템 성능 저하
- 부자연스러운 DB Semantic 초래
- 조회/검색 위주의 응용시스템에 부적합

□ 문제점 해결방안

- 정규화 완료 후 업무 특성과 성능 향상을 위해 Denormalization 수행으로 유연성 확보
- 업무 특성별 정규화 수준
(온라인 처리: 소규모 정규화, 단순 조회용 데이터 : 비정규화, 배치처리 : 비정규화)
- 응답시간이 요구되는 온라인, 실시간 시스템에는 제한적인 정규화 실시

연결함정(Connection Trap)

1. 연결함정(Connection Trap) 개요

□ 연결함정(Connection Trap) 정의

- 정규화 과정에서 무손실 분해의 원칙이 지켜지지 않아 원래 있던 관계성을 잃어버리는 현상
- 데이터 모델링시 개체와 개체 사이에 부여하는 관계성 집합의 의미가 모호하여 원하는 결과를 얻을 수 없거나 향후 업무처리에 영향을 미치게 되는 ER 모델의 문제점

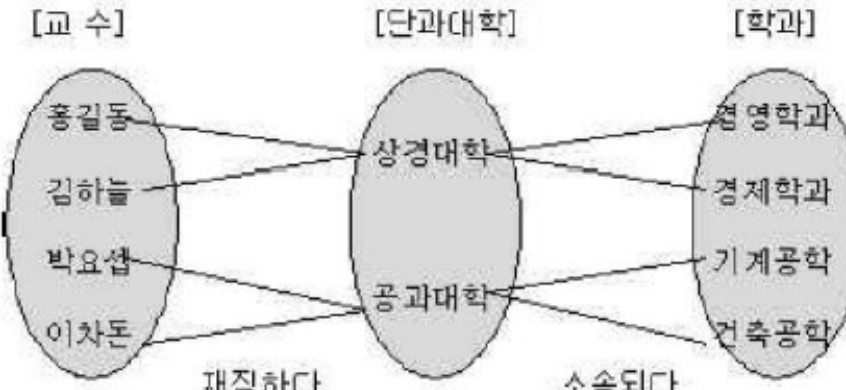
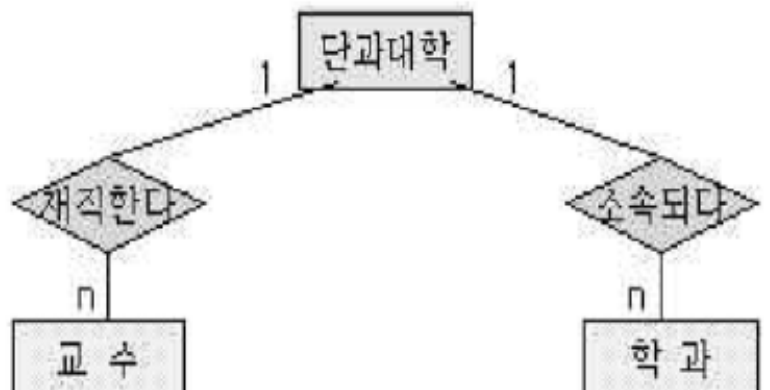
□ 연결함정(Connection Trap)의 유형

종류	내용	원인
부채꼴 함정 (Fan Trap)	개체 집합(Entity Set) 사이에 관계성집합 (Relation set)이 정의되어 있기는 하지만 관계성 예시가 모호한 경우	관계성 모호
균열함정 (Chasm Trap)	개체 집합(Entity Set) 사이에 관계성집합(Relation set)이 정의되어 있기는 하지만 일부 개체집합과 개체집합 사이에 관계성이 존재하지 않는 경우	관계성 미존재

연결함정(Connection Trap)

2. 부채꼴함정(Fan Trap)과 균열함정(Chasm Trap)

□ 부채꼴 함정(Fan Trap)

구분	개념도	ER 모델
문제점		
	- 한 개의 개체집합(Entity set)이 둘 이상의 1:n 관계성 집합을 갖는 경우로 마치 부채꼴 형상을 나타내고 있음 - 어느 교수가 어느 학과에 재직하고 있는지 알고자 할 경우 알 수 있는 방법없음 - 예를 들면 홍길동이라는 교수는 상경대학에 재직하고 있으나 경영학과 교수인지 경제학과 교수인지 알 수 있는 방법이 없음	

연결함정(Connection Trap)

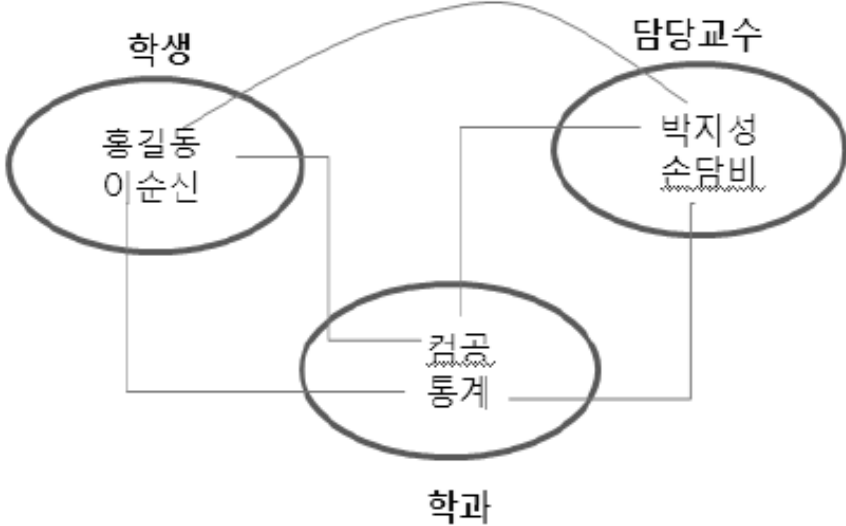
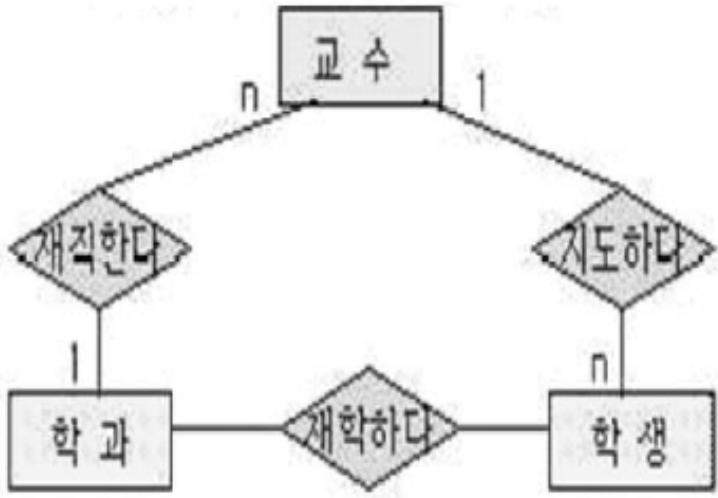
해결 방안	단과대학 상경대학 공과대학 학과 경영학과 경제학과 기계공학 건축공학 교수 홍길동 김하늘 박요셉 이차돈 소속되다 재직하다	학과 소속되다 단과대학 재직하다 교수
	하나의 엔티티 중심에서 1:n으로 펼쳐지는 관계 중 한쪽 관계를 n:1 로 변환시켜 해결함. 위의 사례에서 단과대학을 중심으로 교수라는 엔티티와 학과라는 엔티티가 1:n 의 관계성 집합으로 부채를 얹어 놓은 것처럼 되어있으나 학과 엔티티를 중심으로 n:1, 1:n으로 변환	

연결함정(Connection Trap)

균열함정(Chasm Trap)

구분	개념도	ER 모델
문제점	[학과] 경영학과 경제학과 [교수] 김유신 김하늘 홍길동 이차돈 [학생] 김춘추 이순신 박문수 강태공	교수 재직한다 지도하다 학과 학생
	- 개체집합(Entity set)들이 부분 관계성 집합으로 연결된 경우 발생 - 학생이 어느 학과에 소속되는지는 교수라는 엔티티를 통해 알 수 있는 것처럼 보여 문제가 없는 것처럼 생각됨 - 그러나 만약 어느 학생이 지도교수를 할당 받지 못한 즉 복학생인 경우 어느 학과에 속하는지 알 수 있는 방법 없음	

연결함정(Connection Trap)

해결 방안		
	- 균열이 발생한 엔티티간에 새로운 관계를 추가하여 해결함. - 위의 사례에서 '재학하다'라는 관계성 집합을 '학과'라는 개체 집합과 '학생'이라는 개체집합 사이에 배열시켜 해결	

□ 연결함정 방지를 위한 고려사항

- 정규화 수행 이후 비즈니스 시나리오를 통한 모델 검증으로 누락되는 정보의 파악이 필요함
- M:N 관계의 분해 시 3항 관계 이상의 관계성이 발생하면 관계 간의 확인 과정을 통해 모호성 여부를 검증해야 함

반정규화(De-Normalization)

1. 반정규화(De-Normalization) 개요

□ 반정규화(De-Normalization) 정의

- 정규화로 분해된 데이터 모델을 관련 있는 릴레이션으로 통합하여 DB 성능을 향상시키는 기법
- 정규화된 엔터티, 속성, 관계에 대해 시스템의 성능향상과 개발(Development)과 운영(Maintenance)의 단순화를 위해 중복, 통합, 분리 등을 수행하는 데이터 모델링의 기법

□ 반정규화(De-Normalization)의 필요성

필요성	설명
검색 성능 향상	정규화로 분해된 테이블 간 Join 증가로 검색 성능 저하
모델링 이해 증대	분해된 테이블 간 복잡한 릴레이션으로 모델링 이해의 어려움
개발 생산성 증대	SQL 개발 시 해당 테이블의 잦은 조인으로 개발 생산성 저하

반정규화(De-Normalization)

□ 반정규화(De-Normalization)의 장단점

장점	- Join 횟수를 줄일 수 있어 검색 성능 향상 - 개발 생산성 및 운영의 편의성 향상 - 모델링 이해의 용이
단점	- 데이터 신규 생성, 변경 시 성능 저하 - 동일 데이터의 분산으로 데이터 정합, 무결성, 저하 우려 - 분산된 데이터 관리의 어려움

반정규화(De-Normalization)

2. 반정규화(De-Normalization) 절차 및 주요 기법

□ 반정규화(De-Normalization) 절차

순서	단계	내용
1	반정규화 대상 조사	범위 처리 빈도수 조사, 대량의 범위 처리 조사, 통계성 프로세스 조사, 테이블 조인 개수
2	다른 방법 유도 검토	뷰(View) 테이블, 클러스터링 적용, 인덱스 조정, 응용 어플리케이션
3	반정규화 적용	테이블 반정규화, 속성의 반정규화, 관계의 반정규화

반정규화(De-Normalization)

□ 반정규화(De-Normalization) 주요기법

구분	주요기법	설명
테이블 레벨	테이블 병합	• 1:1 병합, 1:N 테이블 병합(Join 수 감소) • Sub-type 끼리 테이블 병합: 테이블 Sub-type 속성 삽입 • 코드 테이블 병합: 코드명을 속성으로 추가하여 Join 제거
	테이블 분할	• 테이블 수직 분할: 잦은 Access 컬럼과 낮은 컬럼을 분리 • 테이블 수평 분할: 동일한 컬럼을 데이터를 기준으로 수평 분할, Partition 적용으로 성능 향상
	테이블 추가	• 통계 테이블 추가: 통계 정보를 위해 조회 빈도가 높은 컬럼을 테이블로 생성 • 이력 테이블 추가: 이력성 정보 조회를 위한 별도 테이블 생성 • 중복 테이블 추가, 부분 테이블 추가 등

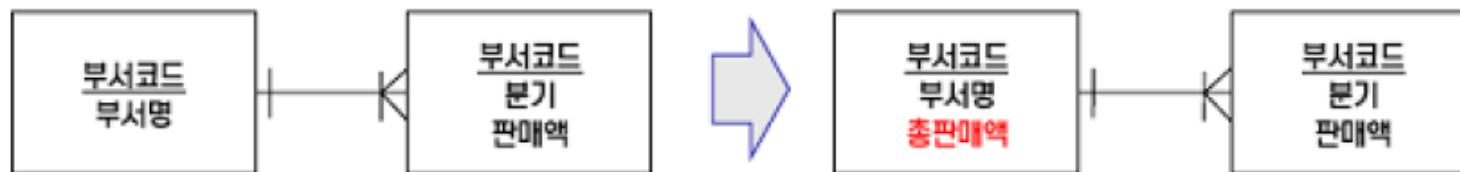
반정규화(De-Normalization)

컬럼 레벨	중복 컬럼 추가	• 갱신보다 조회 성능이 더 중요시 양쪽 테이블에 중복 저장 • 장단점: 조인의 비용은 감소하나, 갱신 비용은 증가
	계산 컬럼 추가	• 여러 테이블 조인 후 계산이 필요한 경우 계산 결과 컬럼 추가 • 장단점: 조인의 비용은 감소하나, 갱신 비용은 증가
	이력 컬럼 추가	• 변경 이력, 발생 이력 정보를 위한 최신 정보 컬럼 추가
관계 레벨	중복관계 추가	• 부모-자식 관계를 추가하여 데이터 접근경로 단축

반정규화(De-Normalization)

□ 반정규화(De-Normalization) 사례

(1) 계산 컬럼의 추가



(2) 테이블 분할 (수직분할)

Key	C1	C2	C3	C4	C5	C6

테이블 A



Key	C1	C2	C3

테이블 A1

Key	C4	C5	C6

테이블 A2

(3) 테이블 분할 (수평분할)

년도	C1
2001	
2002	
2003	
2004	
...	



년도	C1
2001	
2002	

년도	C1
2003	
2004	

반정규화(De-Normalization)

□ 반정규화(De-Normalization) 수행 시 고려사항

고려사항	설명
무결성 고려	데이터의 무결성을 유지하는 범위 내에서 성능 향상을 위한 반정규화 필요
테이블 용도 고려	조회는 빈번하지만 갱신은 적게 발생하는 테이블 위주로 반정규화
정규화 선 진행	기본 정규화를 진행한 이후 성능 저하 요소를 식별하여 반정규화 수행
변경이력 관리	정규화 테이블에서 반정규화 수행 사유 기록으로 업무 추적 가능

감사합니다!



데이터베이스



5. 데이터베이스 프로그래밍

KPC 기본반

5. 데이터베이스 프로그래밍.....	75
옵티마이저(Optimizer)	76
데이터베이스 성능 튜닝	81
데이터베이스 리팩토링	86

옵티마이저(Optimizer)

1. 옵티마이저(Optimizer)의 개요

□ 옵티마이저(Optimizer)의 정의

- 사용자가 질의한 SQL 문의 처리 가능한 실행계획을 탐색하고 각 실행계획에 대한 처리 비용을 추정하여 최적의 실행계획을 수립하는 DBMS의 성능 튜닝 엔진
- Select, Update, Insert, Delete 와 같은 SQL 명령에 대한 최적의 실행계획을 선택하는 역할을 하는 프로세스
- 대표적인 옵티마이저 유형으로 CBO(Cost Based Optimizer)와 RBO(Rule Based Optimizer)가 있음

□ PLAN_TABLE

- 데이터베이스 사용자가 PLAN_TABLE 을 생성해 두었다면, 옵티마이저는 SQL 문의 실행될 때마다 우선 실행계획(Execution Plan)을 작성하여 PLAN_TABLE 에 저장
- 사용자는 PLAN_TABLE 에 저장된 실행계획(execution plan)을 통하여 SQL 문의 실행에 대한 옵티마이저의 경로 확인 가능

옵티마이저(Optimizer)

2. 관계형 DB 의 성능을 결정하는 옵티마이저의 분류

□ RBO(Rule Based Optimizer)

항목	설명
개념	-조건문의 syntax에 따라 조건범위(Selectivity)를 미리 결정해 놓고 이를 기반으로 실행계획을 수립 - 인덱스 구조나 비교연산자에 따른 우선순위를 기준으로 최적 경로를 설정하는 옵티마이저
특징	- 판단이 매우 규칙적이고 분명하여 사용자가 정확히 예측
RBO의 Rule (우선 순위)	번호가 낮을수록 좋은 성능을 내는 조건이기 때문에 우선순위가 높게 부여됨 01. ROWID에 의한 단일 행 액세스 02. Cluster Join에 의한 단일 행 액세스 03. Unique-Key, Primary-Key를 사용한 HASH Cluster Key에 의한 단일 행 액세스 04. Unique-Key, Primary-Key에 의해 생성된 Unique INDEX에 의한 단일 행 액세스 05. CLUSTER 조인 06. Non Unique HASH Cluster Key 07. Non Unique Cluster Key(인덱스화된 Cluster-Key) 08. NON UNIQUE 결합 인덱스

오라클 11g 이후로는
기술지원 중단

옵티마이저(Optimizer)

	09. 단일 칼럼 인덱스에 의한 동등 검색 10. 인덱스가 구축된 칼럼에 대한 제한된 범위 검색(BETWEEN, LIKE, < AND >, = 표현식) 11. 인덱스가 구축된 칼럼에 대한 무제한 범위의 검색(>=, =<표현식) 12. 정렬-병합 조인 13. 인덱스가 구축된 칼럼에 대한 MAX, MIN 14. 인덱스가 구축된 칼럼에 대한 ORDER BY 15. 전체 테이블 스캔
예시	- emp테이블에 'A' 인덱스가 "deptno"로 구성, 'B' 인덱스가 "deptno+empno"로 구성 시 다음의 SQL은 Bounded Range Search(Between)이므로 'A' 인덱스를 사용. - 즉, 'A' 인덱스는 Rank 9이고 'B' 인덱스는 Rank 10 조건이 되므로. <pre>select /*+ rule */ from emp where deptno = 10 and empno between 7888 and 8888;</pre>

옵티마이저(Optimizer)

□ CBO(Cost Based Optimizer)

항목	설명
개념	- 처리방법들에 대한 비용을 산정해보고 그 중에서 가장 적은 비용이 들어가는 처리방법을 선택하는 옵티마이저
특징	- 현실을 감안한 판단, 통계정보의 관리를 통한 최적화 제어, 옵티마이저를 깊이 이해하고 있지 않더라도 최소 성능 보장 - 실행계획을 미리 예측, 제어가 어려움
원리	- 여러가지 경우의 처리시간에 비례한 비용을 산출해 내고, 이들 비용에서 가장 작은 비용을 갖는 플랜을 결정 - 실행계획에 대한 소요시간을 구하기 위해서 데이터 분포도 및 테이블, 인덱스와 클러스터의 저장 공간의 특징 등의 통계 정보를 이용하여 비용을 계산하여 선택하는 방법 - 특정 실행계획을 통한 SQL 명령문을 실행하는데 필요한 메모리, 디스크 I/O, CPU 시간을 계산

옵티마이저(Optimizer)

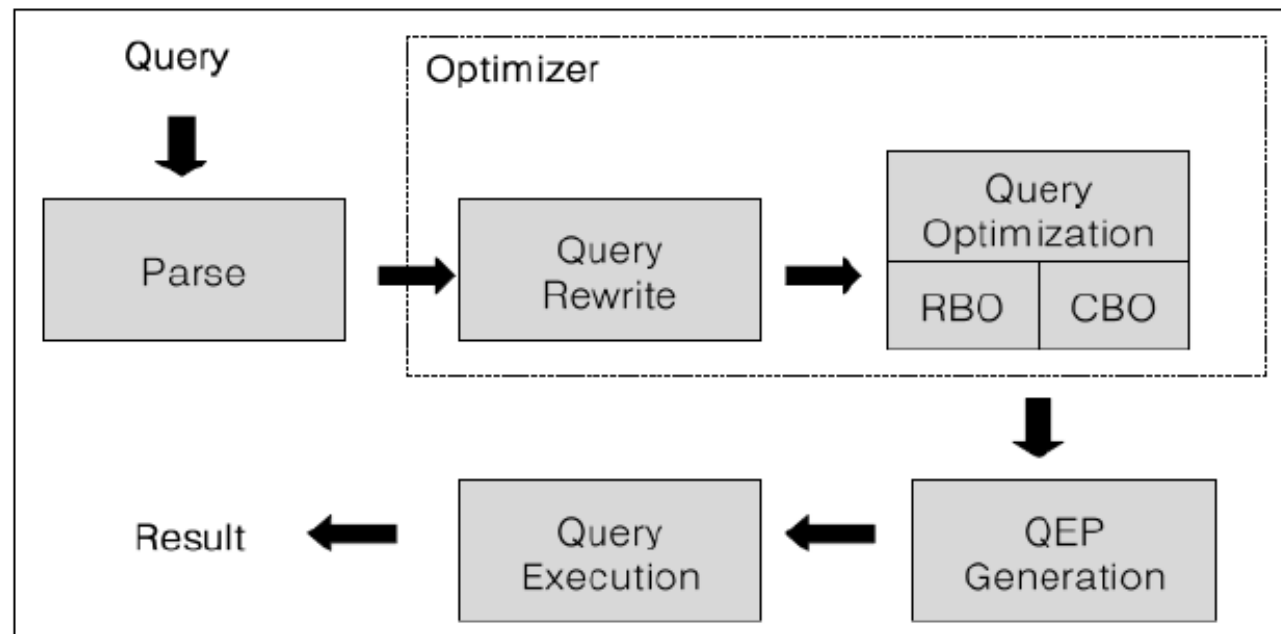
예시

- Select * from dept where deptno= 10의 SQL에서 dept테이블은 deptno에 대한 인덱스가 있고 테이블은 전체 10블록으로 구성되어 있으며 풀 테이블 스캔일 경우 I/O 단위를 결정하는 파라미터가 db_file_multiblock_read_count=8 이라면 RBO는 무조건 인덱스를 타는 플랜을 결정
- CBO는 deptno의 인덱스를 이용해 실행하면 3회의 I/O가 발생한다고 가정하고 풀 테이블 스캔의 경우 2회(8블록+2블록)의 I/O가 발생한다면 인덱스가 있음에도 풀테이블 스캔을 선택함

옵티마이저(Optimizer)

3. 관계형 DB 에서 옵티마이저의 역할 및 CBO 와 RBO 비교

□ 관계형 DB 의 질의 처리 단계에서 옵티마이저의 역할



- 질의처리는 5 단계로 구분할 수 있고, 옵티마이저는 Query Rewrite 단계와 Query Optimization 단계에 참여함

옵티마이저(Optimizer)

질의처리 단계	처리내용
Query Rewrite	- 질의변환기 - 서브질의와 뷰의 병합을 수행하고, OR Expansion 작업을 수행 - 서브질의와 뷰 병합이란, 옵티마이저가 더욱 효과적인 QEP를 찾기 위하여 더 효과적인 플랜이 있는지 그 가능성을 확인하는 과정
Query Optimization	- 비용산정기 - 질의에 대한 액세스 경로를 결정
QEP Generation	- 실행계획 생성기 - 질의실행계획(QEP:Query Execution Plan): 질의를 실행하는데 필요한 상세한 정보를 생성

옵티마이저(Optimizer)

1) CBO 와 RBO 의 주요 특징 비교

구분	CBO	RBO
특징	- 실행계획을 미리 예측하기 힘들고, 실행계획을 특정한 처리경로로 유도하는 등의 제어가 어려움 - 옵티마이저의 최적화된 결정을 위해서는 통계정보의 갱신 필요	- 통계정보를 사용하지 않아 절대적으로 INDEX정보를 활용 - SQL문 실행시 사전에 정의된 규칙에 따라 실행 - 사후에 RBO 규칙을 변경하기가 매우 어려움
경로 선택	- 처리 방법에 대한 비용을 산정한 후 최소 비용이 소요되는 방법 선택	- 인덱스 구조나 비교 연산자에 따른 우선순위를 기준으로 최적 경로 설정
동작 (계산) 방식	-시스템 통계정보(CPU, 디스크엑세스 타임)를 이용해서 처리시간으로 환산한 방식을 사용 - SQL문장에 대한 여러 가지 경우의 수별로 처리시간에 비례한 비용을 산출해내고 이들 비용에서 가장 작은 비용을 갖는 플랜을 결정	- 크게 ROWID를 이용한 직접 접근, INDEX 를 이용한 Random Access, Full Table Scan 을 이용하는 Sequential Scan으로 구분 - 일반적으로 우선순위가 높은 오퍼레이션은 우선순위가 낮은(높은 숫자의) 오퍼레이션보다 더 빨리 실행 - 각 경로의 분류는 하나 이상의 경로가 유용할 때 한 경로를 선택하기 위한 규칙-기반 접근 방식에 의해 사용

옵티마이저(Optimizer)

장점	- 현실을 감안한 판단, 통계정보의 관리를 통한 최적화 제어 - 옵티마이저를 깊이 이해하고 있지 않더라도 최소 성능 보장	- 판단이 매우 규칙적이고 분명하여 수립될 실행계획이 예측 가능하기 때문에 사용자가 원하는 처리 경로로 유도하기에 용이
단점	- 실행 계획을 미리 예측, 제어가 어려움	- 통계 정보라는 현실요소를 무시하여 판단의 오차가 크게 날 수도 있음

옵티마이저(Optimizer)

2) CBO 와 RBO 의 Plan 선택 측면의 비교

쿼리	CBO의 Plan 선택방법	RBO의 Plan 선택방법
<pre>Select * from empwhere job = 'teacher' and empno = '123';</pre>	- 반드시 Index Merge만을 쓰는 것이 아니라 분포도가 좋은 Index 사용가능	- 'and'를 중심으로 양쪽이 모두 '='이므로 Index Merge 사용
<pre>Select * from empwhere ename like '홍길%' and empno = '123';</pre>	- 분포도에 따라 ename Index도 사용가능	- '='이 'like'보다 연산자 우선 순위가 높기 때문에 항상 empno Index만 사용

데이터베이스 성능튜닝

1. 데이터베이스 성능 개선 방안 개요

□ 데이터베이스 성능 튜닝의 정의

- 데이터베이스 응용, 데이터베이스 자체, 그리고 운영체제의 조정 등을 통하여 최적의 자원으로 최적의 성능(시간/응답속도)를 얻을 수 있는 작업
- 최종 사용자인 고객관점에서 빠른 업무 처리를 지원하기 위해 데이터베이스가 처리 성능을 보장하는 모든 개념

□ 데이터베이스 성능 개선을 위한 주요 항목

주요요소	설명	사례
설계관점 (모델링관점)	- 데이터 모델링, 인덱스 설계 - 데이터파일, 테이블 스페이스 설계 - 데이터베이스 용량 산정	- 반정규화 - 분산파일 배치
DBMS관점	- CPU, 메모리 I/O에 관한 관점	- Buffer, Cache크기
SQL관점	- join, Indexing, SQL Execution Plan	- Hash / Join
H/W관점	- CPU, Memory, Network, Disk	- System Resource개선

데이터베이스 성능튜닝

2. 데이터베이스 성능 튜닝의 절차 및 튜닝의 3요소

□ 데이터베이스 성능 튜닝 절차

절차	설명	수행활동
시스템 성능 진단	- 자료수집: AS-IS 도출을 위한 기초자료 수집 - 목표설정: 문제점 파악 및 대상에 대한 목표 구체화, 튜닝 방향 설정	- 사용자 인터뷰, 설계 검토 - 시스템 구성 검토 - 자원사용현황분석 - SQL Trace 분석
튜닝 및 분석	- 진단 결과 도출된 문제 요소에 대하여, 최적화 방안을 수립 및 적용	설계내용 튜닝, DBMS 튜닝, SQL 튜닝, OS, H/W 튜닝
결과평가	- 1차 튜닝 방안 적용 후 2차 테스트 분석이나 결과 평가를 통한 최종안 적용 및 개선효과 보고자료 작성	- 튜닝 후 자료수집/분석 - 튜닝결과 평가 - 산출물 작성

데이터베이스 성능튜닝

□ 데이터베이스 성능 튜닝의 3요소

구분	설 명	주요 기법
DB 설계 (모델링)	- 데이터베이스 설계 단계에서 성능을 고려하여 설계	-Table 분할 및 통합, 식별자 - 지정, 인덱스 전략
DB 환경 (DBMS)	- DBMS config 파라미터 조정 - DBMS 자원 활용 최적화	-CPU 튜닝, 메모리 튜닝 -I/O 튜닝, 네트워크 튜닝
SQL 문 튜닝	- 성능을 고려하여 SQL 문장을 작성	인덱스 활용, 조인방식/조인순서

데이터베이스 성능튜닝

3. 데이터베이스 성능 튜닝 상세

□ 데이터베이스 설계 측면에서의 성능개선

튜닝 기법	내용	기대 효과
테이블의 분할/통합	- 논리적 통합, 파티션 기능 적용한 테이블 구성 - 테이블의 수평 분할 수행 파티션 기능 불가시) - 액세스 패턴에 따른 1:1 수직 분할을 고려	- 액세스 효율 - DB I/O 개선
식별자 지정	- 본질 식별자와 인조식별자의 선택에 따라 정보의 상속과 단절에 영향을 줄 수 있음	- 상속 개선 - 단절 최소화
효율적인 인덱스 전략	- 인덱스 구조 재설계: 최소한의 인덱스로 최대의 효과 - 인덱스 분포도 15% 이하 유용 유도 (15% 이상이면 Full Scan 유리)	- 인덱스 분포대비 전략 수립 개선 - 인덱스 최소화
적절한 데이터 타입 선택	- 컬럼 데이터의 내부적인 변형에 의한 데이터 타입이 다른 경우 - 인덱스 활용하지 못하고, 조인순서나 조인방식을 달리 선택하는 비효율 발생할 수 있음	- 내부 변형 관리 - 비효율성 개선

데이터베이스 성능튜닝

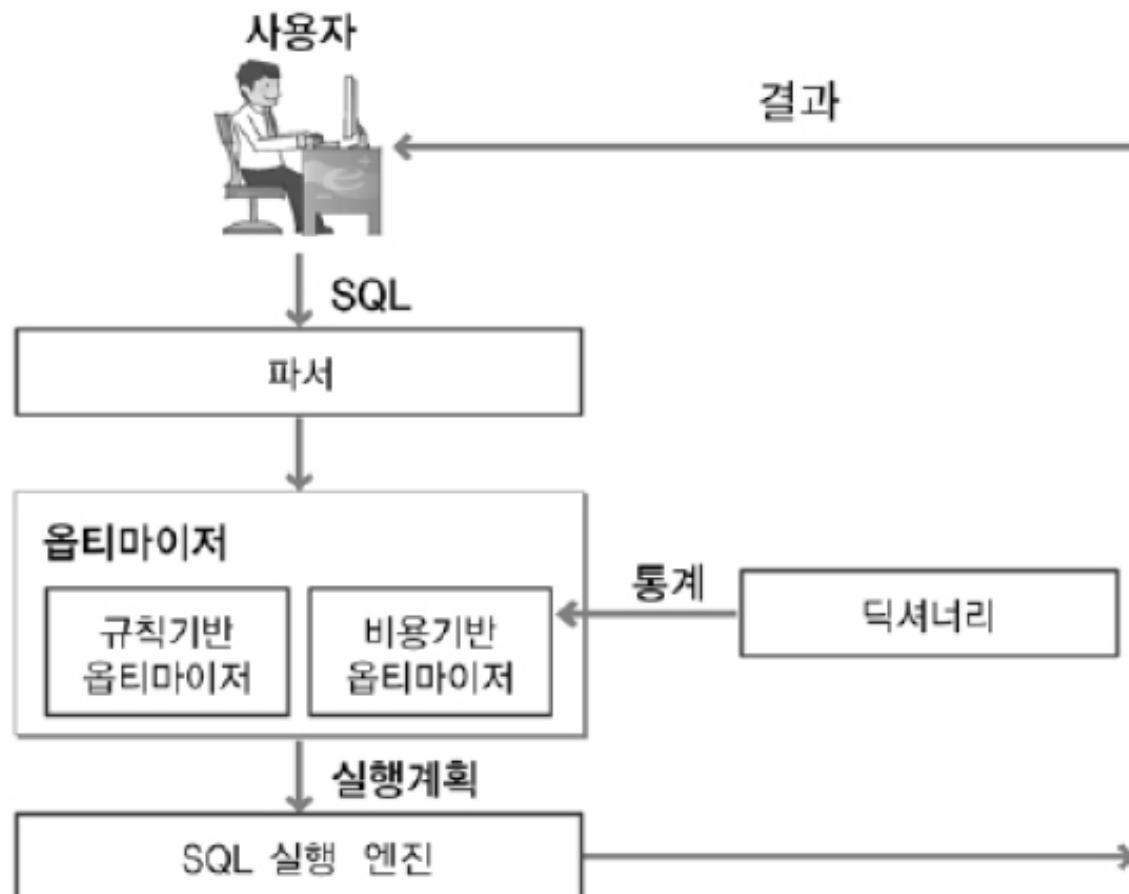
□ DBMS 환경 측면에서의 성능개선

구분	내용
CPU 튜닝	Peak Time 시에 60~70%의 사용량 유지 권고 CPU 증설 또는 CPU 과다 점유하는 프로세스를 찾아서 해결
메모리 튜닝	DBMS 의 사용 메모리의 최적화 필요 업무를 가능하게 하기 위한 충분한 메모리 확보 DBMS 메모리 + Session 메모리
I/O 튜닝	I/O 를 분산시킬 수 있도록 데이터 파일의 재배포 RAID 를 이용 -> I/O 가 많이 일어나는 것은 Raid0,1 에 배치
네트워크 튜닝	Ping, Ftp 를 이용하여 응답시간 분석
작업수행 방식 개선	우선순위, Parallel Processing 구현, Data Sharing 구현

데이터베이스 성능튜닝

□ SQL 튜닝

1) SQL 처리과정의 이해



데이터베이스 성능튜닝

구성요소	기능
파서(Parser)	SQL 문의 구문 구조와 의미를 분석 구문 구조 분석(Syntax analysis) : SQL 문이 올바르게 작성되었는지 검사 의미 분석(Semantic analysis) : 데이터베이스 객체나 참조된 객체들이 올바른지 검사
옵티마이저(Optimizer)	사용자가 실행한 SQL 문을 분석하여 가장 빠르게 실행할 수 있는 방법을 결정해주는 프로세스 규칙 기반 옵티마이저(RBO) : 미리 정해져 있는 규칙에 따라 가장 빠른 실행 방법을 결정 비용 기반 옵티마이저(CBO) : 실제로 가장 빠르게 실행되는 방법을 비용(cost)를 계산하여 선택
행 자원 생성기(Row Source Generator)	옵티마이저로부터 최적의 계획을 받아 SQL 문에 대한 실행 계획을 출력하는 것으로, 실행 계획은 트리 형태의 구조화된 행들의 집합을 의미
SQL 실행 엔진(SQL Execution Engine)	SQL 문과 관련된 실행 계획을 수행하는 구성 요소로, 질의에 대한 결과를 생성하는 것으로 행 소스 생성기에 의해 생성된 각 행들이 SQL 실행 엔진에 의해 실행

데이터베이스 성능튜닝

4. 데이터베이스 성능 저하 예방 방안 및 성능 개선 효과

□ 데이터베이스 성능 저하 예방 방안

예방방안	설명
상시 모니터링 및 관리	- 치명적 문제 발생 전, 주기적 모니터링 및 관리 - 정상 상태의 DB 로그 정보 보관: 성능 저하 시 비교활용
교육	- 개발자에 대한 SQL 튜닝 기법 및 수행 원리 교육 필요 - 프로그램 알고리즘 교육 필요, Source Review 제도 도입
지속적 튜닝 수행	- 지속적인 튜닝 작업의 수행 - 설계 보완, 시스템 성능 개선, SQL 문제 제거
Lessons Learned 관리	- 튜닝 중 발생된 문제, 문제분석 결과, 해결 내용 저장 - 이력 관리, 성능 튜닝 시 Reference로 활용

□ 데이터베이스 성능 개선효과

- 응용 프로그램 응답속도 향상
- 잠재적인 시스템 장애 방지
- 유지보수 비용 절감
- 하드웨어 증설에 대한 투자비 절감
- 개발자 능력 향상

데이터베이스 리팩토링

1. 데이터베이스 리팩토링(Database Refactoring) 개요

□ 데이터베이스 리팩토링(Database Refactoring) 정의

- 데이터베이스 리팩토링은 데이터베이스의 의미에 대한 변화 없이 디자인을 개선하는 작업으로 복잡한 설계문제를 개선하여 어플리케이션 연동의 효율성을 제공하기 위한 활동

□ 데이터베이스 리팩토링(Database Refactoring) 수행목적

- DB 리팩토링은 데이터 품질, 구조, 어플리케이션 개발의 효과적인 지원을 위해 최근 이슈화 되고 있는 추세
- 기업의 시스템이 대형화되고 기존 레거시시스템 개선과 연계, 통합이 수행되면서 데이터베이스 튜닝과 함께 DB 리팩토링에 대한 중요성이 증대
- DB 리팩토링은 DB Smell(데이터베이스의 나쁜 냄새)에 근거하여 수행

데이터베이스 리팩토링

2. 대표적인 DB Smell 및 리팩토링 절차

□ 대표적인 DB Smell

구분	상세설명
Multi-purpose column	단일 컬럼이 의미론적으로 다양한 목적으로 사용하고 있을 경우 다목적 컬럼은 해당 컬럼만으로는 사용자에게 올바른 정보를 주기가 어렵고 부가적인 컬럼을 확인
Multi-purpose table	단일 테이블이 여러 유형의 엔티티를 저장하고, 사용하고 있다면 설계 결함 오류 존재. 예를 들어, 고객 테이블이 고객 정보가 아닌, 기업정보에 대한 내역까지 추가적으로 저장하고 있다면 하나의 목적으로 사용된다고 볼 수 없음
Redundant data	중복데이터는 DB Smell의 가장 대표적인 사례로 여러 테이블, 여러 컬럼에 중복된 데이터는 데이터의 동기화, 데이터 품질에 심각한 영향을 줌. DB의 일관성을 저해하며, 사용자에게 잘못된 정보를 줄 수 있음.

Tables with many columns	너무 많은 컬럼을 가지고 있는 테이블은 검색, 삽입, 삭제 속도에 심각한 영향을 미치며 Big Table을 양산할 수 있음. 컬럼에 대한 의미를 분석하여 정규화시키며 테이블 분할을 고려함
Tables with many rows	대용량 테이블은 보통 수백만, 수천만 건의 데이터를 가지고 있는 테이블 대형 테이블은 보통 정보계 테이블(Fact Table)에 많이 존재하는데, 성능지표에 근거하여 테이블의 수직,수평 분할을 고려
Smart columns	스마트 컬럼은 하나의 컬럼에 대해 다기능적 목적을 수행할 수 있으나, DB 구조를 복잡하게 하며 어플리케이션의 복잡도를 증가 [예] 주민등록번호 (앞자리는 생년월일, 뒷자리는 성별, 주소 등등)
Fear of change	DB를 변화해야 하는 필요성은 이해당사자(DBA, Modeler, Application개발자 등)들이 모두 느끼고 있지만, 변화에 대해 소극적인 태도와 변화에 대한 두려움이 가장 나쁜 냄새

데이터베이스 리팩토링

3. 데이터베이스 리팩토링 유형 및 기법

□ 데이터베이스 리팩토링 유형

유형	설명	주요 기법
구조 리팩토링	데이터베이스 스키마의 테이블 구조를 변경	Merge : table, column Drop : table, view
데이터 품질 리팩토링	DB에 저장되어 있는 값의 일관성 및 사용성 개선	Standard codes Default 값이용
참조 무결성 리팩토링	참조된 컬럼에 대한 무결성 보장을 위한 제약조건 추가	외래키 추가, 계산된 컬럼에 트리거를 적용, Constraint 적용
아키텍처 리팩토링	외부 프로그램이 데이터베이스와 상호 작용하는 전반적인 방법 개선	Encapsulate Table With View, Migrate Method to Database
기능 리팩토링	저장 프로시저의 품질, 저장 기능, 트리거를 개선 변경	Consolidate Conditional Expression
변환	새로운 요소를 추가, 수정하여 데이터베이스 스키마 변경	Insert Data, Introduce New Column

데이터베이스 리팩토링

□ 데이터베이스 리팩토링 기법

구분	유형	기법	설명
품질향상	변환	Alter Table	테이블에 새로운 컬럼을 추가하여 개선
	데이터 품질 리팩토링	표준 코드 준수 Default 값 이용	DB에 저장되어 있는 값의 일관성 및 사용성 개선
	참조 무결성 리팩토링	FK, Trigger 적용, Constraint 적용	참조된 컬럼에 무결성을 보장하기 위해 제약 조건 추가
보안 및 성능 개선	아키텍처 리팩토링	View를 통해 테이블을 캡슐화	외부 프로그램이 데이터 베이스와 상호 작용하는 전반적인 방법 개선
	기능 리팩토링	Procedure, Trigger Drop 및 통합	프로시저의 품질, 트리거를 수정하여 개선
	구조 리팩토링	Merge: table, column Drop: table, view	데이터베이스 스키마의 구조를 변경함

데이터베이스 리팩토링

□ 데이터베이스 리팩토링 수행 시 고려사항

- DB 리팩토링은 데이터베이스 스키마의 변경방식에 따라 세가지 전략으로 수행될 수 있음. 빅뱅 방식, 증분형방식, 진화적방식 기반의 DB 리팩토링 전략을 고려
- DB 리팩토링은 많은 시간과 노력이 들어가며, 또한 전문적인 기술적용을 위한 데이터베이스 전문가가 필요한 작업
- 현업에서 DB 리팩토링 적용을 위해서는 단위 테스트에 대한 계획을 사전 수립한 후에 수행하며, 리팩토링 수행 후에 정보에 대한 변경사항 확인 및 회귀 테스트를 병행

감사합니다!



데이터베이스



6. 데이터 회복과 병행제어 part1

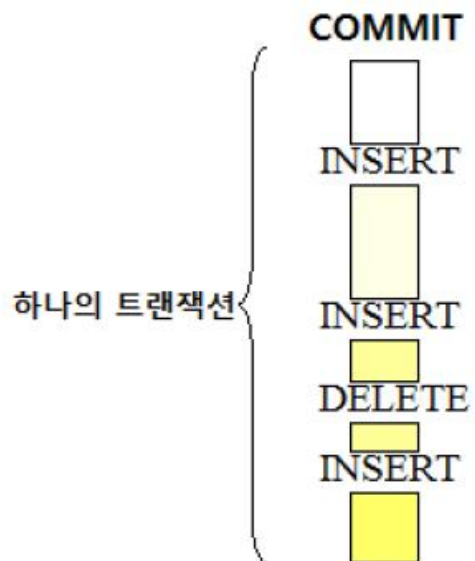
KPC 기본반

6. 데이터 회복과 병행제어	90
트랜잭션(Transaction).....	91
동시성 제어	94
다중 버전 동시성 제어(MVCC).....	101
트랜잭션 고립수준	104
데이터회복(Data Recovery).....	106

트랜잭션(Transaction)

□ 트랜잭션(Transaction) 정의

- 한번에 수행되어야 할 데이터베이스의 일련의 읽기와 쓰기 연산에 대해 수행하는 하나의 단위



데이터베이스의 논리적 연산단위로 분할할 수 없는 최소 단위

커밋(Commit)/롤백(Rollback)하기 전(Before)의 상태

모든 데이터의 변화는 데이터베이스 버퍼(Database Buffer)에 저장

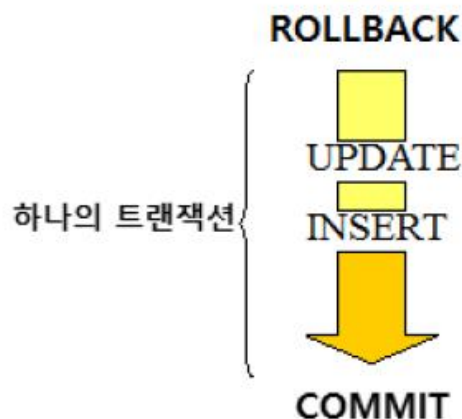
✓ 현재의 사용자는 자신이 변화시킨 데이터를 SELECT문을

통해서 볼 수 있지만 자신 이외의 사용자는 그 변화된
내용을 볼 수 없다.

✓ 자신이 변화시킨 Row에 대해서는 록(Lock)이 걸린 상태이기

때문에 다른 사용자가 해당 Row를 변화시키지 못한다.

(Row Level Locking Mechanism)



커밋(Commit)한 후(After)의 상태

모든 트랜잭션 내부에서 변화된 내용은 DB에 영구적으로 반영

✓ 다른 모든 사용자가 변화된 내용을 볼 수 있다.

해당 Row에 대한 록(Lock)이 제거된다

트랜잭션(Transaction)

□ 트랜잭션의 특징

특징	기본개념	관리
Atomicity (원자성)	- 트랜잭션은 한 개 이상의 동작을 논리적으로 한 개의 작업단위(single unit of work)로서 분해가 불가능한 최소의 단위 - 연산 전체가 성공적으로 처리되거나 또는 한 가지라도 실패할 경우 전체가 취소되어 무결성을 보장(All or Nothing) - Commit/Rollback 연산	트랜잭션 관리자
Consistency (일관성)	- 트랜잭션이 실행을 성공적으로 완료하면 언제나 모순 없이 일관성 있는 데이터베이스 상태를 보존	무결성 제어기
Isolation (고립성)	- 트랜잭션이 실행 중에 생성하는 연산의 중간 결과를 다른 트랜잭션이 접근할 수 없음	병행제어 관리자
Durability (영속성)	- 성공이 완료된 트랜잭션의 결과는 영구(속)적으로 데이터베이스에 저장됨	회복기법

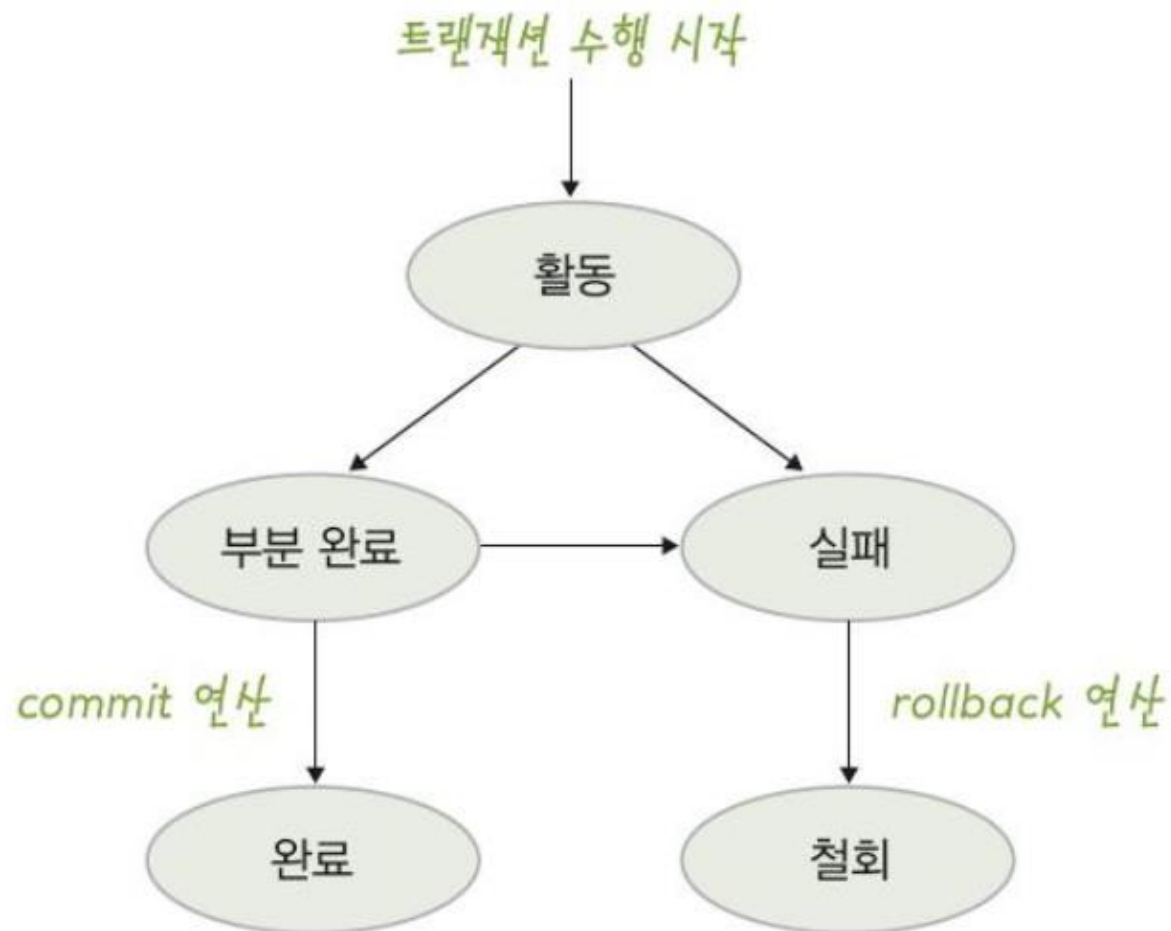
트랜잭션(Transaction)



트랜잭션(Transaction)

2. 트랜잭션(Transaction)의 상태 및 트랜잭션 연산

□ 트랜잭션 상태도

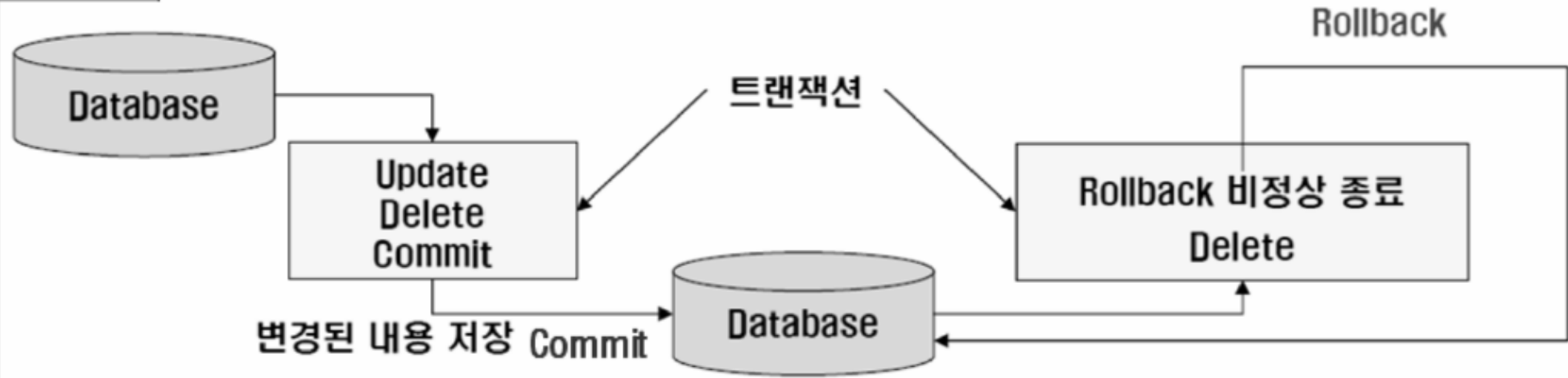


트랜잭션(Transaction)

상태	설명
활동상태(Active)	초기 상태, 트랜잭션이 실행 중이면 동작상태에 있다고 할 수 있음 트랜잭션이 수행을 시작하여 현재 수행 중인 상태
부분완료 상태 (Partial Committed)	마지막 명령문이 실행된 후에 가지는 상태 트랜잭션의 마지막 연산이 실행을 끝낸 직후의 상태
완료상태(Committed)	트랜잭션이 성공적으로 완료된 후 가지는 상태.
실패 상태(Failed)	정상적인 실행이 더 이상 진행될 수 없을 때 가지는 상태
철회 상태 (Aborted)	트랜잭션이 취소되고 데이터베이스가 트랜잭션 시작 전 상태로 환원되었을 때 가지는 상태 트랜잭션의 수행 실패로 rollback 연산을 실행한 상태

트랜잭션(Transaction)

트랜잭션(Transaction)의 연산



1) 철회된 트랜잭션의 연산

연산	설명
트랜잭션 재실행 (Restart)	철회된 트랜잭션을 다시 새로운 트랜잭션으로 취급하여 재 시작하는 방법 철회된 트랜잭션의 내부적인 논리 오류가 아니라, HW, SW 오류인 경우.
트랜잭션 폐기 (Kill)	철회된 트랜잭션을 폐기하는 방법 트랜잭션의 철회 원인이 트랜잭션의 내부적인 논리 오류가 원인

트랜잭션(Transaction)

2) 트랜잭션의 정상 종료 연산

연산	설명
Commit 연산	Data 변경 내용을 데이터베이스에 저장, 새로운 트랜잭션은 Commit 문 다음 시작 가능
프로그램 정상 종료	새로운 트랜잭션은 시작 불가. 변경 내용 저장을 Commit 연산과 동일한 처리

3) 트랜잭션의 비정상 종료 연산

연산	설명
Commit 연산	Data 변경 내용을 데이터베이스에 저장, 새로운 트랜잭션은 Commit 문 다음 시작 가능
Rollback 연산	해당 트랜잭션을 중지, 폐기하고, 데이터베이스 내용을 Rollback. 새로운 Transaction 은 Rollback 문 다음 시작 가능

동시성 제어(Concurrency)

1. 동시성제어(Concurrency) 개요

□ 동시성제어(Concurrency) 정의

- 다중 사용자 환경을 지원하는 데이터 베이스 시스템에서 여러 트랜잭션들이 동시에 성공적으로 실행될 수 있도록 지원하는 기능
- 여러 트랜잭션들이 차례로 번갈아 수행되는 인터리빙(interleaving) 방식으로 진행됨
- 병행 수행 시 같은 데이터에 접근하여 연산을 실행해도 문제가 발생하지 않고 정확한 수행 결과를 얻을 수 있도록 트랜잭션의 수행을 제어하는 것을 의미

□ 동시성제어(Concurrency)의 목적

- 트랜잭션의 직렬성 보장
- 공유도 최대, 응답 시간 최소, 시스템 활동의 최대 보장
- 데이터의 무결성 및 일관성 보장

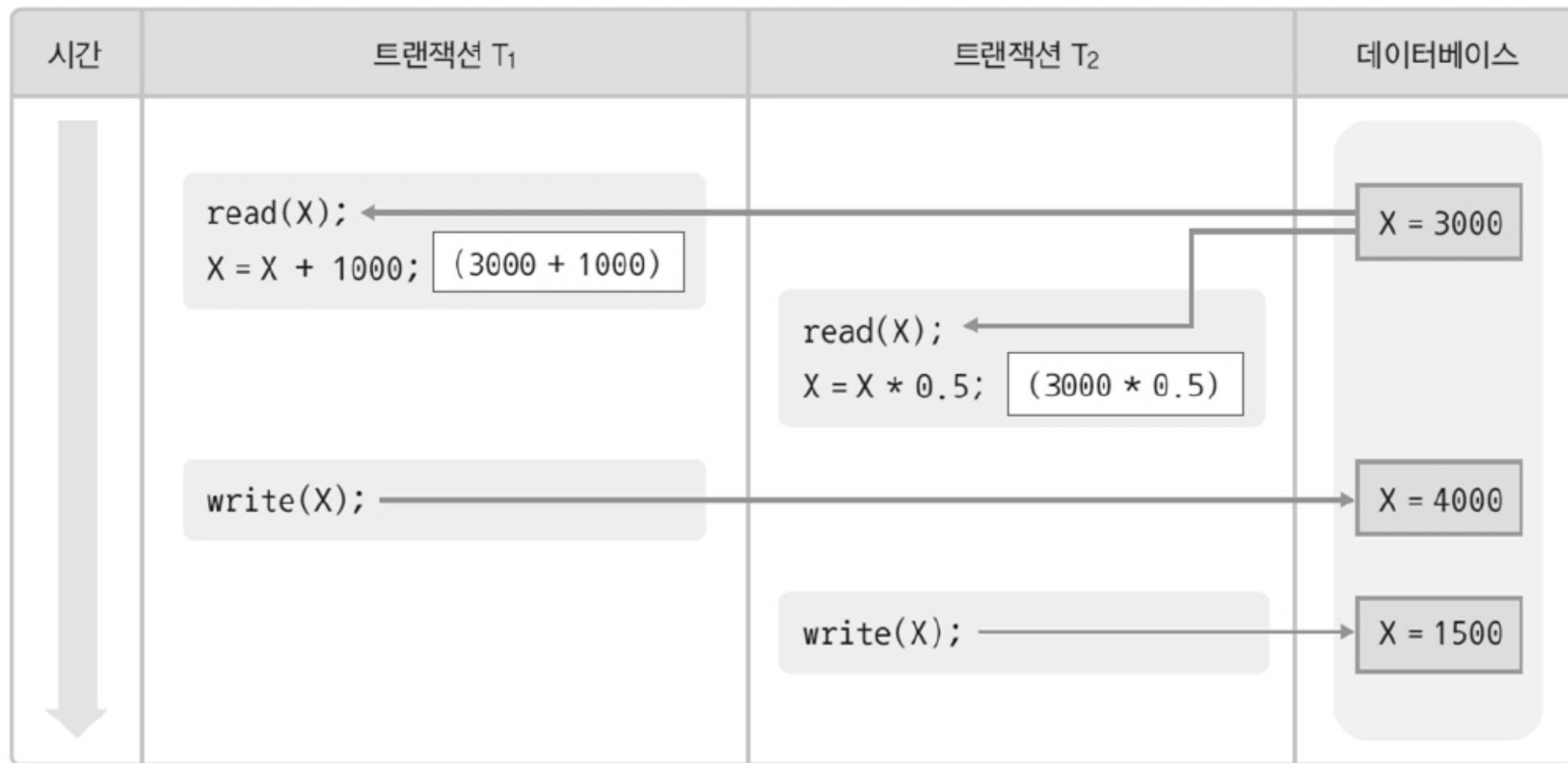
동시성 제어(Concurrency)

□ 병행수행 시 발생할 수 있는 문제점

구분	내용
갱신 손실 (Lost Update)	- 트랜잭션들이 동일 데이터를 동시에 갱신할 경우 발생 - 이전 트랜잭션이 데이터를 갱신한 후 트랜잭션을 종료하기 전에 나중 트랜잭션이 갱신 값을 덮어쓰는 경우 발생
현황파악오류 (Dirty Read)	-트랜잭션의 중간 수행결과를 다른 트랜잭션이 참조함으로써 발생하는 오류
모순성 (Inconsistency)	-두 트랜잭션이 동시에 실행할 때 DB가 일관성이 없는 상태로 남는 문제
연쇄복귀 (Cascading Rollback)	-복수의 트랜잭션이 데이터 공유 시 특정 트랜잭션이 처리를 취소할 경우 다른 트랜잭션이 처리한 부분에 대해 취소 불가능

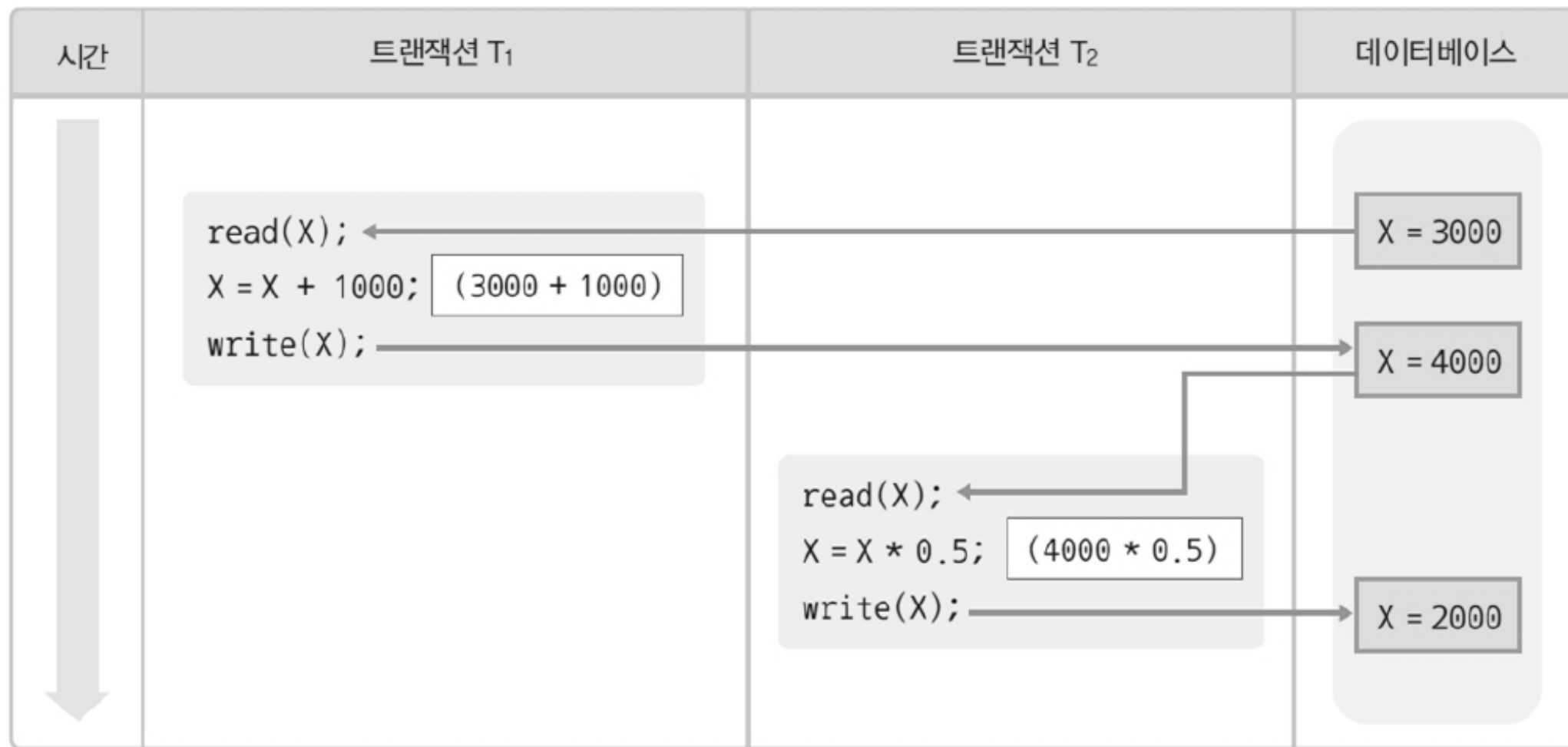
동시성 제어(Concurrency)

□ 갱신분실(lost update)



[두 트랜잭션의 병행 수행으로 발생한 갱신 분실의 예]

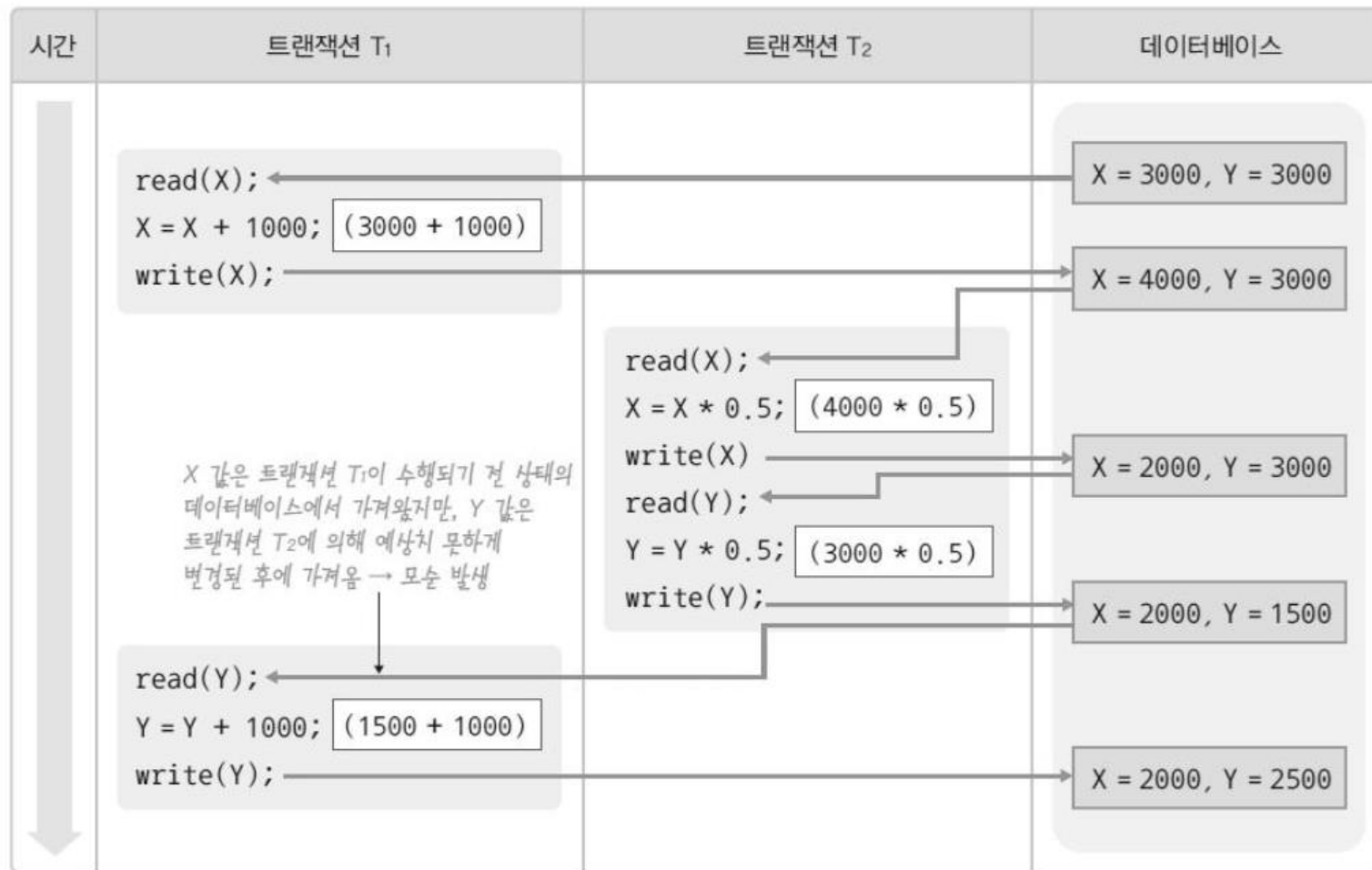
동시성 제어(Concurrency)



[T1을 먼저 수행하고 T2를 수행하여 갱신분실 문제가 발생하지 않은 경우]

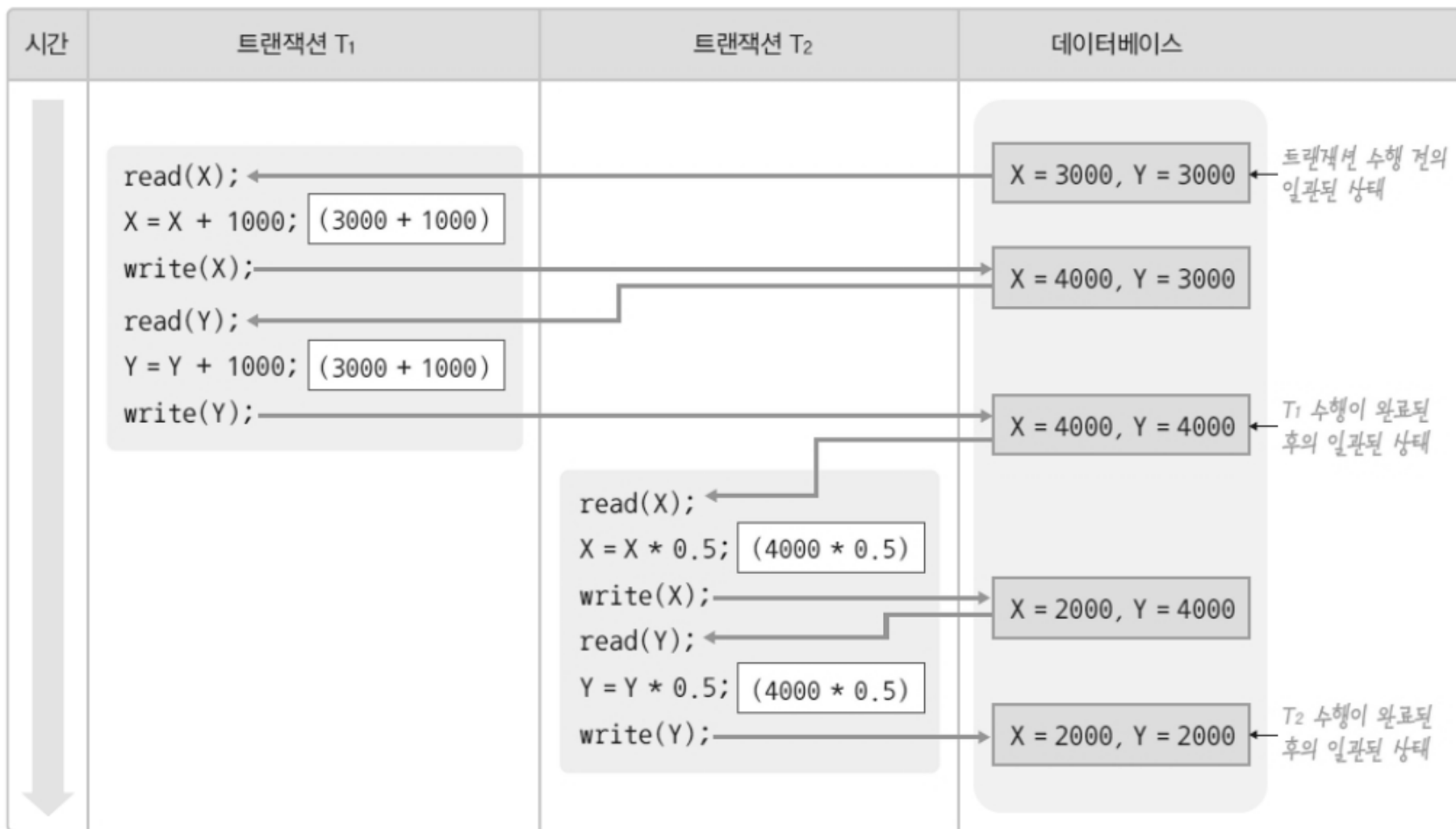
동시성 제어(Concurrency)

□ 모순성(inconsistency)



[두 트랜잭션의 병행 수행으로 발생한 모순성의 예]

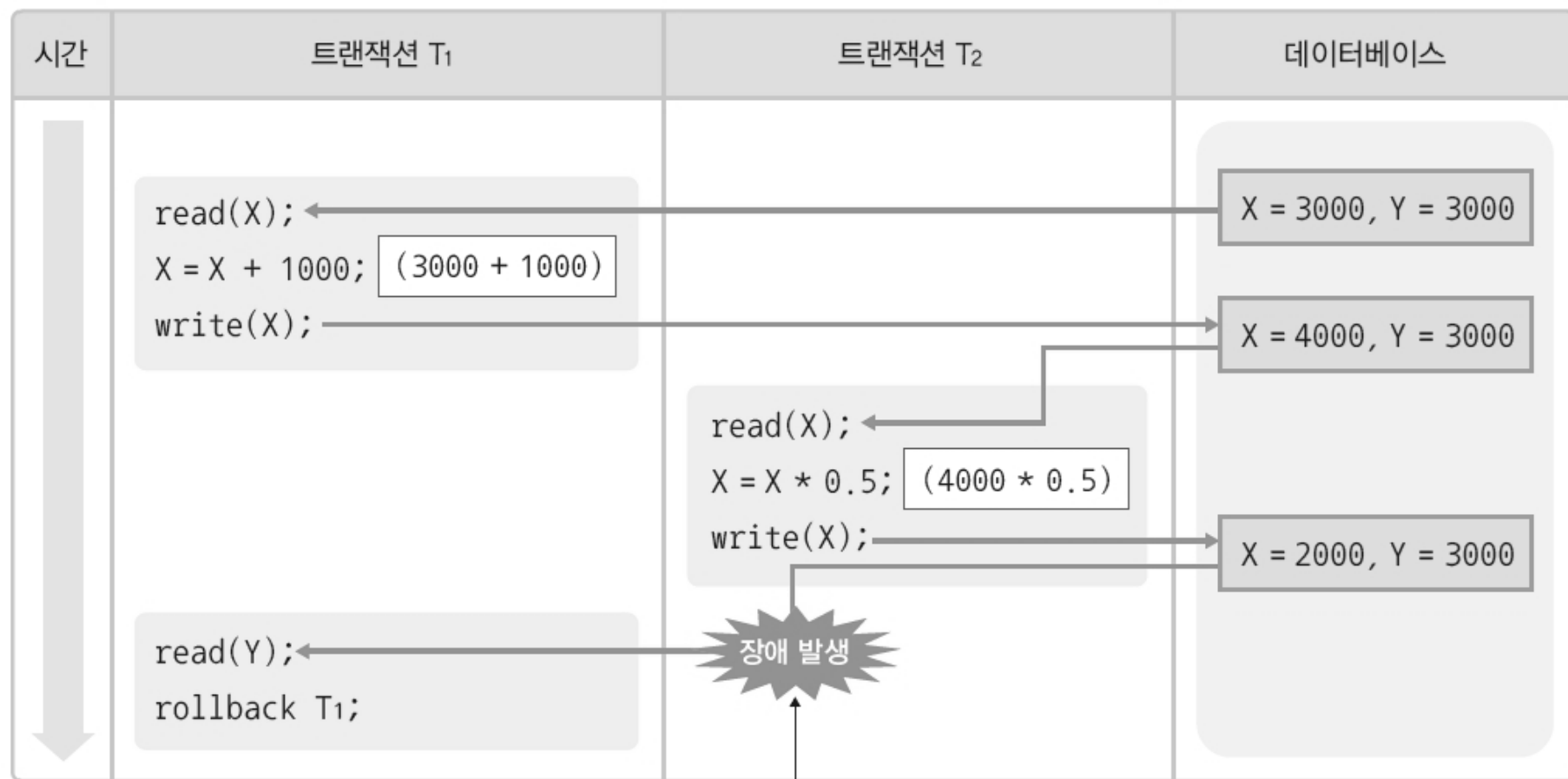
동시성 제어(Concurrency)



[T1을 먼저 수행하고 T2를 수행하여 모순성 문제가 발생하지 않은 경우]

동시성 제어(Concurrency)

□ 연쇄 복귀(cascading rollback)



트랜잭션 T₁이 rollback되면 트랜잭션 T₂도 rollback되어야 하는데
T₂가 이미 완료된 트랜잭션이라 rollback을 할 수 없음

동시성 제어(Concurrency)

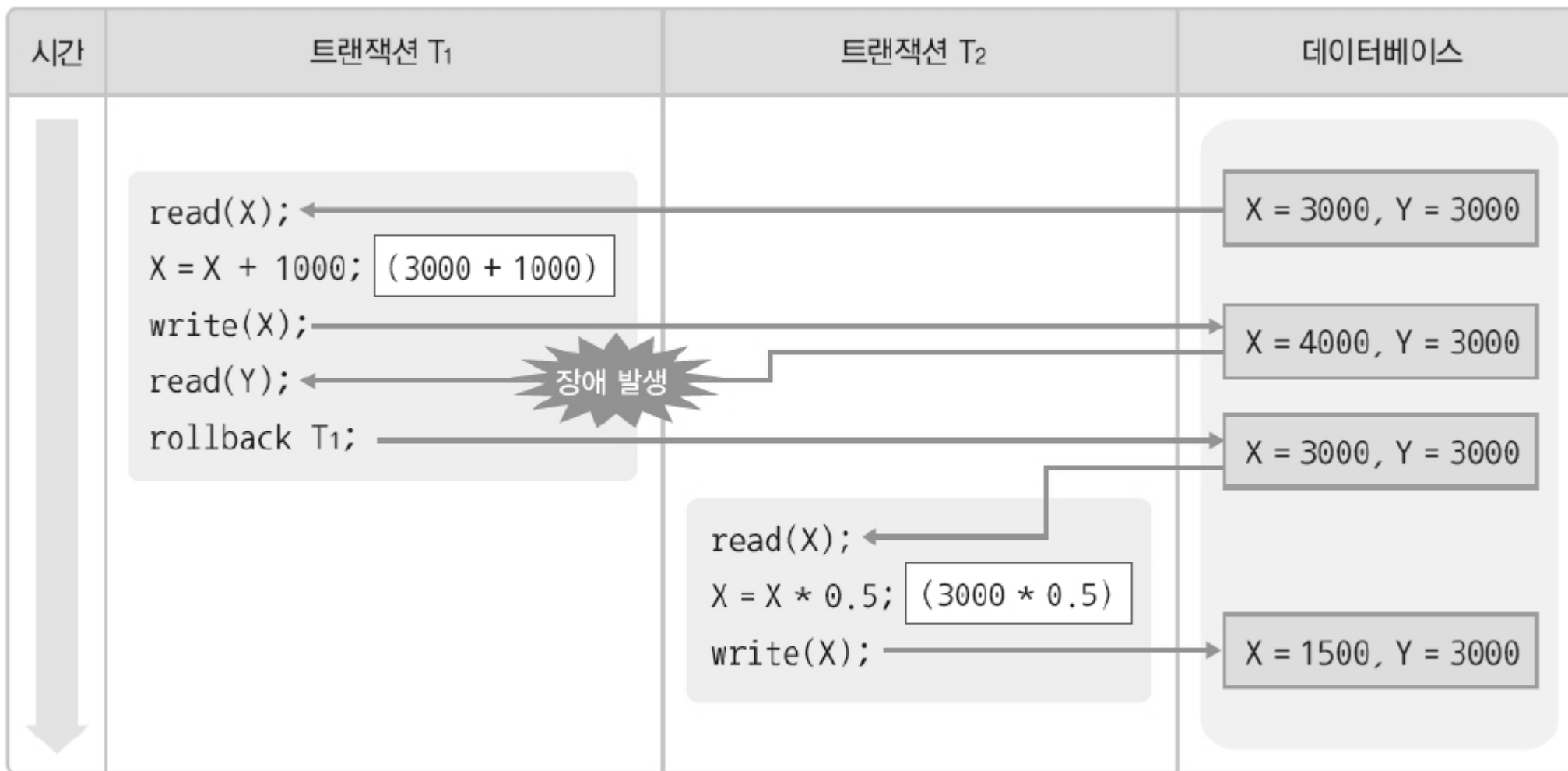


그림 10-35 트랜잭션 T₁을 수행한 후 트랜잭션 T₂를 수행한 결과

동시성 제어(Concurrency)

□ 트랜잭션 스케줄 유형

유형	내용
직렬스케줄	인터리빙 방식을 이용하지 않고 각 트랜잭션별로 연산들을 순차적으로 실행시키는 것 직렬 스케줄에 따라 트랜잭션을 수행하면 다른 트랜잭션의 방해를 받지 않고 독립적으로 수행되므로 항상 모순이 없는 정확한 결과를 얻게 됨
비직렬스케줄	인터리빙 방식을 이용하여 트랜잭션들을 병행해서 수행시키는 것 트랜잭션이 번갈아 연산을 실행하기 때문에 하나의 트랜잭션이 완료되기 전에 다른 트랜잭션의 연산이 실행될 수 있음 비직렬 스케줄에 따라 병행 수행하면 갱신 분실, 모순성, 연쇄 복귀 등의 문제가 발생할 수 있어 결과의 정확성을 보장할 수 없음
직렬가능스케줄	직렬 스케줄과 같이 정확한 결과를 생성하는 비직렬 스케줄 직렬 가능 스케줄인지를 판단하는 것은 간단한 작업이 아니므로 직렬 가능성을 보장하는 병행 제어 기법을 사용하는 것이 일반적임

동시성 제어(Concurrency)

3. 동시성 제어 기법 Locking 기법의 개요

□ Locking 기법의 정의

- 트랜잭션이 사용하는 자원에 대하여 상호 배제(Mutual Exclusive) 기능을 제공하는 기법
- 상호배제는 특정 트랜잭션이 데이터 항목에 대하여 잠금(Lock)을 설정한 트랜잭션이 해제(unlock) 할 때까지 데이터를 독점적으로 사용할 수 있는 것

□ Locking 연산의 종류

종류	주요개념
공유 Lock (Shared Lock)	공유 잠금한 트랜잭션이 데이터 항목에 대하여 읽기(read)만 가능 다른 트랜잭션도 읽기(Read) 만을 실행할 수 있는 형태
전용 Lock (Exclusive Lock)	전용 잠금한 트랜잭션은 데이터 항목에 대해서 읽기(read)와 기록(write) 모두 가능. 다른 트랜잭션은 읽기(read)와 기록(write) 모두 할 수 없음

동시성 제어(Concurrency)

□ 2단계 로킹(2PL : 2 Phase Locking)

- 모든 트랜잭션들이 Lock과 Unlock 연산을 확장단계와 수축 단계로 구분하여 수행함(read_lock(), write_lock(), unlock()) 연산으로 구성
- 기본 로킹 규약의 문제를 해결하고 트랜잭션의 직렬 가능성을 보장하기 위해 lock과 unlock 연산의 수행 시점에 대한 새로운 규약을 추가한 방법

확장 단계

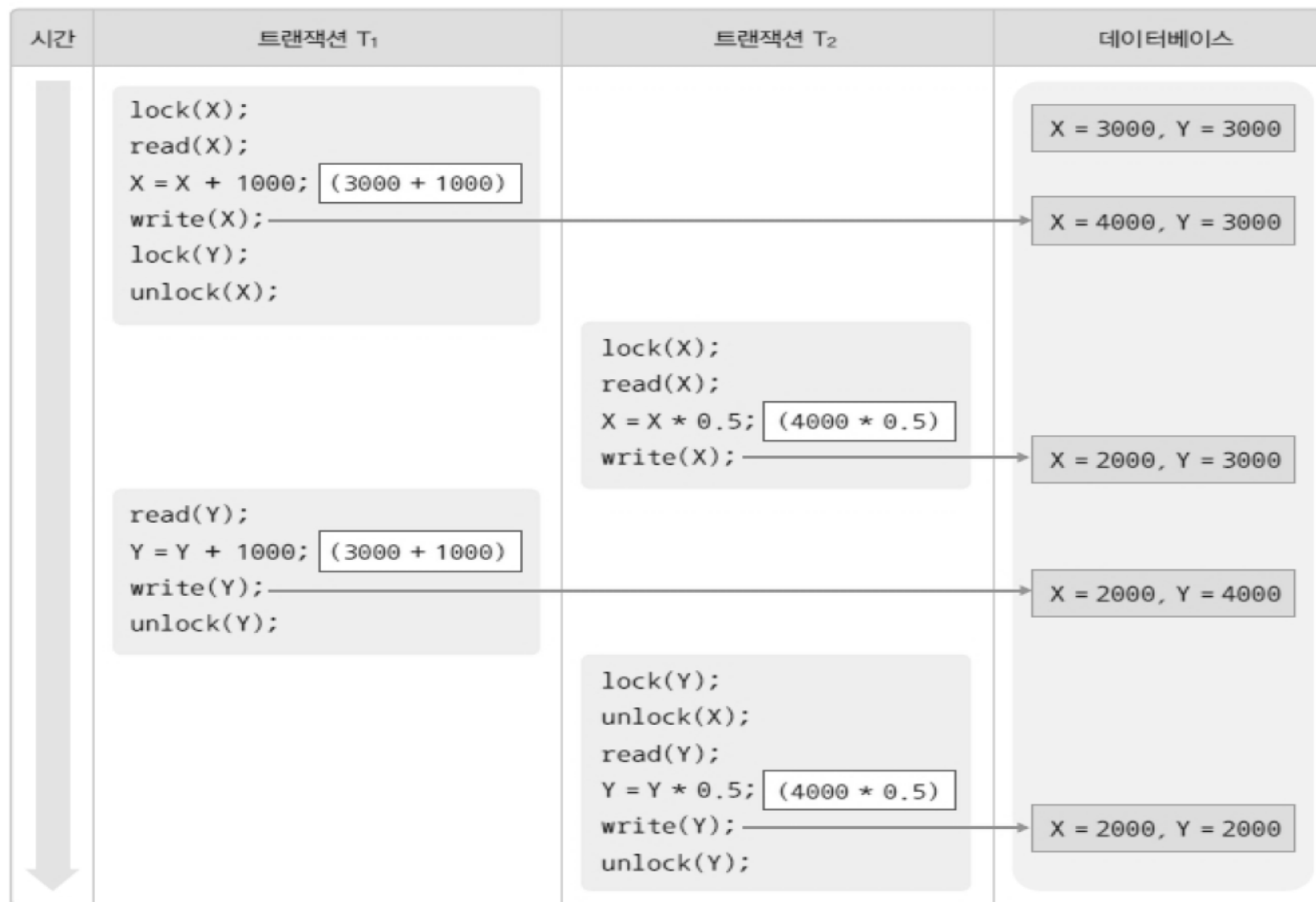
트랜잭션이 lock 연산만 실행할 수 있고, unlock 연산은 실행할 수 없는 단계

축소 단계

트랜잭션이 unlock 연산만 실행할 수 있고, lock 연산은 실행할 수 없는 단계

- 직렬 가능성을 보장할 수 있는 규약으로 가장 많이 사용됨
- 문제점: 교착 상태 발생 가능성 - 교착상태 예방과 교착 상태 탐지로 해결
- 트랜잭션은 첫 번째 unlock 연산 실행 전에 필요한 모든 lock 연산을 실행해야 함

동시성 제어(Concurrency)



동시성 제어(Concurrency)

□ Timestamp Ordering & 낙관적 검증기법

유형	내용	
Timestamp 기법	- 시스템에서 생성하는 Time stamp 를 고유한 트랜잭션의 식별자 로 부여 하여 실행 순서를 미리 선택 System 시계 또는 논리적 계수기 사용 - read_TS(X) 항목 X 의 읽기 타임 스탬프 - write_TS(X) 항목 X 의 쓰기 타임 스탬프	
	- Write_item(X) 연산을 수행 1) read_TS(X) > TS(T) or write _TS(T) > TS(T) -> T를 철회 및 복귀 2) Else, 트랜잭션 수행하고 Writ e_TS(X)를 TS(T) 로 설정	- read _ X) 연산을 수행 1) write _TS(T) > TS(T) -> T를 철회 및 복귀 2) Else, 트랜잭션 수행하고 read _TS(X)를 TS(T) 로 설정

동시성 제어(Concurrency)

낙관적 검증 기법	- 실행 전 실행 여부에 대한 검증을 수행하지 않고 일단 트랜잭션을 수행하고 트랜잭션 종료시 검증을 수행하는 기법 - 장기 트랜잭션 철회시 자원 낭비 가능성 동시 사용 빈도가 낮은 시스템 사용 The diagram illustrates the timeline of the Optimistic Validation Method. It is divided into three phases by vertical lines: Read Phase: Labeled '검증 없이 트랜잭션을 실행' (Execute transaction without verification). Validation Phase: Labeled '트랜잭션 종료 시 동시성 검증 수행' (Perform concurrency verification at transaction end). Execution Phase: Labeled '문제 없을 경우, 반영 Else 철회' (Reflect if no problem, Else rollback). - Start(T): 트랜잭션 T 가 판독 단계에 들어가면서 실행을 시작한 시간 - Validation(T): 트랜잭션 T 가 판독 단계를 끝내고 확인을 시작한 시간 - Finish(T): 트랜잭션 T 가 최종 기록 단계를 완료한 시간
------------------	---

동시성 제어(Concurrency)

1. 다중 버전 동시성 제어(Multi Version Concurrency Control)의 개요

□ 다중 버전 동시성 제어(Multi Version Concurrency Control) 정의

- 데이터베이스 관리시스템에서 서로 다른 트랜잭션이 동일 데이터에 접근하였을 때 각 세션마다 스냅샷 이미지를 보장하여 동시성을 높이는 매커니즘

□ 다중 버전 동시성 제어(Multi Version Concurrency Control) 현황

- 기존 로킹(Locking) 방식을 통한 상호배제 방법은 필연적으로 대기 현상이 발생하게 되어 동시성을 낮추는 결과를 초래

- 이를 해결하기 위해 1970년대 후반 논문에서 제시되었고 1984년 인터베이스(Interbase) DB에서 처음으로 구현

- 현재 대부분의 상용 DBMS는 MVCC 매커니즘을 지원하고 있음

동시성 제어(Concurrency)

2. 다중 버전 동시성 제어(Multi Version Concurrency Control)의 유형

□ MGA(Multi Generation Architecture) 방식

1) MGA(Multi Generation Architecture) 정의

- Interbase DB에 적용된 MVCC 매커니즘을 1999년 MGA 아키텍처를 채택하여 PostgreSQL 6.5에 처음으로 MVCC 도입한 모델
- 튜플을 업데이트할 때 동일한 데이터 페이지 내에서 New 튜플을 추가하고 Old 튜플은 유효 범위를 마킹하여 처리하는 방식

Xmin	10	Xmax	--	Tuple1
Xmin	10	Xmax	--	Tuple2
Xmin	10	Xmax	--	Tuple3

튜플2를
업데이트 함



Xmin	10	Xmax	—	Tuple 1
Xmin	10	Xmax	30	Old Tuple 2
Xmin	10	Xmax	—	Tuple 3
Xmin	30	Xmax	—	New Tuple 2



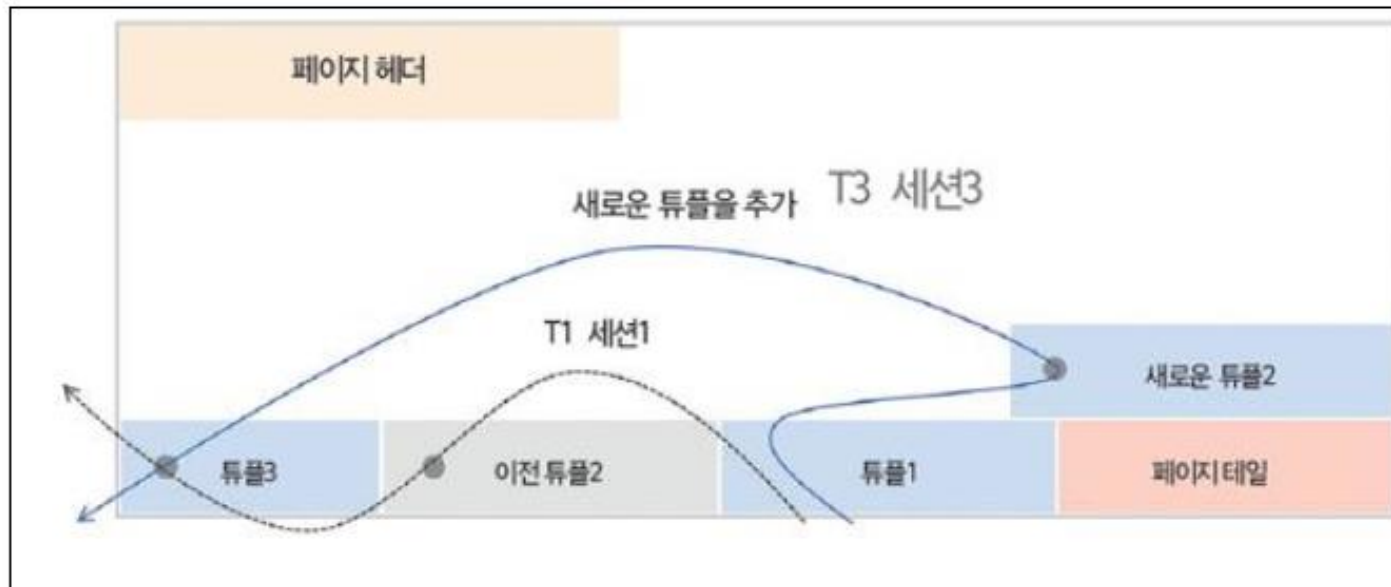
동시성 제어(Concurrency)

2) MGA(Multi Generation Architecture) 동작방식

- T0 -> xid '10', T1 -> xid '20', T2 -> xid '30', T3 -> xid '40' 을 가정



동시성 제어(Concurrency)



③ T1시점에 오픈된 세션1에서 SELECT를 하면 튜플1, 이전 튜플2, 튜플3이 반환되고 T3 시점에서 오픈된 세션3에서 SELECT 하면 튜플1, 새로운 튜플2, 튜플3이 반환

동시성 제어(Concurrency)

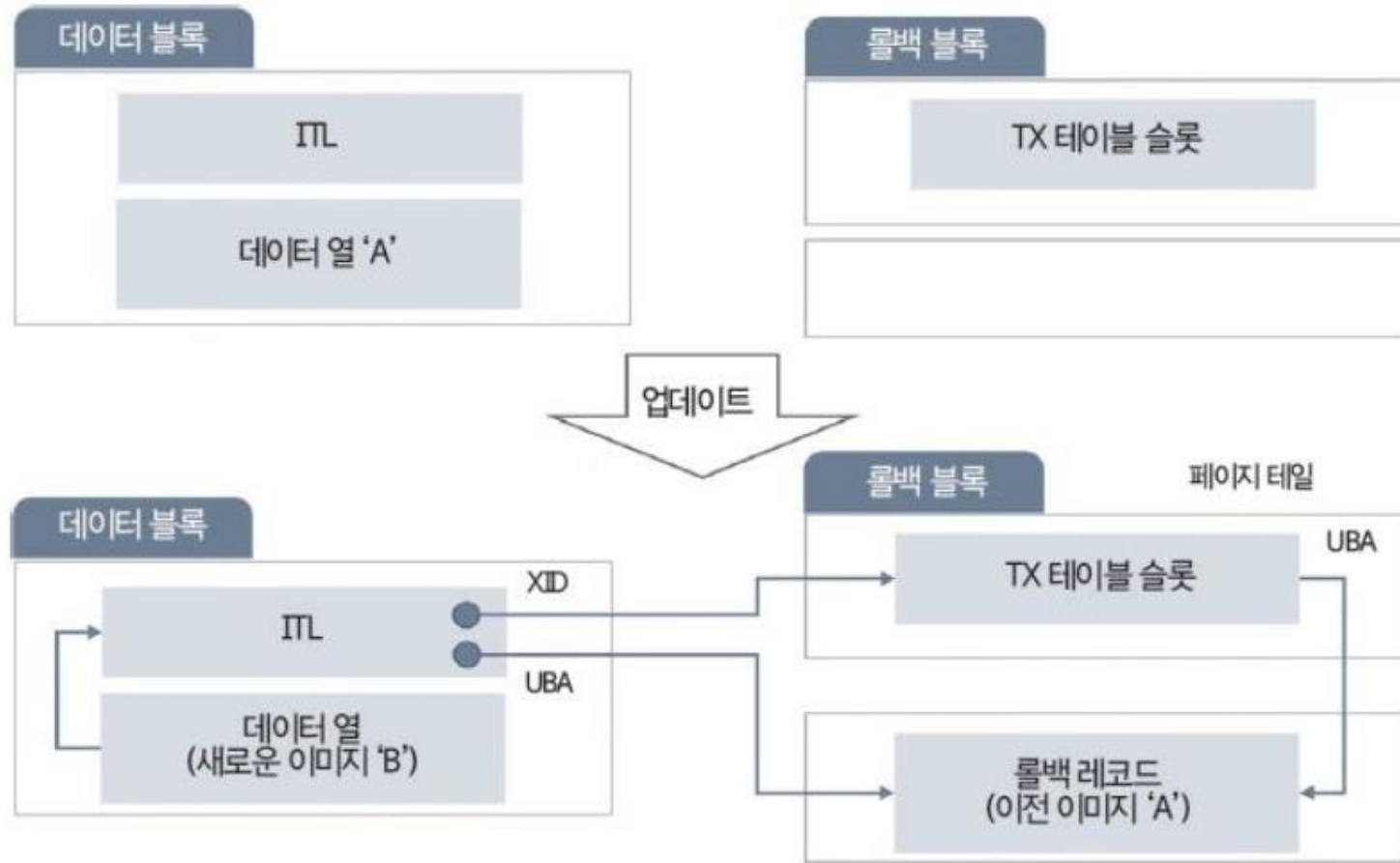
□ 롤백 세그먼트(Rollback Segment) 방식

1) 롤백 세그먼트(Rollback Segment) 매커니즘

- 1980년대 후반 오라클의 Bob Miner는 롤백 세그먼트를 도입하여 MVCC 구현
- 롤백 세그먼트 매커니즘은 업데이트 문장이 실행되면 기존 데이터 블록 내의 레코드를 New Version으로 바꾸고 Old Version을 별도 저장소인 롤백 세그먼트에 보관
- SELECT 실행시 SELECT SCN과 데이터 블록의 SELECT SCN를 비교하여 Consistent Read가 필요하다 판단되면 롤백 세그먼트의 Old Version을 읽어 버퍼 캐시에 CR 블록을 생성하는 방법
- ora-1555 snapshot too old 라는 한계점을 제외하고 동시성을 극대화해주는 최고의 방법
- 오라클은 롤백 세그먼트를 통해 Read Committed Isolation Level에서 거의 완벽한 MVCC 구현

동시성 제어(Concurrency)

2) 롤백 세그먼트(Rollback Segment) 구성



- 데이터 블록에 있는 현재 버전인 A를 B로 업데이트 하면 Old Version인 'A'를 별도 언두 블록에 저장하고 데이터 블록은 New Version인 'B'로 Replace

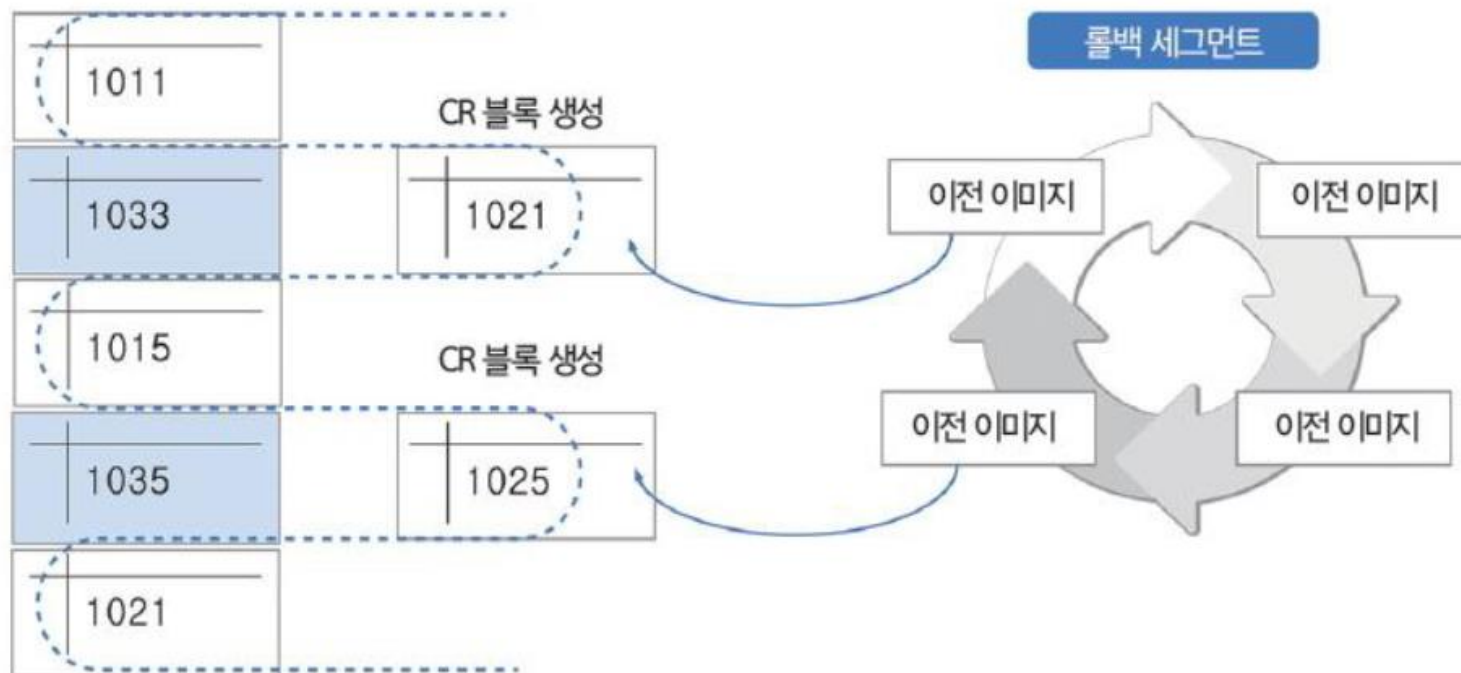
동시성 제어(Concurrency)

3) 롤백 세그먼트(Rollback Segment) 동작방식

- SELECT 시점에 SCN을 비교하여 Consistent Read를 위해 언두 세그먼트에 존재하는 해당 Old Version을 찾아 버퍼 캐시에 CR 블록을 생성
- CR 블록을 통해 SELECT 시점의 스냅샷 이미지를 Read 가능

★SCN(System Commit Number): Commit이 발생 시 부여되는 고유한 번호로 DB를 다시 생성하지 않는 한 0으로 돌아가지 않음

선택트 ... (SCN 1031)



트랜잭션 고립수준

1. 트랜잭션(Transaction)의 Isolation Level

□ 트랜잭션(Transaction)의 Isolation Level 정의

- 트랜잭션에서 일관성이 없는 데이터를 허용하도록 하는 수준
- 반대로 말하면 Transaction 수행 시 Locking을 얼마나 할 것인가를 지정하는 것

□ Transaction 의 Isolation Level 유형

연산	설명
Read Uncommitted Isolation Level	SELECT 문장을 수행하는 경우 해당 데이터에 Shared Lock 이 걸리지 않는 Level 어떤 사용자가 A 라는 데이터를 B 라는 데이터로 변경하는 동안 다른 사용자는 B 라는 아직 완료되지 않은(Uncommitted 혹은 Dirty) 데이터 B 를 읽을 수 있음.
Read Committed Isolation Level	Default 로 사용하는 Isolation Level 임. 다른 사용자가 완료한(Committed) 데이터만 읽음. DBMS 별로 Shared Lock 이 걸리거나, 걸리지 않을 수 있음

트랜잭션 고립수준

Repeatable Read Isolation Level	트랜잭션이 완료될 때까지 SELECT 문장이 사용하는 모든 데이터에 Shared Lock 이 걸림. 다른 사용자는 그 영역에 해당되는 데이터에 대한 수정이 불가능함 Select col1 from A where col1 between 1 and 10 을 수행 하였고 이 범위에 해당하는 데이터가 2 건이 있는 경우(col1=1 과 5) 다른 사용자가 col1 이 1 이나 5 인 Row 에 대한 UPDATE 가 불가능함. 하지만, col1 이 1 과 5 를 제외한 나머지 이 범위에 해당하는 Row 를 INSERT 하는 것은 가능함. 즉, 한번 읽은 집합을 다시 읽었을 때 값이 유지됨을 보장함 하지만, 범위에 새로운 값이 발생하는 것은 허용함. (Phantom read 가능)
Serializable Isolation Level	Repeatable Read 수준에서 범위에 대한 Shared Lock 이 추가됨. Repeatable Read 의 경우 1 에서 10 사이에 해당되는 데이터에 대한 INSERT 가 가능하지만 이 Level 에서는 INSERT 작업도 허용하지 않음. 즉, 범위에 대한 Shared Lock 을 통해 Phantom read 를 허용하지 않음.

트랜잭션 고립수준

☐ 팬텀충돌(Phantom Conflict)

```
T1 : SELECT SUM(SAL)
      FROM EMP
      WHERE DEPT = 'COMP ENG'
```

```
T2 : INSERT INTO EMP
      VALUES(E123, 'LEE', 'COMP ENG', 150)
```

T1과 T2는 데이터 베이스에서 공통 튜플을 접근하지 않음 즉, 트랜잭션 T1과 T2는 실제 데이터에 있어서 서로 충돌하지 않음

$\langle T1, T2 \rangle \neq \langle T2, T1 \rangle$

원인: 데이터 베이스에 삽입되어질 튜플, 즉 팬텀 튜플에 대해 T1과 T2 가 서로 충돌되기 때문

트랜잭션 고립수준

□ 팬텀충돌 해결방안

해결책	설명
Locking 단위변경	로킹 대상 데이터 단위를 튜플 -> 릴레이션
인덱스 로킹 기법 이용	- 모든 릴레이션은 적어도 하나의 인덱스를 가지고 있어야 한다 θ트랜잭션 T_i는 접근하려는 릴레이션의 튜플 t에 대한 포인터가 있는 인덱스 버킷에 S형 로크를 걸었을 때에만 그 튜플 t에 대해 S형 로크를 걸 수 있다 - 트랜잭션 T_i는 갱신하려는 릴레이션의 튜플 t에 대한 포인터가 있는 인덱스 버킷에 X형 로크를 걸었을 때에나 그 튜플 t에 대해 X형 락을 걸 수 있음 - 트랜잭션 T_i는 튜플을 삽입하기 전에 릴레이션의 모든 인덱스를 갱신하여야 하고 갱신하려는 모든 인덱스 버킷에 X형 로크를 걸어야 한다 - 로킹은 2단계 로킹 규약에 따라야 한다

감사합니다!



데이터베이스



6. 데이터 회복과 병행제어 part2

KPC 기본반

6. 데이터 회복과 병행제어	90
트랜잭션(Transaction).....	91
동시성 제어	94
다중 버전 동시성 제어(MVCC).....	101
트랜잭션 고립수준	104
데이터회복(Data Recovery).....	106

데이터회복(Data Recovery)

1. 데이터 회복(Data Recovery) 개요

□ 데이터 회복(Data Recovery) 정의

- 데이터베이스 운영 도중 예기치 못한 장애(Failure)가 발생할 경우 데이터베이스를 장애 발생 이전의 일관성과 무결성을 복원
- 데이터 베이스를 장애 시점 이전의 일관된 상태로 복원하여 내구성을 보장하는 행위

□ 데이터 회복(Data Recovery) 필요성

필요성	설명
데이터 일관성	장애 시점 이전과 동일한 내구성 보장
데이터 신뢰성	잘못된 데이터 갱신으로 인한 이전시점으로의 Rollback

데이터회복(Data Recovery)

□ 데이터베이스 장애의 유형

유형	주요내용
트랜잭션 장애	- 논리적 오류: 내부적인 오류로 트랜잭션을 완료할 수 없음. - 시스템 오류: Deadlock 등의 오류 조건으로 활성 트랜잭션을 강제로 종료
시스템 장애	- 전원, 하드웨어, 소프트웨어 등의 고장 - 시스템 장애로 인해 저장 내용이 영향받지 않도록 무결성 체크
디스크 장애	- 디스크 스토리지의 일부 또는 전체가 붕괴되는 경우 - 가장 최근의 덤프와 로그를 이용하여 덤프 이후에 완결된 트랜잭션을 재실행(REDO)
사용자 장애	- 사용자들의 데이터베이스에 대한 이해 부족으로 발생 - DBA 가 데이터베이스 관리를 하다가 발생시키는 실수

데이터회복(Data Recovery)

2. 데이터 회복을 위한 주요요소 및 회복 기법의 종류

□ 데이터 회복을 위한 주요요소

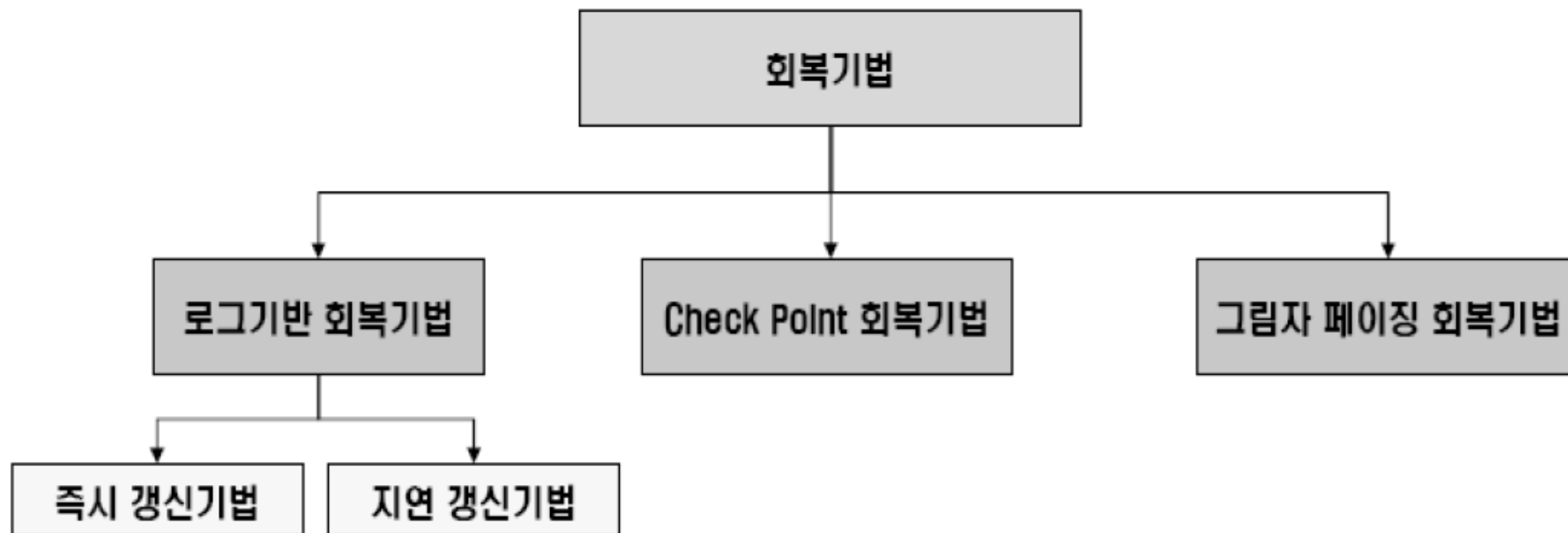
구분	요소	개념
회복의 기본원칙	데이터	- 데이터의 중복
	Archive	- 다른 저장장치로 자료의 복사 및 덤프
	Log 또는 journal	- 데이터베이스 내용이 변경 시마다 변경내용을 로그파일에 저장 - 갱신된 속성의 과거값/갱신값을 별도의 파일에 유지 - 온라인로그(디스크), 보관로그(테이프)

데이터회복(Data Recovery)

회복을 위한 조치	REDO	- 최근 변경된 내용을 로그파일에 기록하고, 장애발생시 로그 파일을 읽어서 재실행함으로써 데이터베이스 내용을 복원 - Archive 사본 + log : commit 후의 상태
	UNDO	- 장애발생 시 모든 변경된 내용을 취소함으로써 원래의 데이터베이스 상태로 복원 - Log + Backward 취소연산 : 해당 트랜잭션 수행이전 상태
시스템	회복관리기능	- 신뢰성 제공을 위한 DBMS 서브시스템

데이터회복(Data Recovery)

□ 데이터 회복기법의 종류



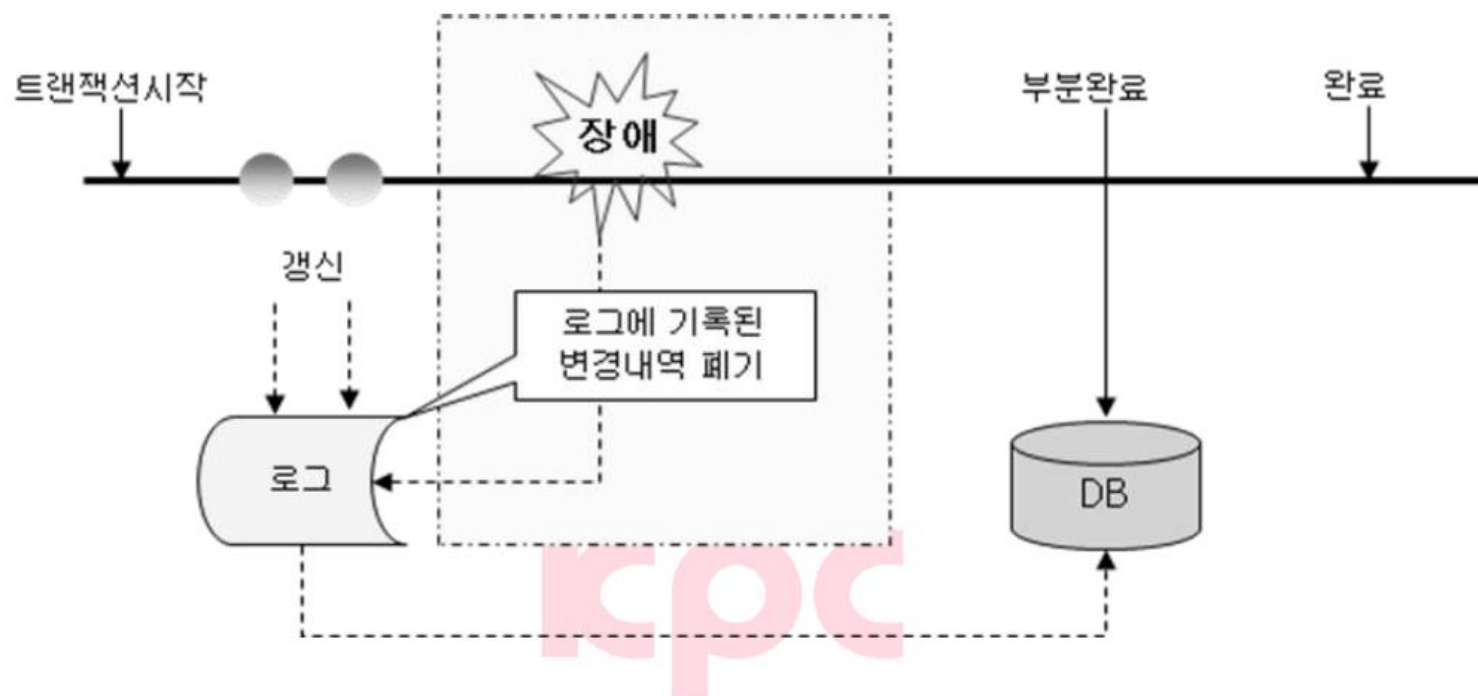
데이터회복(Data Recovery)

3. 데이터 회복 기법 상세

□ 지연 갱신기법(Deferred Update)

갱신	- 트랜잭션 단위가 종료될 때까지 DB에 Write 연산을 지연시키고 - 동시에 DB 변경내역을 Log 에 보관한 후 - 트랜잭션이 완료되면 Log 를 이용하여 데이터베이스에 Write 연산 수행
회복	- 트랜잭션이 종료된 상태이면 회복 시 Undo 없이 Redo 만 실행함. - 트랜잭션이 종료가 안된 상태였으면 Log 정보는 무시함.

데이터회복(Data Recovery)



< T1, Start > < T1, A, 900 > < T1, B, 2100 >	< T1, Start > < T1, A, 900 > < T1, B, 2100 > < T1, Commit > < T2, Start > < T2, C, 2800 >	< T1, Start > < T1, A, 900 > < T1, B, 2100 > < T1, Commit > < T2, Start > < T2, C, 2800 > < T2, Commit >
아무런 회복 조치 필요 없음	Redo(T ₁)	Redo(T ₁) Redo(T ₂)

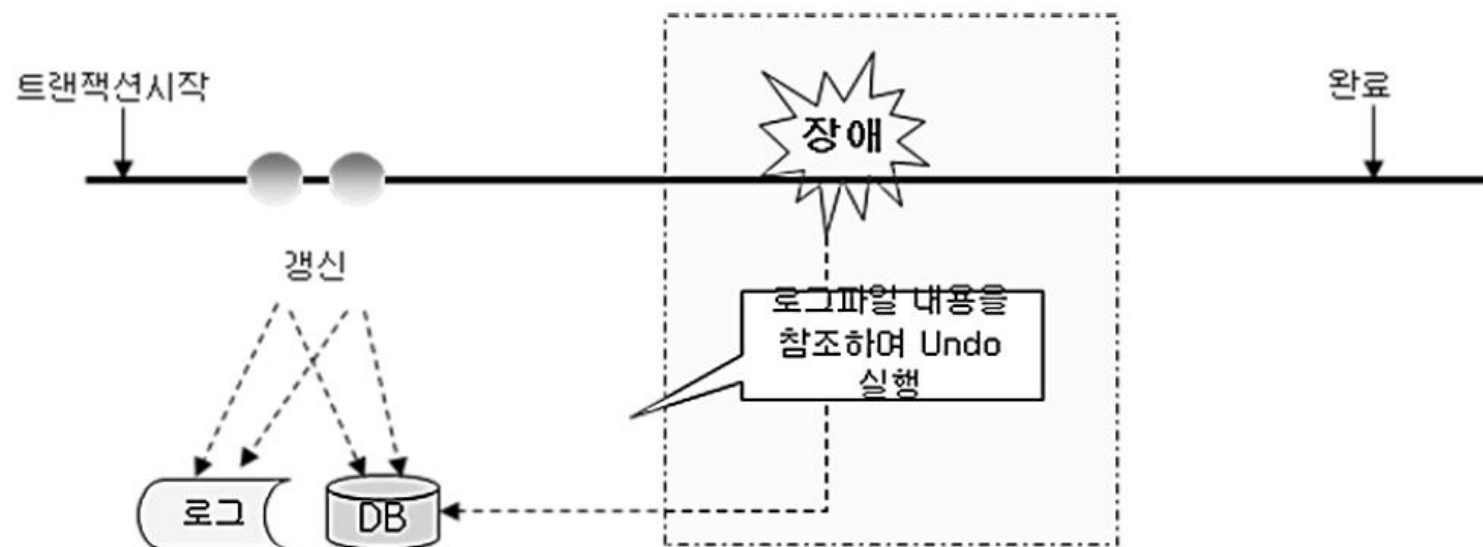
- UNDO연산이 불필요하므로, 로그 레코드는 이전 값은 기록하지 않음
<트랜잭션 id, 데이터 아이템, 변경된 값>
- 로그에 <Ti, Start> 레코드만 있고, <Ti, Commit> 레코드가 없으면 무시함
- 로그에 <Ti, Start> 레코드와 <Ti, Commit> 레코드가 모두 있는 트랜잭션 Ti에 대해서만 Redo

데이터회복(Data Recovery)

□ 즉시 갱신기법(Immediate Update)

갱신	-트랜잭션 활동상태에서 갱신결과를 DB 에 즉시 반영하고 log 기록
회복	-트랜잭션 수행 도중 실패상태에 도달하여 트랜잭션을 철회할 경우에는 로그 파일에 저장된 내용을 참조하여 Undo 연산 수행

데이터회복(Data Recovery)



< T1, Start >
 < T1, A, 1000, 900 >
 < T1, B, 2000, 2100 >
 < T1, Commit >
 < T2, Start >
 < T2, C, 3000, 2800 >

Undo(T2)
 Redo(T1)

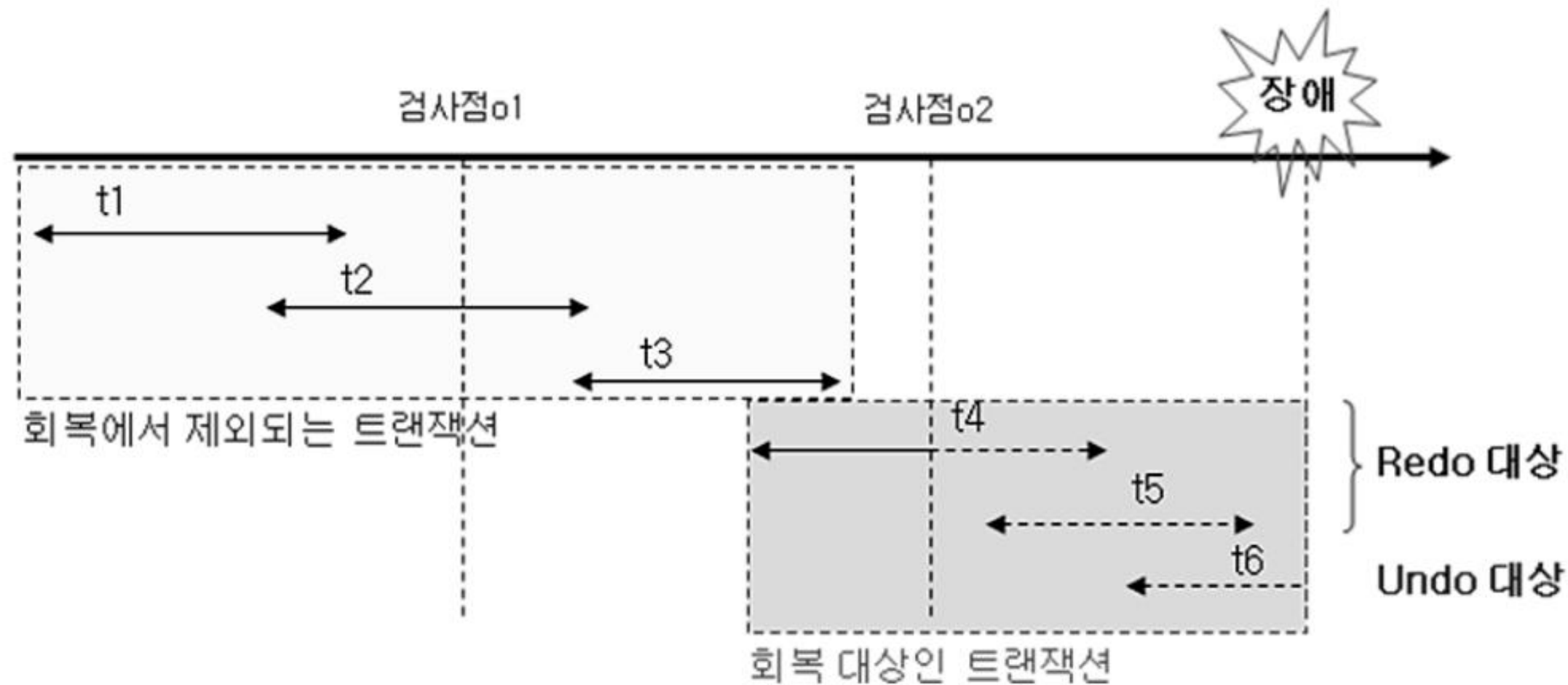
- UNDO연산이 필요하므로, 로그 레코드는 이전 값을 기록함
<트랜잭션 id, 데이터 아이템, 변경전 값, 변경된 값>
- 로그에 <Ti, Start> 레코드만 있고, <Ti, Commit> 레코드가 없으면 트랜잭션 Ti에 대해서 Undo 수행
- 로그에 <Ti, Start> 레코드와 <Ti, Commit> 레코드가 모두 있는 트랜잭션 Ti에 대해서 Redo 수행
- 일반적으로 모든 Undo연산을 마친 뒤에 Redo연산을 수행함
(Undo연산은 로그에 기록된 역순으로 수행하고, Redo연산은 로그에 기록된 순서로 수행하기 때문)

데이터회복(Data Recovery)

□ Check Point 회복기법

갱신	- 검사점(Check Point)을 Log 파일에 기록하고 장애 발생시에 검사시점 이전에 처리된 트랜잭션은 회복작업에서 제외하고 검사시점 이후에 처리된 내용에 대해서만 회복작업을 수행하는 회복기법
회복	- 트랜잭션 수행도중 문제점이 발생하면, 로그 파일의 정보를 모두 검사하여, Redo 와 Undo 연산을 실행할 트랜잭션과 체크포인트를 선정 - 검사점의 Log 파일 기록을 이용하여 실행함. - 장애 발생시 검사점 이전에 처리된 트랜잭션들을 회복대상에서 제외 - 검사점 이후에 처리된 트랜잭션에 대해서만 회복작업을 시행함. - 새로 시작한 트랜잭션은 Undo 리스트 - Commit 된 트랜잭션은 Redo 리스트 - 로그 역방향으로 Undo 실행 후 로그 방향으로 Redo 실행

데이터회복(Data Recovery)



데이터회복(Data Recovery)

4. 그림자페이지(Shadow Paging) 기법

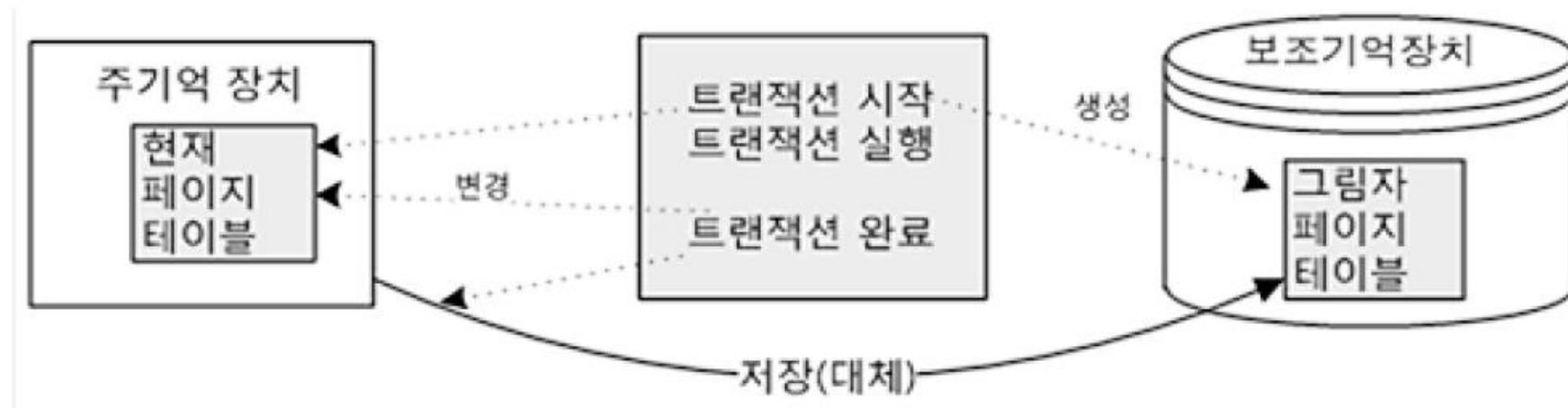
□ 그림자페이지(Shadow Paging) 기법의 정의

- 로그를 이용하지 않고 데이터베이스에 대한 페이지 테이블 전체를 교체하는 방법으로 로그 레코드를 출력하는 오버헤드를 없애고, 디스크 접근횟수를 줄이기 위한 방법
- 2개의 페이지 테이블 (현 페이지와 그림자 페이지)을 유지하고, 트랜잭션 실행 중에는 현 페이지 테이블만을 사용하는 것으로 Undo 및 Redo 연산이 필요 없는 방법

□ 그림자페이지 기법의 수행방법

- 1) 트랜잭션이 실행되는 동안 현재 페이지 테이블과 그림자 페이지 테이블을 이용
- 2) 현재 페이지 테이블은 주기억장치, 그림자 페이지 테이블은 하드디스크에 저장함.
- 3) 데이터베이스 트랜잭션의 시작시점에 현재 페이지 테이블의 내용과 동일한 그림자 페이지 테이블을 생성함.
- 4) 트랜잭션의 변경 연산이 수행되면, 현재 페이지 테이블의 내용만 변경하고 그림자 페이지 테이블의 내용은 변경하지 않음.
- 5) 트랜잭션이 성공적으로 완료될 경우, 현재 페이지 테이블의 내용을 그림자 페이지 테이블의 내용으로 저장함.

데이터회복(Data Recovery)



[복구방법]

- 수정된 데이터베이스 페이지를 반환하고, 현재의 페이지 테이블을 폐기
- 그림자 페이지 테이블을 현재 페이지 테이블로 설정

□ 그림자페이지 기법의 단점

- 데이터 단편 (data fragmentation)
- 쓰레기 수집 (garbage collection)
- 병행 트랜잭션 (concurrent transaction) 지원이 곤란 → 병행수행환경에서는 로그와 검사시점 기법을 함께 사용

감사합니다!



데이터베이스



8. 데이터베이스 응용

KPC 기본반

8. 데이터베이스 응용	136
분산 데이터베이스(Distributed Database)	137
인메모리 데이터베이스(In-memory Database)	140

분산 데이터베이스(Distributed Database)

1. 분산 데이터베이스(Distributed Database) 개요

□ 분산 데이터베이스 정의

- 논리적으로 하나의 시스템으로 구현되어 있으나, 물리적으로 네트워크를 통하여 분산화된 형태로 관리되는 데이터베이스
- 하나의 데이터베이스에 집중되는 부하를 지역적으로 분리하여 부하를 줄이고 사용자에게 논리적으로 하나로 통합된 데이터베이스

□ 분산 데이터베이스의 목적

목적	설명
데이터처리의 지역화	통신비용의 감소 및 데이터처리 집중화 방식
데이터 운영 및 관리의 지역화	데이터에 대한 이해도가 높은 집단이 관리
데이터의 처리 속도 향상	데이터 처리 부하의 분산 및 병렬 데이터 처리하여 속도향상
데이터의 가용도와 신뢰성 향상	데이터를 복제

분산 데이터베이스(Distributed Database)

2. 분산 데이터베이스의 투명성 및 기능, 장단점

□ 분산 데이터베이스의 투명성(Transparency)

- 분산데이터베이스가 되기 위해서는 6가지 투명성을 만족해야 함

구분	주요개념
분할 투명성	하나의 논리적 Relation이 여러 단편으로 분할되어 각 단편의 사본이 여러 site에 저장
위치 투명성	사용하려는 데이터의 저장 장소 명시 불필요. 위치정보가 System Catalog에 유지되어야 함
지역사상 투명성	지역DBMS와 물리적 DB사이의 Mapping 보장. 각 지역시스템 이름과 무관한 이름 사용 가능
중복 투명성	DB 객체가 여러 site에 중복되어 있는지 알 필요가 없는 성질
장애 투명성	구성요소(DBMS, Computer)의 장애에 무관한 Transaction의 원자성 유지
병행 투명성	다수 Transaction 동시 수행 시 결과의 일관성 유지, Time Stamp, 분산 2단계 Locking을 이용 구현

분산 데이터베이스(Distributed Database)

□ 분산 데이터베이스의 기능

기능	설명
데이터에 관한 정보 유지	분산 DBMS 카탈로그를 확장하여 데이터의 분산, 단편화, 중복에 관한 정보를 유지
분산 질의 처리	원격지 사이트 사이에서 발생하는 질의와 데이터를 처리하고 전송하는 기능
분산 트랜잭션 관리	전체 데이터베이스에서 무결성이 유지되도록, 두 개 이상의 사이트에서 실행하는 트랜잭션들의 분산 데이터에 대한 접근을 동기화
중복 데이터 관리	중복된 데이터 항목들 중에서 접근할 복사본을 식별하고, 중복된 데이터 항목들이 속한 복사본들의 일관성을 유지해 주는 기능
분산 데이터베이스 회복	디렉터리의 위치와 분산은 설계 방법에 따라 결정
분산 데이터베이스 보안	분산 데이터베이스를 관리하기 위해서는 더 세밀한 데이터 보안 및 사용자 권한 관리가 필요
분산 카탈로그 관리	데이터에 대한 다양한 정보를 포함하는 디렉토리(카탈로그) 관리

분산 데이터베이스(Distributed Database)

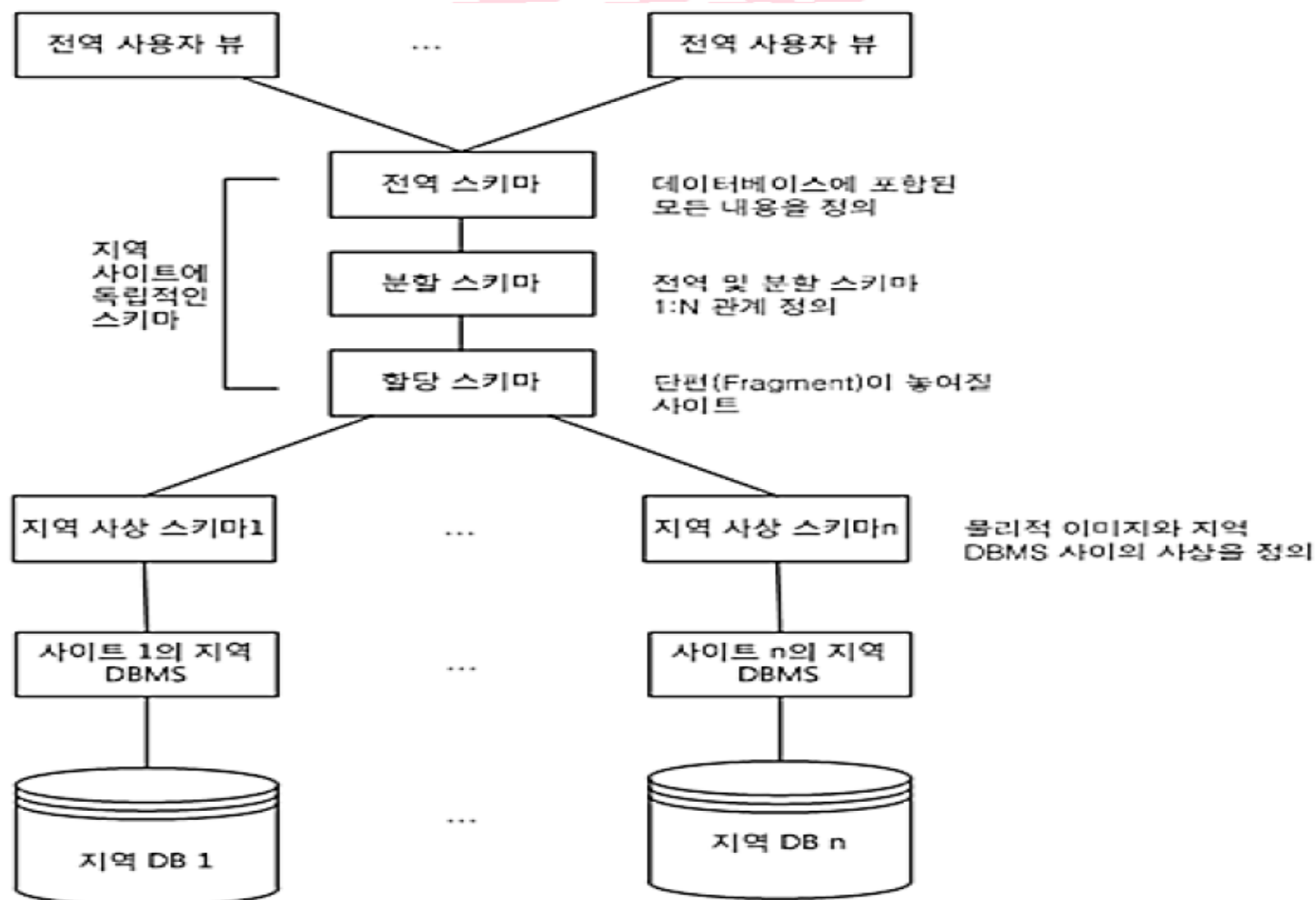
□ 분산 데이터베이스 투명성의 장단점

장점	단점
지역 자치성, 점증적 시스템 용량 확장 신뢰성과 가용성 효용성과 융통성 빠른 응답속도와 통신비용 절감 데이터의 가용성과 신뢰성 증가 시스템 규모의 적절한 조절 각 지역 사용자의 요구 수용 증대	소프트웨어 개발 비용 오류의 잠재성 증대 처리 비용의 증대 설계, 관리의 복잡성과 비용 불규칙한 응답 속도 통제의 어려움 데이터 무결성에 대한 위협

분산 데이터베이스(Distributed Database)

3. 분산 데이터베이스의 구성

□ 분산 데이터베이스의 구성도



분산 데이터베이스(Distributed Database)

□ 분산 데이터베이스의 구성요소

구성요소	설명
전역 스키마	지역 데이터베이스 스키마들을 모두 포함하는 상위 집합으로 사용자들은 전역 스키마를 통해 마치 하나의 데이터베이스를 사용하는 것처럼 이용
분할 스키마	단편화 스키마로 전역 릴레이션과 단편과 사상을 정의
할당 스키마	분산을 위한 단편들과 할당될 사이트 간의 관계를 표현하는데, 한 단편의 이미지가 2개 이상의 사이트에 할당되는 경우 중복 저장됨
지역사상 스키마	물리적 이미지를 해당 DBMS가 관리하는 객체로 취급할 수 있도록 사상

분산 데이터베이스(Distributed Database)

□ 분산 데이터베이스 설계전략

전략	내용	
복제 (Replication)	같은 Data를 두 장소 이상에 저장하는 관리 방법론	
	장점	빠른 응답속도, DB의 HA보장, 신리성 보장
	단점	데이터의 갱신의 복잡, 높은 비용, 저장 공간의 낭비발생
수평분할 (Horizontal Partitioning)	Table은 어떤 행들은 한 지역으로 저장하고, 다른 행은 다른 지역에 저장 관리	
	장점	자료를 그 자료가 사용되는 지역에 저장, 빠른 속도 보장 가능
	단점	여러 지역의 분할된 자료를 동시에 필요할 경우 통신속도 때문에 성능 저하
수직적 분할 (Vertical Partitioning)	테이블의 어떤 column은 한 지역에 저장하고 나머지는 다른 지역에 저장 관리 (Field별 분할)	
	장점	데이터의 보안 강화, 관리가 용이
	단점	모든 속성을 알기 위한 많은 시간 및 통신 필요
혼합방식 (Hybrid Partitioning)	여러 방식을 혼합, 중복저장, 수평 분할, 수직 분할 실시	
	장점	각 데이터의 용도에 맞는 정책 적용
	단점	Object별 성능차이 발생, 전체적 성능 저하 가능성 있음

분산 데이터베이스(Distributed Database)

□ 분산 데이터베이스 고려사항

- 성능이 중요한 사이트에 적용해야 함
- 공통코드, 기준정보, 마스터 데이터 등에 대해 분산환경을 구성하면 성능이 좋아짐
- 실시간 동기화가 요구되지 않을 때 좋고 거의 실시간 의 업무적인 특징을 가지고 있을 때도 분산 환경을 구성할 수 있음
- 특정 서버에 부하가 집중될 때 부하를 분산할 때도 좋음
- 백업사이트를 구성할 때 간단하게 분산기능을 적용하여 구성할 수 있음

인메모리 데이터베이스

1. 인메모리 데이터베이스(In-memory Database) 개요

□ 인메모리 데이터베이스 정의

- 데이터의 저장, 연산을 디스크를 거치지 않고 모두 메인 메모리 에서 처리하는 데이터베이스
- 데이터 스토리지의 메인 메모리에 설치되어 운영되는 방식의 데이터베이스 관리 시스템으로 디스크에 설치되는 방식에 비해 처리속도가 빠름

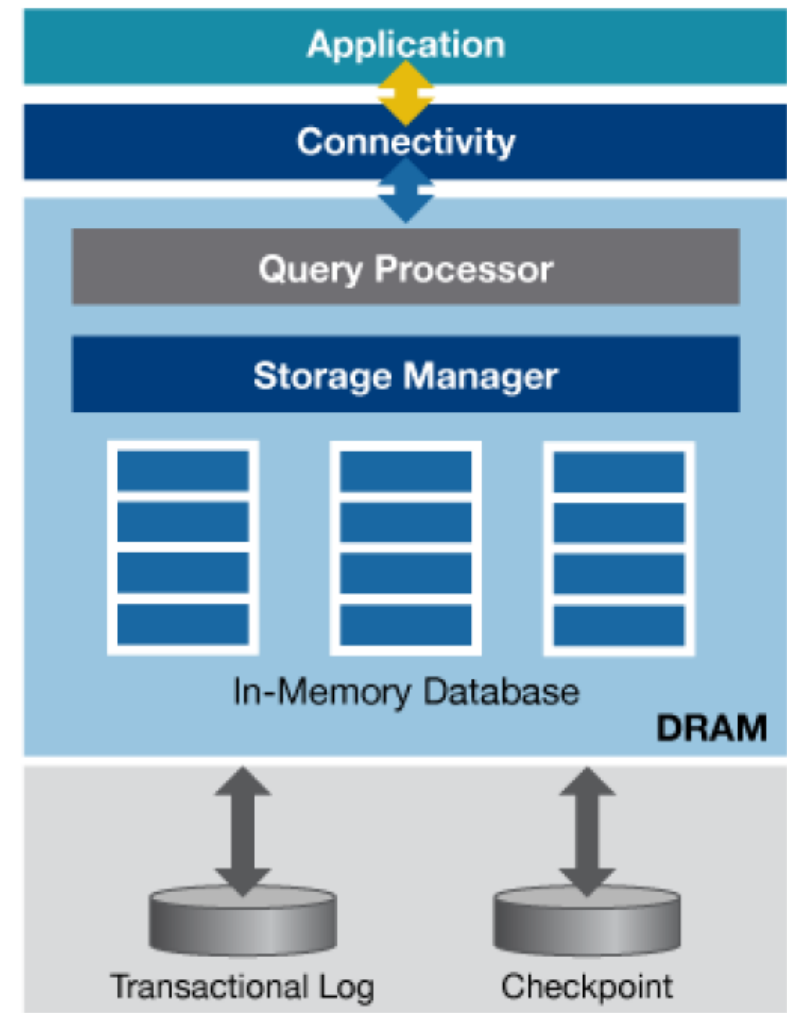
□ 인메모리 데이터베이스 특징

특징	설명
속도	매체 접근 속도가 우수하고 단순한 알고리즘으로 적은 CPU 인스트럭션 수행 디스크 검색보다 자료 접근이 훨씬 빠름
성능예측	메인 메모리에 저장된 데이터를 접근하는 것은 성능 예측이 가능하여 즉각적인 응답시간과 높은 처리량을 요구하는 실시간 환경 응용에 사용
비용절감	메모리 데이터의 접근이라는 전체를 가지고 만들어진 인덱싱 기법을 이용하여 인덱스 크기를 줄이며 관리비용 절감

인메모리 데이터베이스

2. 인메모리 데이터베이스 구성 및 구성요소

□ 인메모리 데이터베이스 구성



기술요소	설명
T-Tree 인덱스	B트리의 장점과 AVL Tree의 장점 결합 데이터 접근 계산 최소화
메모리 기반	데이터 접근 비용, 계산, 평가비용이 디스크 대비 낮음
Backup DB	메모리 내부 데이터를 디스크에 백업용 동기화

인메모리 데이터베이스

3. 디스크 기반 데이터베이스 비교 및 장단점, 활용분야

□ 디스크 기반 데이터베이스와 비교



인메모리 데이터베이스

구분	인 메모리 DB	디스크 기반 DB
데이터 저장처리	메인 메모리	디스크
운용목표	트랜잭션의 빠른 수행	데이터의 안정적 운영
동시성 제어	인덱스에 의한 동시성 제어	데이터 접근 트랜잭션 중심
처리속도	10~500 배 빠름(DB 연산시간)	1 배(DB 연산 + 데이터 전송)
Back up 매체	Disk	Disk
Size 제한	Physical Memory Size	하드디스크 Size
회복 기법	하드웨어적 회복 기법	Undo/Redo 로 로그 관리

인메모리 데이터베이스

□ 인메모리 데이터베이스 장단점

장점	단점
처리 속도: 상용 DB 의 3 ~ 10 배 속도 경량성: 간결한 DB 접근 알고리즘으로 라이브러리는 기존 DB 보다 훨씬 적음	영속성: 메모리상의 데이터를 backup 디스크로 저장한 후 Reloading 시간이 상당히 걸림 64bit 인 경우 메인 메모리 64~128GB 인 인 메모리는 최대 32 ~ 64GB 정도

감사합니다!



데이터베이스



9. 빅데이터와 NoSQL

KPC 기본반

9. 빅데이터와 NoSQL.....	142
빅데이터 (Big Data).....	143
NoSQL.....	147

빅데이터(Big Data)

1. 빅데이터(big Data)개요

□ 빅데이터 정의

- 기존 데이터베이스의 데이터 저장 및 관리와 분석의 영역을 넘어서는 대량의 정형 또는 비정형 데이터 집합
- 최근에는 대용량 데이터의 수집, 저장, 분석 및 체계화를 위한 기술을 포괄하는 용어로 사용

□ 빅데이터 확산 배경

특징	설명
스마트 기기 보급 확산 및 인터넷 기술 발전	- 정형, 비정형 데이터의 폭발적 증가 - 사용자 Log, 소셜 정보 기반의 분석 및 활용 요구 증가
기존 RDBMS의 관리 한계 봉착	- 정형화된 테이블 구조 기반으로 인한 비정형 데이터 관리 한계 - 데이터 일관성 외 제공 적시성 및 운영 효과성 요구 증가

빅데이터(Big Data)

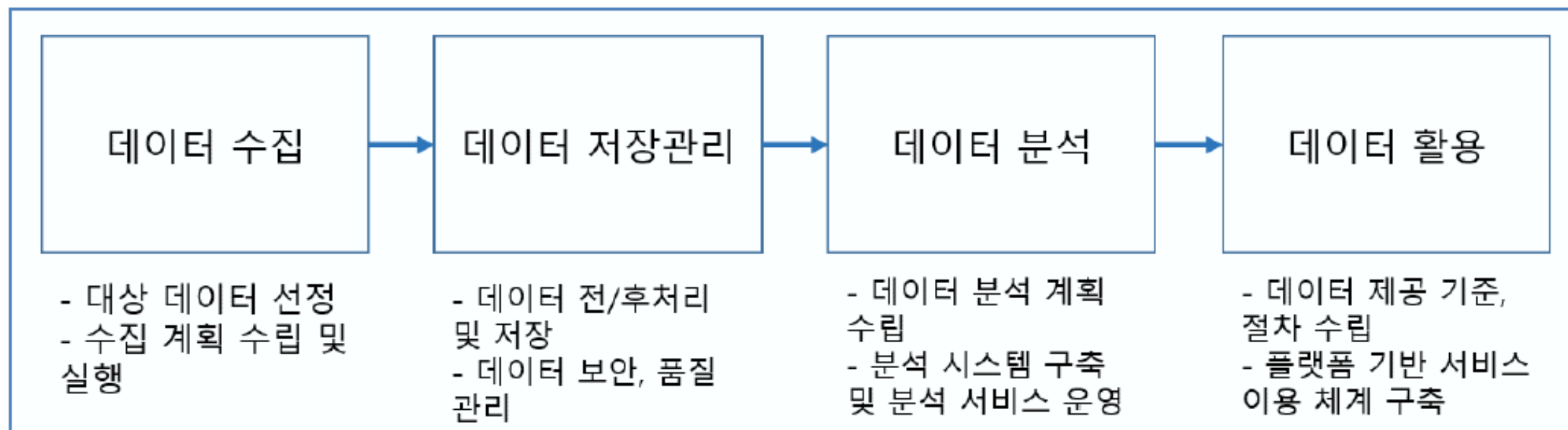
□ 구조적 데이터와 비구조적 데이터 차이점

구분	차이점	구조적 데이터	비구조적 데이터
데이터 특성 측면	존재 비중	15%	85 %
	데이터 형태	동일 타입 반복, 예측가능 형태	불규칙한 타입, 생성 시 변경
	데이터 값	숫자, 문자, 날짜	사용자 로그, 소셜 데이터
데이터 관리 측면	데이터 저장	Data Warehouse	Data Lake
	데이터 처리	OLAP, SQL	GFS, MapReduce
	DBMS	RDBMS	NoSQL

빅데이터(Big Data)

2. 빅데이터 활용을 위한 업무 절차 및 주요 기술

□ 빅데이터 업무 절차



빅데이터(Big Data)

□ 빅데이터 업무 절차별 주요 수행 내역

절차	세부 절차	수행 내역
데이터 수집	수집 대상 데이터 선정	- 분석 목적에 적합한 데이터 선정, 세부 목록 및 항목 작성 - 가능성, 보안, 정확성, 수집비용 고려
	수집 세부계획 수립 및 실행	- 데이터 유형, 수집 기술, 주기 등 세부 계획 작성 및 진행 - 정형/반정형/비정형 분류, Batch/실시간 수집 주기 결정
데이터 저장 관리	데이터 처리 및 저장	- 데이터 전/후 처리기법 선정 및 데이터 유형따른 DB 선정 - 필터링/변환/정제/통합/축소 처리, RDB/NoSQL 선정
	데이터 보안, 품질 관리	- 데이터 유출 방지, 개인정보보호 및 지침서 기반 품질관리 - 인증/접근제어/암호화/비식별화 조치, 일관성/정확성 보장

각 단계별로 기술요소에 대한 정리필요(오픈소스 중심으로)

빅데이터(Big Data)

데이터 분석	데이터 분석 계획 수립	- 분석 목적 정의, 시스템 환경 및 방법론 수립 - 분석 시나리오 작성, 분석 인프라 구축 방식 결정
	분석 시스템 구축 및 운영	- 빅데이터 수용 용량, 작업 부하 기반 HW/SW 인프라 구축 - Data Warehouse/Data Lake 구축, 분석 SW 도구 선정
데이터 활용	데이터 제공 기준, 절차 수립	- 데이터 활용을 위한 표준화된 체계 마련 - 정보 공개 및 보호 관련 법 참조, Open API 등 표준 적용
	분석 결과 이용 체계 구축	- 플랫폼 기반의 데이터 수집, 분석, 공유 기능 서비스 제공 - 서비스 유형, 이용 승인 및 데이터 분석 도구 제공

NoSQL(Not only SQL)

1. NoSQL(Not only SQL) 개요

□ NoSQL 정의

- 대용량 처리, 분산 환경 결함 수용 및 비구조적 데이터 처리 통한 기존 RDBMS 한계 보완, 빅데이터 환경의 신속하고 효율적 처리를 위한 데이터베이스 시스템 기술
- 기존 관계형 데이터베이스의 대체 기술이 아닌 빅데이터 처리를 위한 최적화 기술

□ 빅데이터 환경의 데이터베이스 요구 특성

요구 특성	설명
분산 처리 지원	결함 포용, 장애 회복, 지리적 분산 수용 필요
수평적 확장성	저비용, 범용 H/W를 통한 수평적 확장성(Scale-out) 제공 필요
향상된 가용성	모든 데이터가 반드시 강한 일관성, 트랜잭션을 필요로 하지는 않음
데이터 다양성	비구조적 데이터에 대한 처리 필요

- RDBMS의 Scale-up 기반의 확장 한계 및 Schema 기반 관리 어려움으로 인해 NoSQL 활용

NoSQL(Not only SQL)

2. CAP 이론

□ CAP 이론 정의

- 분산 시스템이 가져야 할 특성을 일관성, 가용성, 분할 내성으로 정의하고 구현 가능한 특성의 조합을 제시한 이론(Eric Brewer, Berkley)

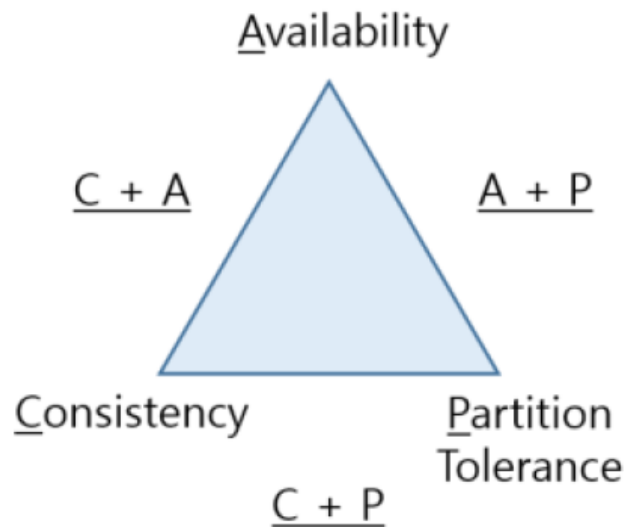
□ 분산 시스템이 지녀야 할 특성

특성	설명
일관성 (Consistency)	- 분산 노드 중 어느 노드로 접근하여도 같은 데이터 값 제공 - 같은 시간, 동일 데이터에 대해 같은 내용을 사용자에게 반환
가용성 (Availability)	- 다수 사용자가 본인이 원하는 시간에 읽기 및 쓰기 가능 - 몇몇 노드에 장애가 발생하더라도 다른 노드가 정상적 서비스 제공
분할 내성 (Partition Tolerance)	- Tolerance To Network Partition - 노드 간 메시지 통신 중 장애 발생 시에도 정상적 서비스 수행

- 세 가지 특성 중 두 가지 특성만 완벽하게 보장 가능하며 나머지 하나는 낮은 수준으로 제공

NoSQL(Not only SQL)

□ CAP 이론에 기반한 데이터베이스 유형



유형	보장 특성 및 Risk	사례
C + A	- 모든 노드가 일관성과 함께 장애 시에도 서비스 보장 - 네트워크 장애 시 시스템 중단 Risk	Oracle, MySQL
C + P	- 모든 노드가 함께 성능을 내며 일관성 보장 - 일부 데이터 활용 불가 Risk	BigTable, Hbase, Mongo DB
A + P	- 장애시에도 서비스 보장 - 일관성 미확보 데이터 조회 Risk	Dynamo, Cassandra

NoSQL(Not only SQL)

3. PACLEC 이론

□ 기존 CAP 이론의 문제점

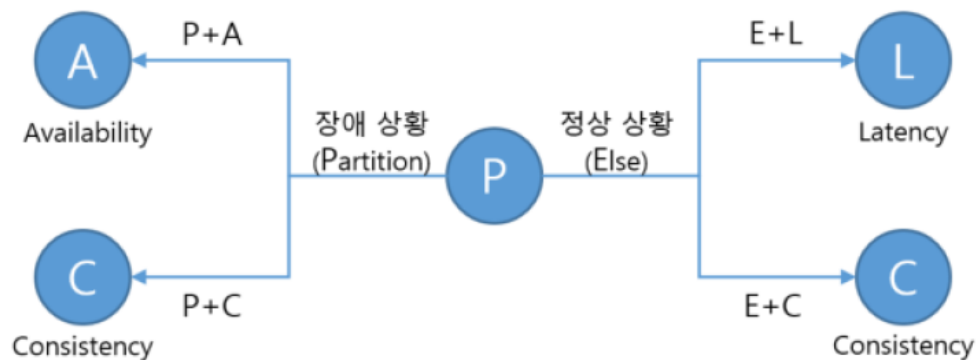
문제점
- 생존성(P) 선택을 배제한 C+A의 경우 NW 장애(메시지 손실)가 발생하지 않는 경우를 전제 → 분산 환경에서 존재할 수 없는 Case - 따라서 P를 전제로 C+P, A+P 조합만 가능 → 단, 이 경우 정상 상황 일때의 유형에 대해 설명하지 못하는 한계 발생
- PACLEC 이론(2013 년 예일대 Daniel Abadi)을 통해 정상 상황과 장애 상황으로 나누어 설명

□ PACELC 이론

개선 내용	이론 설명
정상 상황과 장애 상황일 경우를 나누어 설명	- 발생 상황을 네트워크 장애 발생(P)과 정상(E)으로 구분 - 장애 상황(P) 시 상충하는 가용성(A)과 일관성(C) 중 하나만 선택 - 정상 상황(E) 시 상충하는 지연(L)과 일관성(C) 중 하나만 선택

NoSQL(Not only SQL)

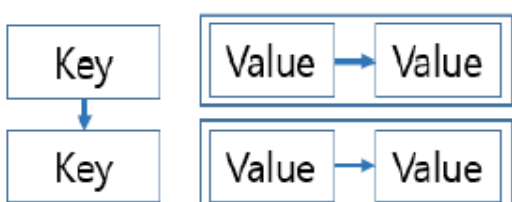
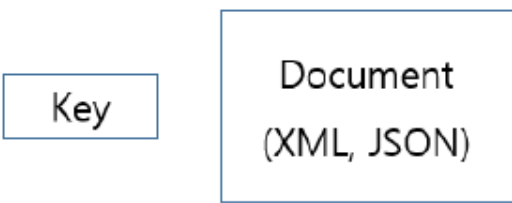
□ PACELC 이론에 기반한 NoSQL 유형



유형 구분		유형 특성	구현 시스템
장애상황	정상상황		
P+A	E+L	- 장애 상황 시 가용 노드만 반영, 복구 시 전체 반영 - 정상 상황 시 Latency 우선 고려	Cassandra DynamoDB
P+A	E+C	- 장애 상황 시 가용 노드만 반영, 복구 시 전체 반영 - 정상 상황 시 모든 노드 동일 메시지 보장	Hazelcast IMDG
P+C	E+L	- 장애 상황 시 Timeline Consistency 수준 보장 - 정상 상황 시 Latency 우선 고려	PNUTS
P+C	E+C	- 장애 상황 시 일관성 우선 보장 - 정상 상황 시 모든 노드 동일 메시지 보장	VoltDB HBase

NoSQL(Not only SQL)

□ NoSQL 데이터 모델

데이터 모델	개념도		모델 특성
키-값 모델 (Key-Value Model)			- 키-값 쌍 데이터 표현 - 단순구조 빠른연산, 비효율적 범위질의 - Dynamo, Redis
순서화된 키-값 모델 (Ordered Key-Value Model)			- 순서화된 키-값 쌍 데이터 표현 - 효율적 범위 질의, 값 모델링 미지원 - ArangoDB, MemcacheDB
열 지향 모델 (Column Oriented Model)			- 값(Value) 열(Column) 모델링 표현 - 값의 지속적 다차원 Map 구성 - HBase, Cassandra
문서 데이터베이스 모델 (Document Database Model)			- 값(Value) 부분에 문서 저장 - 임의적 스키마 기반 자유로운 속성 추가 - CouchDB, MongoDB

NoSQL(Not only SQL)

5. NoSQL 데이터 모델링

□ NoSQL 데이터 모델링 원칙

원칙	적용 방안	적용 효과
Denormalization	- 데이터 중복 허용 - 같은 데이터를 여러 문서, 테이블에 복제	질의처리 I/O 비용 감소
Aggregation	- 유연한 Schema 허용 - 엔티티 구조 다양화, 1:N 관계 최소화	- Join 연산 감소 - 테이블 구조 확장/변동성
Application Join	- Application의 데이터 사용 형태 최적화 - 데이터 간 관계, 속성(Join) 고려 최소화	Server-Side 연산 부하 감소

- 기존 RDBMS와는 달리 SQL 중심이 아닌 검색속도 및 처리 효율성 중심의 모델링 원칙 적용

NoSQL(Not only SQL)

□ NoSQL 데이터 모델링 절차

구분	수행내역	산출물
1. 도메인 모델 파악	- 도메인(시스템) 파악, 저장 대상 데이터 식별 - 데이터 개체, 개체 간 관계 분석 및 도식화	ERD(Entity-Relationship Diagram)
↓ 식별 데이터		
2. Query Design	- Application 쿼리 결과 값 선정 - 데이터 출력 형태 기반 테이블 구조 설계	데이터 출력 화면, 논리 테이블구조, Query
↓ 논리 설계		
3. 패턴기반 모델링	- Put(key, Value), Get(Key) 기반 테이블 재정의 - Denormalization, Aggregation 통한 I/O 효율화	Key-Value 테이블 구조
↓ 테이블 설계		
4. 최적화 필요기능 리스팅	- Application 및 데이터 특성 기반 상세/최적화 - Order Key, Secondary Index 등 모델 최적화	(Ordered)Key-Value, Key-Document 테이블
↓ 최적 모델		
5. NoSQL 테스트	- NoSQL 구조 및 특징 분석, 솔루션 선정 - 부하, 안정성, 확장성 테스트 검증	Cassandra, MongoDB, Redis, HBase
↓ 선정 솔루션		
6. 모델 최적화 및 H/W 설계	- 선정 NoSQL 기반 데이터 모델 최종 최적화 - Application I/F, 하드웨어 설계	I/F, 하드웨어 설계서

데이터 지향인 RDBMS와 Query 지향인 NoSQL의 설계 관점으로 인한 모델링 방식의 차이점 발생

NoSQL(Not only SQL)

□ NoSQL 모델링과 RDBMS 모델링 차이점 비교

비교 항목	NoSQL 모델링	RDBMS 모델링
모델링 절차	Query 설계 → 테이블 설계	테이블 설계 → Query 설계
주요 활용 기법	비정규화(Denormalization)	정규화(Normalization)
설계 중점 사항	Query 효율성	데이터 일관성
고려 사항	Join, Sorting, Grouping 등 기능 제약	I/O 비용
활용 사례	통계 화면, 로그 모니터링, Cache	계정 관리, 과금, 금융 이체

NoSQL(Not only SQL)

6. NoSQL과 RDBMS활용한 통합 시스템 구축 방안 사례

통합 시스템 구성도	구축 방안	
	역할	활용
<pre> graph LR WA((Web/App)) <--> WWS((WEB/WAS 서버)) WWS --> RDBMS((RDBMS 계정, 과금 관리)) WWS --> REDIS((REDIS Cache)) WWS --> MongoDB((Mongo DB 로그, 통계 관리)) </pre>	중요, 민감 데이터 관리	- RDBMS(Oracle 등) 활용 - 계정, 결제/과금 정보 관리 - 중요 Data 일관성 보장
	Cache 성격 데이터 관리	- Redis 활용 - Cache 정보 인메모리 관리 - Latency 향상
	대용량/비정형 데이터 관리	- MongoDB 활용 - 로그, 통계 정보 관리 - 다양한 Value 수용

- NoSQL은 RDBMS의 대체 시스템이 아닌 빅데이터 경쟁력 및 운영 효과성 향상이 목적임. 따라서 각 DBMS의 장점을 활용한 실무적 통합 고려 필요

감사합니다!